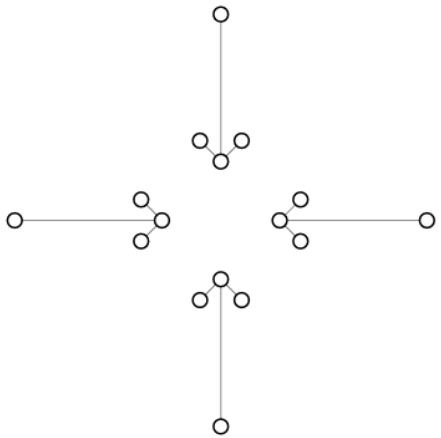
Hợp tác – Đồng bộ hoá
(Coordination - Synchronization)

Distributed Systems - Coordination

Overview



- Trong các chương trước, chúng ta đã tìm hiểu về các tiến trình và cách giao tiếp giữa các tiến trình. Tuy nhiên, giao tiếp không phải là toàn bộ câu chuyện. Một khía cạnh quan trọng khác là cách các tiến trình hợp tác và đồng bộ hóa với nhau.

- Trong chương này, chúng ta chủ yếu tập trung vào cách các tiến trình đồng bộ hóa và phối hợp hành động với nhau.

Distributed Systems - Coordination

Overview

- Ví dụ về đồng bộ hoá như: đảm bảo nhiều tiến trình không truy cập đồng thời vào 1 tài nguyên dùng chung, đồng bộ thứ tự sự kiện giữa các tiến trình,...

- Phối hợp (coordination) và Đồng bộ (synchronization) có sự liên quan chặt chẽ. Theo góc nhìn trong **[Malone và Crowston, 1994]: "**Khi nói đến phối hợp (coordination), mục tiêu là quản lý sự tương tác và sự phụ thuộc giữa các hoạt động trong một hệ thống phân tán" thì Phối hợp bao gồm cả Đồng bộ hoá.

- **Phối hợp** trong hệ thống phân tán thường khó khăn hơn nhiều khi so với **Phối hợp** trong hệ thông đơn xử lý hoặc đa xử lý. Các vấn đề và giải pháp được thảo luận trong chương này, theo bản chất của chúng, khá tổng quát và xuất hiện trong nhiều tình huống khác nhau trong các hệ thống phân tán.

Distributed Systems - Coordination

Main content

- Đồng bộ hoá theo thời gian thực.
- Đồng bộ hoá theo thứ tự tương đối của sự kiện.

Clock synchronization

Logical clocks

Mutual exclusion

Election algorithm

Gossip-based coordination

Distributed event matching

Location systems

Distributed Systems - Coordination

Main content

- Các thuật toán điều phối loại trừ lẫn nhau để truy cập tài nguyên chung.
- Thuật toán bầu chọn: Chọn một tiến trình làm điều phối viên.

Clock synchronization

Logical clocks

Mutual exclusion

Election algorithm

Gossip-based coordination

Distributed event matching

Location systems

Distributed Systems - Coordination

Main content

- Ba vấn đề điều phối về truyền tin dạng gossip
  1. Tổng hợp mẫu dữ liệu
  2 .Lấy mẫu peer
  3 .Xây dựng overlay network

- Clock synchronization
- Logical clocks
- Mutual exclusion
- Election algorithm
- **Gossip-based coordination**
- Distributed event matching
- Location systems

Distributed Systems - Coordination

Main content

- Thảo luận về cách ăn khớp giữa Subcription – Publication trong hệ thống Subcribe – Publish.

Clock synchronization

Logical clocks

Mutual exclusion

Election algorithm

Gossip-based coordination

Distributed event matching

Location systems

Distributed Systems - Coordination

Main content

- Hệ thống định vị: Định vị một tiến trình trong không gian nhiều chiều, giúp ích khi xử lý hệ thống phân tán quy mô lớn.

Clock synchronization

Logical clocks

Mutual exclusion

Election algorithm

Gossip-based coordination

Distributed event matching

Location systems

1) Clock synchronization

*Đôi điều về thời gian trong hệ thống tập trung:

Trong một hệ thống tập trung, tiến trình muốn biết thời gian, nó chỉ cần gọi đến hệ điều hành
Nếu tiến trình A yêu cầu giá trị về thời gian, sau đó đến tiến trình B yêu cầu thì giá trị thời gian của B sẽ lớn hơn (hoặc bằng) giá trị thời gian của A.

*Một ví dụ đơn giản về thời gian trong hệ thống Unix:

Chương trình trong Unix được chia thành nhiều tệp nguồn nhỏ.
Một tệp thay đổi, chỉ cần biên dịch lại tệp đó mà không cần dịch lại toàn bộ tệp của chương trình.
→Tăng tốc độ làm việc của lập trình viên

Chương trình chia thành n tệp

1	...	k	k+1	k+2	k+3	k+4	k+5	...	n

Clock synchronization
Logical clocks
Mutual exclusion
Election algorithm
Gossip-based coordination
Distributed event matching
Location systems

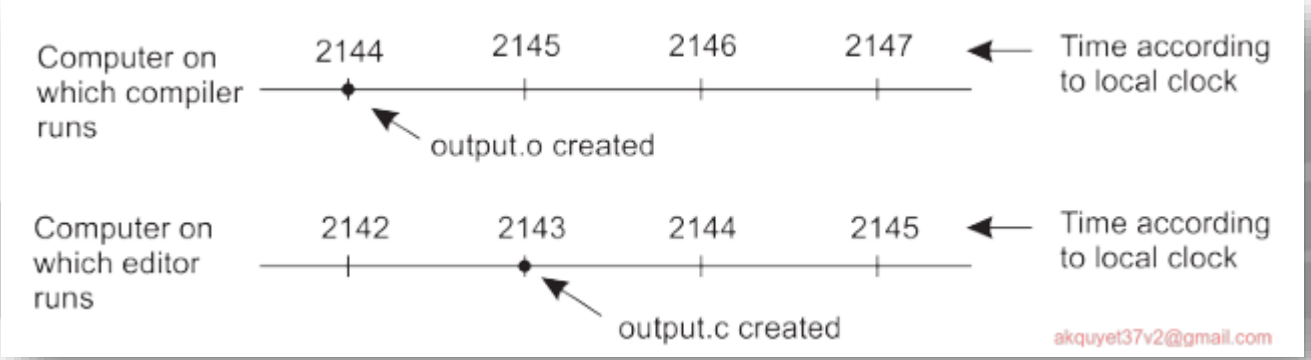
1) Clock synchronization

*Một ví dụ đơn giản về thời gian trong hệ thống Unix:

Khi biên dịch một chương trình, “make” trong Linux sẽ tiến hành kiểm tra thời gian sửa đổi cuối cùng của các tệp, bao gồm: tệp mã nguồn, tệp đối tượng (tệp được dịch ra từ tệp mã nguồn tương ứng).

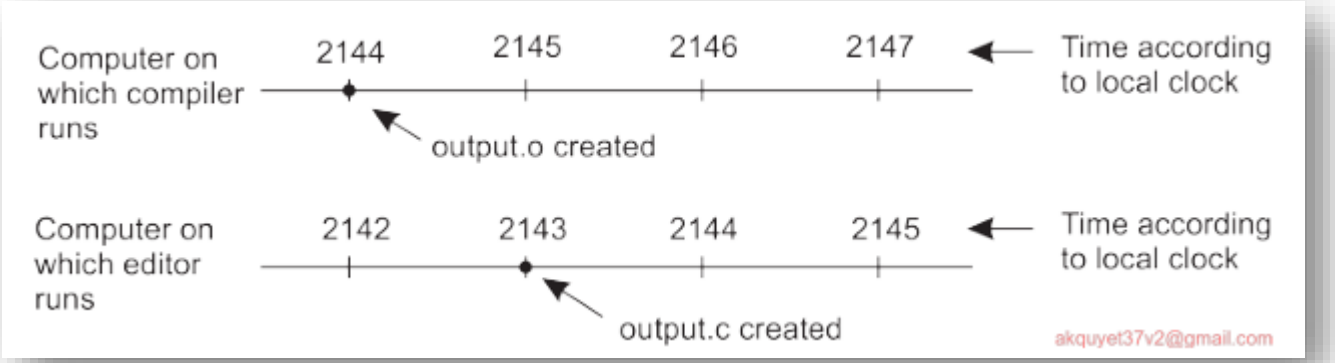
Nếu tệp mã nguồn có thời gian lớn hơn tệp đối tượng thì tức là tệp mã nguồn đã có sự thay đổi, cần phải biên dịch lại tệp mã nguồn này.
Ngược lại nếu tệp mã nguồn có thời gian nhỏ hơn thời gian của tệp đối tượng thì không cần phải biên dịch tệp mã nguồn nữa.

→Điều gì xảy ra nếu thời gian sửa đổi tệp mã nguồn và tệp đối tượng không đúng??



1) Clock synchronization

*Một ví dụ đơn giản về thời gian trong hệ thống Unix:



Khi chạy chương trình, lỗi xuất hiện và lập trình viên sẽ fix bug đến phát điên ☺
Lý do gây nên lỗi: **thời gian ở bên compiler và bên editor không đồng nhất.**
Giây thứ 2143 về giá trị thì nhỏ hơn giây thứ 2144. Nhưng thực tế, có thể thấy rõ rằng giây thứ 2143 bên editor tương ứng với giây thứ 2145 bên compiler.

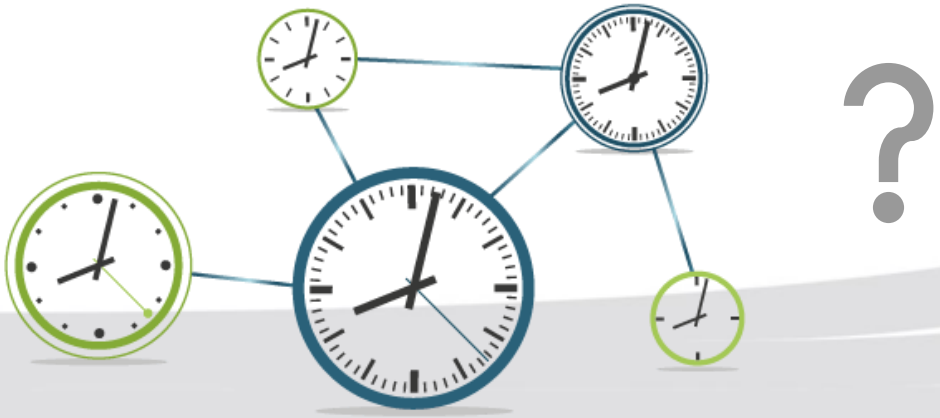
Clock synchronization
Logical clocks
Mutual exclusion
Election algorithm
Gossip-based coordination
Distributed event matching
Location systems

1) Clock synchronization

*Thời gian chính xác là rất quan trọng

Còn nhiều ví dụ chứng minh sự quan trọng của thời gian trong các hệ thống máy tính khác.

Vì thời gian là một yếu tố cơ bản trong cách con người suy nghĩ và ảnh hưởng của việc không đồng bộ hóa tất cả các đồng hồ có thể rất nghiêm trọng
→ Quá hợp lý khi chúng ta bắt đầu nghiên cứu về **đồng bộ hóa** với câu hỏi đơn giản sau: Liệu có thể đồng bộ hóa tất cả các đồng hồ trong một hệ thống phân tán không? Câu trả lời hóa ra lại rất phức tạp.



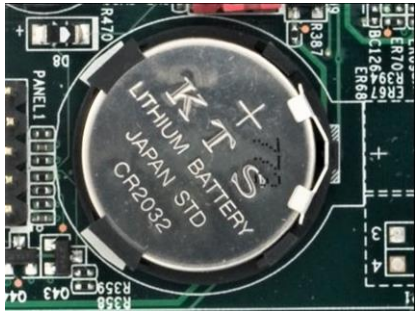
?

Clock synchronization
Logical clocks
Mutual exclusion
Election algorithm
Gossip-based coordination
Distributed event matching
Location systems

1) Clock synchronization - Physical clocks

* ”Đồng hồ” trong máy tính

“Đồng hồ” của máy tính còn được gọi là: Timer – bộ đếm thời gian.
Cấu tạo vật lý của Timer và cách hoạt động của Timer.
CMOS và pin dự phòng.



* Máy tính duy nhất + Đồng hồ duy nhất

Ở trường hợp này, việc đồng hồ chạy chệch một chút cũng không quá quan trọng. Vì tất cả các tiến trình trên máy tính đều sử dụng cùng một đồng hồ, chúng vẫn sẽ nhất quán nội bộ.
Ví dụ, nếu tệp input.c có thời gian 2151 và tệp input.o có thời gian 2150, make sẽ biên dịch lại tệp nguồn, ngay cả khi đồng hồ thực tế bị lệch 2 đơn vị và thời gian thực sự là 2153 và 2152. Điều thực sự quan trọng ở đây chỉ là thời gian tương đối.

Clock synchronization
Logical clocks
Mutual exclusion
Election algorithm
Gossip-based coordination
Distributed event matching
Location systems

1) Clock synchronization - Physical clocks

* Nếu có nhiều CPU, mỗi CPU có riêng một đồng hồ ?

Các đồng hồ riêng có thể được đồng bộ đều với nhau. Nhưng chúng sẽ lệch dần theo thời gian do cấu tạo vật lý đặc thù.
Sự khác biệt đó (độ lệch) được gọi là clock skew.

→ Các chương trình yêu cầu thời gian liên quan đến tệp, đối tượng, tiến trình hoặc thông điệp phải chính xác và không phụ thuộc vào máy tính nào đã tạo ra chúng (tức là đồng hồ nào được sử dụng)

* UTC – Coordinated Universal Time

Một số hệ thống cần có thêm các đồng hồ ngoài. Nhưng việc này sinh ra 2 vấn đề:

- Làm thế nào để đồng bộ hóa chúng với đồng hồ thực tế trong thế giới thực?
- Làm thế nào để đồng bộ hóa các đồng hồ với nhau?

→UTC ra đời. UTC là cơ sở của tất cả các hệ thống đo thời gian dân dụng hiện đại và là một tiêu chuẩn toàn cầu.

1) Clock synchronization - Algorithms

* Thông tin liên quan

Các máy tính hiện nay được lắp các bộ thu UTC để thu tín hiệu UTC từ hơn 40 trạm phát mặt đất, thậm chí là các vệ tinh.

The LORADD-UTC receiver provides stable frequency outputs and a Loran derived 1 PPS output synchronised to Coordinated Universal Time (UTC). To provide the best possible timing stability, the LORADD-UTC is equipped with an ovenised crystal oscillator, which allows longer integration times.

- Nếu một máy chứa bộ thu UTC thì mục tiêu là đồng bộ tất cả các máy khác với máy có bộ thu.
- Nếu không có máy nào có bộ thu UTC, mỗi máy sẽ tự theo dõi thời gian của chính nó, và mục tiêu là giữ cho tất cả các máy đồng bộ với nhau càng tốt.

Nhiều thuật toán đã được đề xuất để thực hiện đồng bộ hóa này. Các nghiên cứu khảo sát đã được thực hiện bởi Ramanathan et al. (1990), Horauer (2004), Shin et al. (2011), và Levesque và Tipper (2016).

1) Clock synchronization - Algorithms

* Thông tin liên quan

Độ lệch thời gian giữa hai đồng hồ (của hai máy tính) trong 1 hệ thống được giới hạn nhỏ hơn bằng **độ chính xác π.**
Độ lệch thời gian giữa một đồng hồ (trong một hệ thống) với đồng hồ UTC giới hạn nhỏ hơn hoặc bằng **độ chính xác tuyệt đối α.**
Ý tưởng của đồng bộ hoá là: giữ cho các đồng hồ **chính xác nội bộ** hoặc **chính xác theo thời gian thực**.

Các đồng hồ đồng bộ chính xác với thời gian thực → chính xác nội bộ
Các đồng hồ chính xác nội bộ → ko thể chắc chắc là chính xác với thời gian thực.

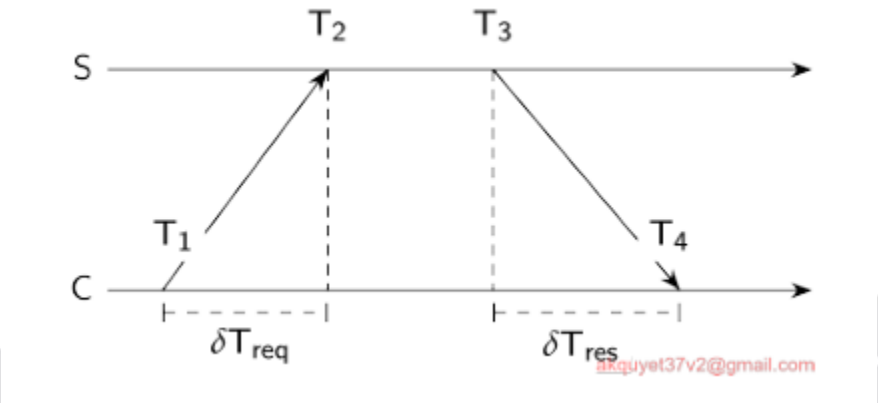
Điều kiện hoàn hảo: Các đồng hồ đều đạt được **độ chính xác tuyệt đối α** bằng 0.
Tuy nhiên, các đồng hồ này là các đồng hồ vật lý, do cấu tạo đặc thù và điều kiện môi trường hoạt động khác nhau mà chúng sẽ có **tốc độ trôi ρ** khác nhau.
Vì vậy mà các nhà thiết kế hệ thống sẽ tìm cách giữ các đồng hồ này không lệch nhau quá **π** → đồng bộ lại ít nhất mỗi π/(2ρ) giây.
→ Các thuật toán khác nhau sẽ có các cách tiếp cận khác nhau để thực hiện quá trình đồng bộ hóa này.

1) Clock synchronization - Algorithms

* Network Time Protocol

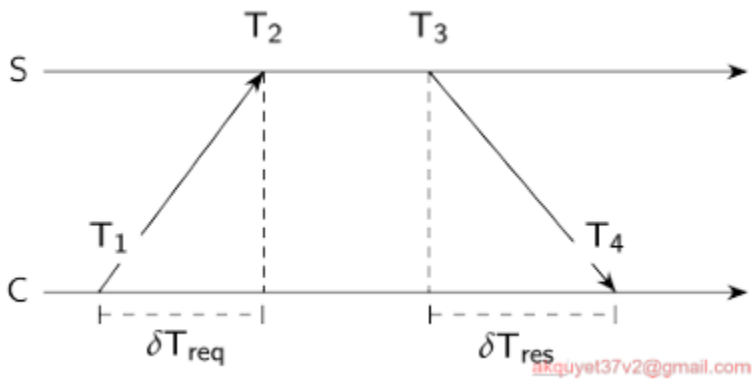
Một phương pháp phổ biến trong nhiều giao thức đồng bộ thời gian, được đề xuất lần đầu tiên bởi Cristian [1989], là để các client liên hệ với một **timeserver.** Máy chủ này có thể cung cấp thời gian chính xác vì nó được trang bị một bộ thu UTC hoặc một đồng hồ chính xác.

→Vấn đề nảy sinh khi mạng có độ trễ, làm cho thời gian mà server cung cấp bị lỗi thời khi đến tay client.



1) Clock synchronization - Algorithms

* Network Time Protocol



Giả sử thời gian truyền từ A đến B và từ B về A là xấp xỉ nhau (δTreq ≈ δTres), ta có thể ước lượng **độ lệch thời gian (offset) θ** giữa hai đồng hồ như sau:

$$\theta = (T2 - T1) + (T3 - T4) / 2$$

Nếu θ<0, nghĩa là đồng hồ của A chạy nhanh hơn B. Trong lý thuyết, A có thể lùi thời gian lại, nhưng điều này không được phép vì có thể gây lỗi nghiêm trọng, chẳng hạn như một tệp tin được biên dịch sau khi thay đổi thời gian có thể có timestamp cũ hơn mã nguồn.

1) Clock synchronization - Algorithms

* Network Time Protocol

Thay vì đặt lại thời gian ngay lập tức, hệ thống thực hiện điều chỉnh dần dần bằng cách thay đổi tốc độ cập nhật đồng hồ:

- Nếu cần làm chậm đồng hồ, mỗi lần ngắt thời gian chỉ tăng 9 msec thay vì 10 msec.
- Nếu cần tăng tốc đồng hồ, mỗi lần ngắt sẽ cộng 11 msec thay vì 10 msec.

* Cơ chế hoạt động của Network Time Protocol

Trong NTP, quá trình đồng bộ hóa được thực hiện theo cặp máy chủ. Máy B cũng có thể kiểm tra thời gian từ máy A để tính toán **độ lệch θ và độ trễ δ** như sau:

$$\delta = (T4 - T1) - (T3 - T2) / 2$$

NTP lưu trữ 8 cặp giá trị (θ, δ), sau đó chọn giá trị **δ nhỏ nhất** làm ước lượng tốt nhất về độ trễ và **lấy giá trị θ tương ứng làm độ lệch thời gian chính xác nhất.**

1) Clock synchronization - Algorithms

* Cơ chế hoạt động của Network Time Protocol

Trong NTP, quá trình đồng bộ hóa được thực hiện theo cặp máy chủ. Máy B cũng có thể kiểm tra thời gian từ máy A để tính toán **độ lệch θ và độ trễ δ** như sau:

$$\delta = (T4 - T1) - (T3 - T2) / 2$$

NTP lưu trữ 8 cặp giá trị (θ, δ), sau đó chọn giá trị **δ nhỏ nhất** làm ước lượng tốt nhất về độ trễ và **lấy giá trị θ tương ứng làm độ lệch thời gian chính xác nhất.**

* Phân tầng trong Network Time Protocol

Vì không phải tất cả các máy chủ đều có độ chính xác như nhau, NTP sử dụng hệ thống phân tầng:

Stratum-0: Đồng hồ tham chiếu (UTC receiver, đồng hồ nguyên tử).

Stratum-1: Máy chủ kết nối trực tiếp với stratum-0.

Stratum-2, 3, …

Máy chủ đồng bộ với cấp trên của nó.

1) Clock synchronization - Algorithms

*Đồng bộ hóa thời gian trong mạng không dây: Giao thức RBS

Ở hệ thống phân tán truyền thống, việc triển khai máy chủ thời gian (timeserver) là đơn giản và hiệu quả vì hầu hết các máy có thể liên lạc trực tiếp với nhau.

Ở hệ thống không dây, đặc biệt là mạng cảm biến, điều đó không còn đúng 100% !!! Do các yếu tố:

- Tài nguyên hạn chế: Các nút trong mạng cảm biến có khả năng xử lý và bộ nhớ hạn chế.
- Định tuyến nhiều bước (multihop) đắt đỏ: Các thông điệp phải truyền qua nhiều nút trung gian, làm tăng chi phí năng lượng và thời gian.
- Tối ưu hóa năng lượng: Việc đồng bộ hóa đồng hồ cần phải được thiết kế sao cho tiết kiệm năng lượng nhất có thể.

→Thuật toán NTP không còn hợp lý, RBS là một thuật toán hợp lý hơn.

1) Clock synchronization - Algorithms

*Nguyên lý hoạt động của RBS

Ở RBS, **không yêu cầu** có một nút chính xác tuyệt đối làm máy chủ thời gian.
Thay vì cố gắng đồng bộ tất cả các nút theo giờ UTC, RBS chỉ tập trung vào việc **đồng bộ tương đối** giữa các nút trong mạng.

Cách thức hoạt động của RBS:

1. Một nút phát tín hiệu tham chiếu (reference message), được xem là tín hiệu chuẩn để đồng bộ hóa.
2. Mọi nút nhận đều ghi lại thời gian nhận được tín hiệu theo đồng hồ cục bộ của mình.
3. Các nút chia sẻ thời điểm nhận tín hiệu với nhau, từ đó tính toán độ lệch thời gian (offset) giữa chúng

Ưu điểm của RBS so với NTP:

- Không cần tạo dấu thời gian trước khi truyền tin, giúp loại bỏ độ trễ không xác định do xử lý thông điệp.
- Không bị ảnh hưởng bởi độ trễ ngẫu nhiên của mạng không dây, vì chỉ xem xét thời gian nhận tại các nút.
- Tập trung vào đồng bộ hóa tương đối giữa các nút, không cần thiết bị có độ chính xác cao như máy chủ thời gian trong NTP.

1) Clock synchronization - Algorithms

*Nguyên lý hoạt động của RBS

Tính toán độ lệch thời gian trong RBS:
Mỗi nút p và q khi nhận được một tín hiệu tham chiếu thứ k sẽ ghi lại thời gian Tp,k và Tq,k . Sau khi trao đổi thông tin, chúng có thể tính độ lệch thời gian trung bình như sau:

$$\text{Offset[p,q]}=\frac{1}{M}\sum_{k=1}^{M}(T_{p,k}-T_{q,k})$$

Với M là tổng số tín hiệu tham chiếu đã được gửi. Điều này giúp nút p biết được giá trị đồng hồ của nút q so với đồng hồ của nó.

1) Clock synchronization - Algorithms

*Nguyên lý hoạt động của RBS

Xử lý trôi đồng hồ (Clock Drift Compensation):
Vấn đề lớn của việc đồng bộ hóa đồng hồ là đồng hồ có thể trôi dần theo thời gian, dẫn đến sai số ngày càng lớn. Nếu chỉ lấy giá trị trung bình của độ lệch thời gian, các phép đo cũ có thể làm giảm độ chính xác.
→Giải pháp:Thay vì tính trung bình, RBS sử dụng hồi quy tuyến tính (linear regression) để tính toán mối quan hệ giữa thời gian nhận tín hiệu của hai nút dưới dạng phương trình:

$$\text{Offset}[p,q](t) = \alpha t + \beta$$

Trong đó:

- α biểu diễn tốc độ trôi của đồng hồ.
- β là độ lệch thời gian ban đầu.

Các giá trị α và β được tính toán dựa trên các cặp giá trị thời gian nhận tín hiệu (Tp,k,Tq,k). Nhờ mô hình này, một nút có thể ước lượng chính xác hơn đồng hồ của nút khác tại bất kỳ thời điểm nào, ngay cả khi đồng hồ tiếp tục trôi dạt theo thời gian.

Distributed Systems - Coordination

Overview

- Trong hệ thống phân tán, việc đảm bảo các tiến trình trên các node khác nhau có thể sắp xếp sự kiện theo đúng thứ tự là rất quan trọng.

- Tuy nhiên, không cần thiết phải đồng bộ hóa đồng hồ thời gian thực trên toàn hệ thống. Thay vào đó, các node chỉ cần thống nhất về thứ tự xảy ra của các sự kiện để đảm bảo tính chính xác.

- Logical clocks đóng vai trò quan trọng trong việc xác định quan hệ thứ tự giữa các sự kiện mà không cần đồng bộ hóa thời gian thực. Điều này đặc biệt hữu ích trong nhiều ứng dụng như hệ thống build (make), cơ sở dữ liệu phân tán, blockchain và các giao thức đồng thuận.

Distributed Systems - Coordination

1) Lamport's logical clocks

- Năm 1978, nhà khoa học máy tính Leslie Lamport đã giới thiệu quan hệ happens-before (xảy ra trước) để mô tả thứ tự xảy ra của các sự kiện trong hệ thống phân tán.
- Theo quan hệ này, nếu $a \rightarrow b$ ("sự kiện a xảy ra trước sự kiện b"), thì tất cả các tiến trình trong hệ thống đều đồng ý rằng sự kiện a diễn ra trước b.

	Đúng	**Sai**
Các trường hợp đúng/sai của quan hệ happens-before	+) a và b là các sự kiện trong cùng một tiến trình và a xảy ra trước b thì ta có a -> b. +) a là sự kiện của tiến trình gửi tin nhắn và b là sự kiện của tiến trình khác nhận tin nhắn đó thì a -> b cũng vẫn đúng.	+) 2 sự kiện a và b xảy ra trong các tiến trình khác nhau mà không trao đổi thông tin (kể cả gián tiếp qua bên thứ ba) thì không thể xác định được quan hệ giữa chúng. -> Cả a -> b và b -> a đều không đúng. -> Đây là các sự kiện đồng thời (concurrent events), không thể (không cần thiết) xác định sự kiện xảy ra trước sau.

1) Lamport’s logical clocks

- Quan hệ happens-before có tính bắc cầu, nếu a -> b và b -> c thì ta có a -> c

- Lamport nghĩ ra cách đo "thời gian logic“:

  - Với mỗi sự kiện a, có thể gán cho nó một giá trị thời gian C(a) mà tất cả các tiến trình đều đồng ý.

  - Các giá trị thời gian này phải đảm bảo rằng nếu a -> b thì C(a) < C(b)

  - Đồng hồ C phải luôn tiến về phía trước (tăng dần), không bao giờ lùi lại. Khi cần điều chỉnh thời gian, chỉ có thể cộng thêm một giá trị dương, không được phép trừ đi.

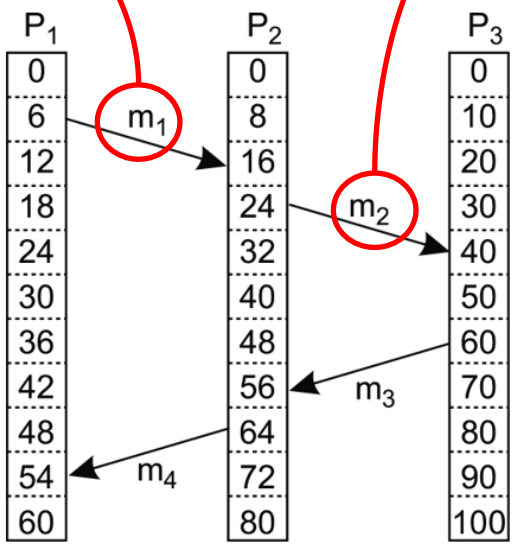
1) Lamport's logical clocks

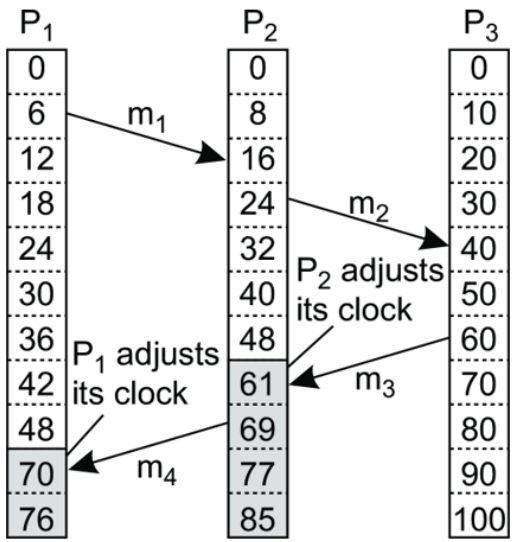
- Minh họa thuật toán Lamport

- Giả sử có 3 tiến trình chạy trên 3 máy khác nhau, mỗi máy có đồng hồ logic riêng. Đồng hồ của tiến trình P1 tăng mỗi lần 6 đơn vị, đồng hồ của tiến trình P2 tăng mỗi lần 8 đơn vị và đồng hồ của tiến trình P3 tăng mỗi lần 10 đơn vị.

m1 được đính kèm mốc thời gian gửi của P1 là 6
-> P2 có thể kết luận mất 10 đơn vị thời gian để tiến hành việc gửi (16 – 6 = 10)

Tương tự, tin nhắn m2 từ P2 đến P3 mất 16 đơn vị thời gian





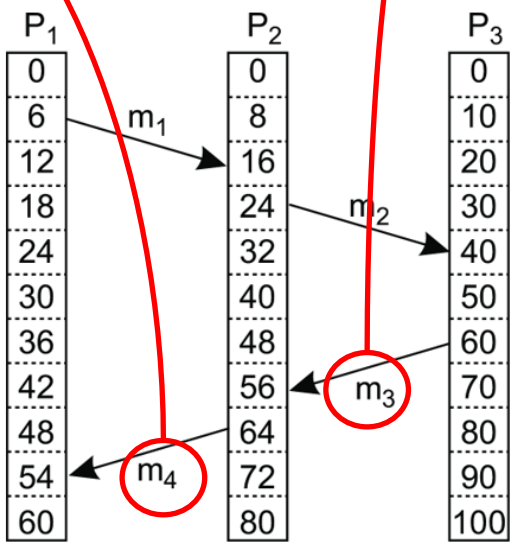
1) Lamport's logical clocks

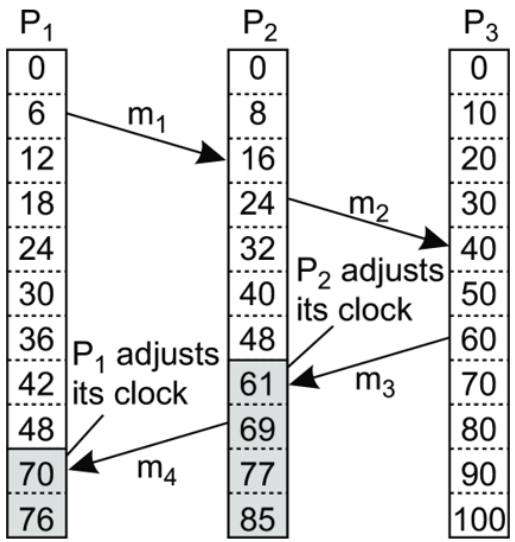
- Minh họa thuật toán Lamport

- Giả sử có 3 tiến trình chạy trên 3 máy khác nhau, mỗi máy có đồng hồ logic riêng. Đồng hồ của tiến trình P1 tăng mỗi lần 6 đơn vị, đồng hồ của tiến trình P2 tăng mỗi lần 8 đơn vị và đồng hồ của tiến trình P3 tăng mỗi lần 10 đơn vị.

Tin nhắn m4 được gửi từ P2 đến P1 ở mốc thời gian 54 -> Không hợp lý

Tin nhắn m3 từ P3 tại mốc 60 đến P2 tại mốc 56 -> mất 54 – 60 = - 4 đơn vị thời gian, không hợp lý





➡ **Đây chính là trường hợp cần phải ngăn chặn và chỉnh sửa!**

1) Lamport's logical clocks

- Minh họa thuật toán Lamport

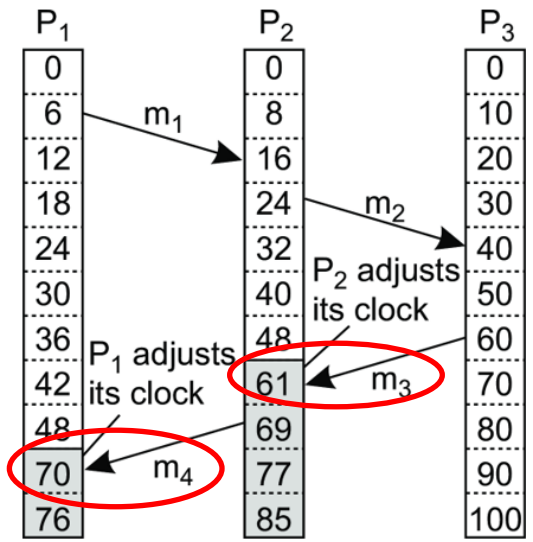
- Giả sử có 3 tiến trình chạy trên 3 máy khác nhau, mỗi máy có đồng hồ logic riêng. Đồng hồ của tiến trình P1 tăng mỗi lần 6 đơn vị, đồng hồ của tiến trình P2 tăng mỗi lần 8 đơn vị và đồng hồ của tiến trình P3 tăng mỗi lần 10 đơn vị.

- Phương pháp Lamport theo dõi trực tiếp quan hệ happens-before:

+ Khi tin m3 được gửi tại thời điểm 60, theo logic, nó sẽ được nhận ở mốc thời gian 61 hoặc muộn hơn.

➡ Giải pháp: Mỗi tin nhắn đều được gắn kèm thời gian gửi, dựa trên đồng hồ của máy gửi.

+ Khi tin nhắn đến đích, nếu đồng hồ của máy nhận hiển thị thời gian sớm hơn thời gian gửi trong tin nhắn, máy nhận sẽ điều chỉnh đồng hồ của mình chậm hơn mốc thời gian gửi một đơn vị.



1) Lamport's logical clocks

- Tổng quan thuật toán của Lamport

- Để thực hiện đồng hồ logic Lamport, mỗi tiến trình $P(i)$ duy trì một bộ đếm $C(i)$, được cập nhật theo các bước sau:

+ B1: Trước khi thực hiện một sự kiện bất kỳ (gửi tin nhắn qua mạng, chuyển tiếp tin nhắn đến ứng dụng hoặc một sự kiện nội bộ), tiến trình $P(i)$ tăng bộ đếm của mình:

$$C(i) = C(i) + 1$$

+ B2: Khi $P(i)$ gửi tin nhắn m đến tiến trình $P(j)$, nó gán mốc thời gian cho m là $ts(m)$, $ts(m)$ có giá trị bằng $C(i)$ sau khi cập nhật ở B1.

+ B3: Khi $P(j)$ nhận được tin nhắn m, sau khi tự tăng bộ đếm của mình như ở B1, nó điều chỉnh giá trị bộ đếm thành:

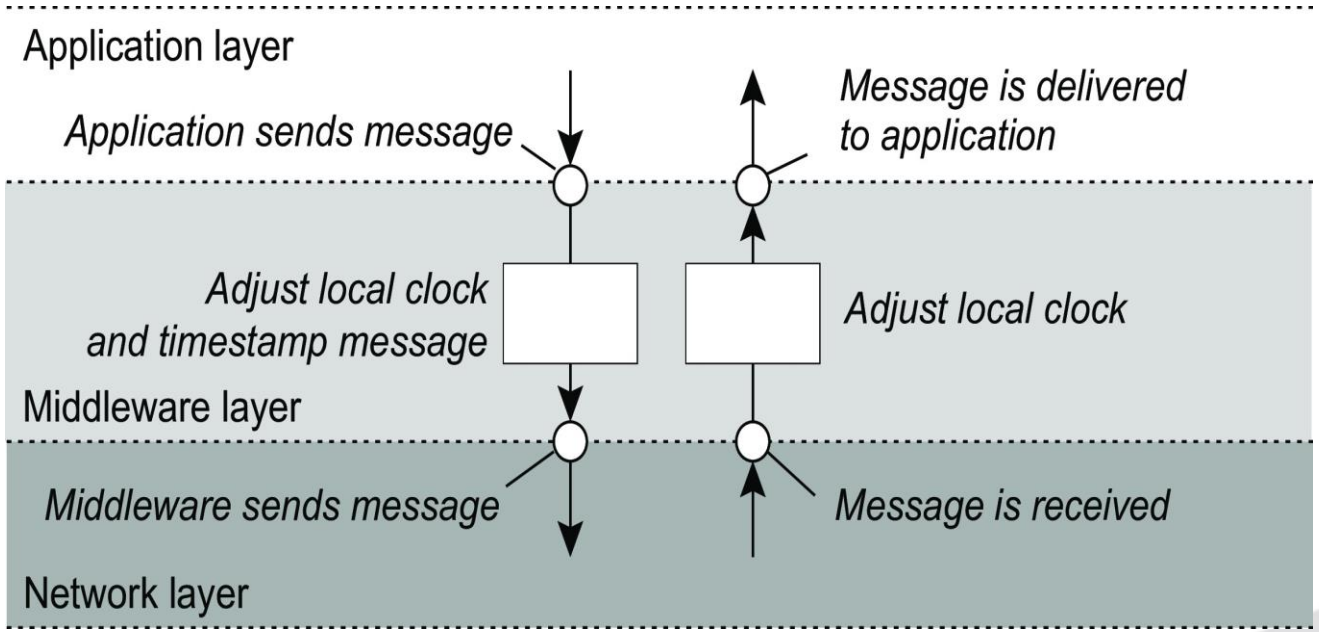
$$C(j) = \max\{C(j), ts(m)\}$$

***Chú ý:** Trong 1 số tình huống, để đảm bảo không có 2 sự kiện xảy ra tại cùng một thời điểm, cần sử dụng mã định danh cho mỗi tiến trình và mỗi tiến trình được biểu diễn thời gian bằng tuple thay cho bộ đếm.

VD: một sự kiện xảy ra tại thời điểm 40 của tiến trình $P(i)$ sẽ được gán nhãn thời gian (timestamp) là $\langle 40,i \rangle$. Nếu một sự kiện khác có nhãn $\langle 40,j \rangle$ với $i < j$, thì quy ước thứ tự: $\langle 40,i \rangle < \langle 40,j \rangle$

1) Lamport's logical clocks

- Vị trí của đồng hồ logic Lamport trong hệ thống phân tán



1) Lamport's logical clocks

*Ứng dụng của đồng hồ logic Lamport: Totally ordered multicasting

- Được sử dụng trong tình huống mà cơ sở dữ liệu được sao chép, lặp lại ở nhiều nơi.
- VD: Để cải thiện hiệu suất truy vấn, ngân hàng có thể duy trì nhiều bản sao của cơ sở dữ liệu tài khoản tại hai thành phố khác nhau, chẳng hạn như New York và San Francisco.

✓ Các truy vấn sẽ luôn được chuyển tiếp đến bản sao gần nhất để tối ưu tốc độ phản hồi.

✗ Đổi lại, chi phí cập nhật sẽ cao hơn, vì mỗi lần cập nhật phải được áp dụng đồng thời trên tất cả các bản sao.

1) Lamport's logical clocks

*Ứng dụng của đồng hồ logic Lamport: Totally ordered multicasting

- VD: Để cải thiện hiệu suất truy vấn, ngân hàng có thể duy trì nhiều bản sao của cơ sở dữ liệu tài khoản tại hai thành phố khác nhau, chẳng hạn như New York và San Francisco.

  - Trên thực tế, việc cập nhật dữ liệu đòi hỏi các quy tắc nghiêm ngặt hơn.

  - Giả sử một khách hàng ở San Francisco gửi thêm 100$ vào tài khoản có sẵn 1.000$, trong khi cùng thời điểm, một nhân viên ngân hàng ở New York cập nhật mức lãi suất 1% cho tài khoản đó.

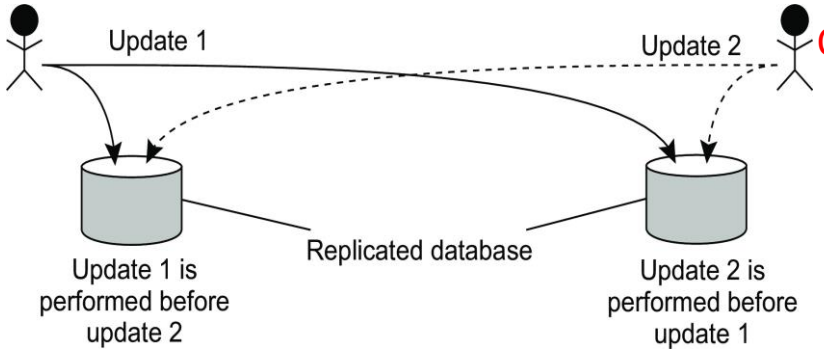
  - Cả hai cập nhật này phải được đồng bộ trên cả hai bản sao của cơ sở dữ liệu. Tuy nhiên, do độ trễ trong truyền thông mạng, hai bản cập nhật có thể đến theo thứ tự khác nhau tại mỗi bản sao, dẫn đến sự sai lệch tiềm ẩn trong dữ liệu.

1) Lamport's logical clocks

*Ứng dụng của đồng hồ logic Lamport: Totally ordered multicasting

- VD: Để cải thiện hiệu suất truy vấn, ngân hàng có thể duy trì nhiều bản sao của cơ sở dữ liệu tài khoản tại hai thành phố khác nhau, chẳng hạn như New York và San Francisco.

Tại San Francisco, bản cập nhật của khách hàng sẽ được áp dụng trước khi cập nhật lãi suất.
-> Ghi nhận số dư: 1.111$
(1.000$ + 100$ + 1.100$ × 0.01)



Ở New York, hệ thống sẽ cập nhật lãi suất 1% trước, sau đó mới ghi nhận khoản tiền gửi của khách hàng.
-> Ghi nhận số dư: 1.110$
(1.000$ + 1.000$ × 0.01 + 100$)

➡ Sự khác biệt này xuất phát từ thứ tự cập nhật không đồng nhất giữa hai bản sao, dẫn đến tình trạng dữ liệu không nhất quán!

➡ Cần đến **totally ordered multicast**, một giao thức multicast mà tất cả các tin nhắn được chuyển và nhận theo cùng một thứ tự đến từng người nhận.

1) Lamport's logical clocks

*Ứng dụng của đồng hồ logic Lamport: Totally ordered multicasting

- Với giao thức multicast, mỗi tin nhắn được gắn mốc thời gian logic của tiến trình gửi và cũng được gửi đến chính tiến trình đó. Các tiến trình đều có một hàng đợi (queue) để lưu trữ thứ tự các tin nhắn.
- Khi nhận tin nhắn, tiến trình đưa vào hàng đợi, sắp xếp theo mốc thời gian, rồi gửi xác nhận multicast đến các tiến trình khác.
  - Theo thuật toán Lamport, mốc thời gian của tin nhắn nhận được sẽ nhỏ hơn mốc thời gian của xác nhận. Kết quả là mọi tiến trình đều có bản sao hàng đợi giống nhau, miễn là không có tin nhắn nào bị xóa.
- Một tiến trình chỉ có thể thực thi tin nhắn khi nó ở đầu hàng đợi và tiến trình đó đã nhận được xác nhận từ tất cả các tiến trình khác. Khi đó, tin nhắn được chuyển đến ứng dụng, xóa khỏi hàng đợi cùng với các xác nhận liên quan.

➡ Nhờ có bản sao hàng đợi đồng nhất, tất cả tiến trình xử lý tin nhắn theo cùng một thứ tự, tạo ra totally ordered multicasting.

1) Lamport's logical clocks

*Sử dụng đồng hồ Lamport để đạt được **mutual exclusion**

- Mutual exclusion: Tại một thời điểm, chỉ một tiến trình hoặc một luồng có thể truy cập vào tài nguyên chia sẻ
- Ý tưởng: Áp dụng phương pháp totally ordered multicasting ở phần trước để điều phối quyền truy cập vào vùng tới hạn (critical region)
- Quy tắc chính:

- Tại một thời điểm, chỉ có một tiến trình được vào vùng tới hạn.
- Tiến trình chỉ được phép vào vùng tới hạn khi:

1) Yêu cầu ENTER của nó phải đứng đầu hàng đợi (có giá trị thời gian thấp nhất). Khi một tiến trình gửi yêu cầu vào vùng tới hạn, nó cũng gửi cho chính mình để đảm bảo hang đợi của các tiến trình đều giống nhau.

2) Tất cả các tiến trình khác đã gửi ACK (xác nhận đồng ý). Sau khi nhận yêu cầu ENTER từ tiến trình khác, tiến trình phải gửi ACK lại cho tiến trình đó.

Distributed Systems - Coordination

1) Lamport's logical clocks

*Sử dụng đồng hồ Lamport để đạt được **mutual exclusion**

```
class Process:
  def __init__(self, chanID, procID, procIDSet):
    self.chan.join(procID)
    self.procID     = int(procID)
    self.otherProcs.remove(self.procID)
    self.queue      = []                        # The request queue
    self.clock      = 0                         # The current logical clock

  def requestToEnter(self):
    self.clock = self.clock + 1                          # Increment clock value
    self.queue.append((self.clock, self.procID, ENTER)) # Append request to q
    self.cleanupQ()                                      # Sort the queue
    self.chan.sendTo(self.otherProcs, (self.clock, self.procID, ENTER)) # Send request

  def ackToEnter(self, requester):
    self.clock = self.clock + 1                          # Increment clock value
    self.chan.sendTo(requester, (self.clock, self.procID, ACK)) # Permit other

  def release(self):
    tmp = [r for r in self.queue[1:] if r[2] == ENTER]  # Remove all ACKs
    self.queue = tmp                                     # and copy to new queue
    self.clock = self.clock + 1                          # Increment clock value
    self.chan.sendTo(self.otherProcs, (self.clock, self.procID, RELEASE)) # Release

  def allowedToEnter(self):
    commProcs = set([req[1] for req in self.queue[1:]]) # See who has sent a message
    return (self.queue[0][1] == self.procID and len(self.otherProcs) == len(commProcs))
```

Figure 5.10: (a) Using Lamport's logical clocks for mutual exclusion.

- Clock synchronization
- Logical clocks
- Mutual exclusion
- Election algorithm
- Gossip-based coordination
- Distributed event matching
- Location systems

1) Lamport's logical clocks

*Sử dụng đồng hồ Lamport để đạt được **mutual exclusion**

- Hàm tạo tiến trình:

- Mỗi tiến trình duy trì một hàng đợi để lưu trữ cả yêu cầu truy cập vùng tới hạn của chính nó và của các tiến trình khác.

- Đồng thời, mỗi tiến trình sử dụng một đồng hồ logic để gán mốc thời gian cho yêu cầu, giúp sắp xếp thứ tự trong hàng đợi một cách chính xác.

```
class Process:
  def __init__(self, chanID, procID, procIDSet):
    self.chan.join(procID)
    self.procID      = int(procID)
    self.otherProcs.remove(self.procID)
    self.queue       = []                    # The request queue
    self.clock       = 0                     # The current logical clock
```

1) Lamport's logical clocks

*Sử dụng đồng hồ Lamport để đạt được **mutual exclusion**

- Hàm gửi yêu cầu vào vùng tới hạn:

- Đầu tiên, tiến trình tăng đồng hồ logic lên 1 đơn vị. Sau đó, nó thêm yêu cầu vào hàng đợi của mình, bao gồm ba giá trị: mốc thời gian gửi, ID tiến trình, và loại tin nhắn (ENTER).

- Tiến trình tiếp tục sắp xếp hàng đợi theo thứ tự mốc thời gian, rồi gửi yêu cầu này đến các tiến trình khác.

```
class Process:
  def __init__(self, chanID, procID, procIDSet):
    self.chan.join(procID)
    self.procID     = int(procID)
    self.otherProcs.remove(self.procID)
    self.queue      = []                        # The request queue
    self.clock      = 0                         # The current logical clock

  def requestToEnter(self):
    self.clock = self.clock + 1                          # Increment clock value
    self.queue.append((self.clock, self.procID, ENTER)) # Append request to q
    self.cleanupQ()                                      # Sort the queue
    self.chan.sendTo(self.otherProcs, (self.clock, self.procID, ENTER)) # Send request
```

1) Lamport's logical clocks

*Sử dụng đồng hồ Lamport để đạt được **mutual exclusion**

- Hàm xác nhận yêu cầu:

- Khi một tiến trình khác gửi yêu cầu ENTER, tiến trình có trách nhiệm phải gửi lại tin nhắn loại ACK.

- Tiến trình đồng thời cũng tăng đồng hồ logic lên 1 đơn vị.

```
class Process:
  def __init__(self, chanID, procID, procIDSet):
    self.chan.join(procID)
    self.procID     = int(procID)
    self.otherProcs.remove(self.procID)
    self.queue      = []                        # The request queue
    self.clock      = 0                         # The current logical clock

  def requestToEnter(self):
    self.clock = self.clock + 1                          # Increment clock value
    self.queue.append((self.clock, self.procID, ENTER)) # Append request to q
    self.cleanupQ()                                      # Sort the queue
    self.chan.sendTo(self.otherProcs, (self.clock, self.procID, ENTER)) # Send request

  def ackToEnter(self, requester):
    self.clock = self.clock + 1                          # Increment clock value
    self.chan.sendTo(requester, (self.clock, self.procID, ACK)) # Permit other
```

1) Lamport's logical clocks

*Sử dụng đồng hồ Lamport để đạt được **mutual exclusion**

- Hàm rời khỏi vùng tới hạn:

- Trước tiên, hàm sẽ loại bỏ tất cả các tin nhắn ACK trong hàng đợi, chỉ giữ lại các tin nhắn dạng ENTER.

- Sau đó, đồng hồ logic của tiến trình được tăng thêm một đơn vị. Tiến trình cũng gửi tin nhắn RELEASE đến các tiến trình khác để thông báo rằng nó đã rời khỏi vùng tới hạn.

```
class Process:
  def __init__(self, chanID, procID, procIDSet):
    self.chan.join(procID)
    self.procID     = int(procID)
    self.otherProcs.remove(self.procID)
    self.queue      = []                        # The request queue
    self.clock      = 0                         # The current logical clock

  def requestToEnter(self):
    self.clock = self.clock + 1                          # Increment clock value
    self.queue.append((self.clock, self.procID, ENTER)) # Append request to q
    self.cleanupQ()                                      # Sort the queue
    self.chan.sendTo(self.otherProcs, (self.clock, self.procID, ENTER)) # Send request

  def ackToEnter(self, requester):
    self.clock = self.clock + 1                          # Increment clock value
    self.chan.sendTo(requester, (self.clock, self.procID, ACK)) # Permit other

  def release(self):
    tmp = [r for r in self.queue[1:] if r[2] == ENTER]  # Remove all ACKs
    self.queue = tmp                                     # and copy to new queue
    self.clock = self.clock + 1                          # Increment clock value
    self.chan.sendTo(self.otherProcs, (self.clock, self.procID, RELEASE)) # Release
```

1) Lamport's logical clocks

*Sử dụng đồng hồ Lamport để đạt được **mutual exclusion**

- Hàm kiểm tra điều kiện để vào vùng tới hạn:
  - Tiến trình chỉ vào vùng tới hạn nếu đáp ứng đồng thời 2 điều kiện:
    + Tiến trình chỉ vào vùng tới hạn nếu đáp ứng đồng thời 2 điều kiện.
    + Đã nhận đủ tin nhắn loại ACK từ tất cả các tiến trình khác.

```python
class Process:
  def __init__(self, chanID, procID, procIDSet):
    self.chan.join(procID)
    self.procID     = int(procID)
    self.otherProcs.remove(self.procID)
    self.queue      = []                        # The request queue
    self.clock      = 0                         # The current logical clock

  def requestToEnter(self):
    self.clock = self.clock + 1                          # Increment clock value
    self.queue.append((self.clock, self.procID, ENTER)) # Append request to q
    self.cleanupQ()                                      # Sort the queue
    self.chan.sendTo(self.otherProcs, (self.clock, self.procID, ENTER)) # Send request

  def ackToEnter(self, requester):
    self.clock = self.clock + 1                          # Increment clock value
    self.chan.sendTo(requester, (self.clock, self.procID, ACK)) # Permit other

  def release(self):
    tmp = [r for r in self.queue[1:] if r[2] == ENTER]  # Remove all ACKs
    self.queue = tmp                                     # and copy to new queue
    self.clock = self.clock + 1                          # Increment clock value
    self.chan.sendTo(self.otherProcs, (self.clock, self.procID, RELEASE)) # Release

  def allowedToEnter(self):
    commProcs = set([req[1] for req in self.queue[1:]]) # See who has sent a message
    return (self.queue[0][1] == self.procID and len(self.otherProcs) == len(commProcs))
```

Figure 5.10: (a) Using Lamport's logical clocks for mutual exclusion.

Distributed Systems - Coordination

1) Lamport's logical clocks

*Sử dụng đồng hồ Lamport để đạt được **mutual exclusion**

```
def receive(self):
  msg = self.chan.recvFrom(self.otherProcs)[1]      # Pick up any message
  self.clock = max(self.clock, msg[0])              # Adjust clock value...
  self.clock = self.clock + 1                       # ...and increment
  if msg[2] == ENTER:
    self.queue.append(msg)                          # Append an ENTER request
    self.ackToEnter(msg[1])                         # and unconditionally allow
  elif msg[2] == ACK:
    self.queue.append(msg)                          # Append a received ACK
  elif msg[2] == RELEASE:
    del(self.queue[0])                              # Just remove first message
  self.cleanupQ()                                   # And sort and cleanup
```

Figure 5.10: (b) Using Lamport's logical clocks for mutual exclusion: handling incoming requests.

1) Lamport's logical clocks

*Sử dụng đồng hồ Lamport để đạt được **mutual exclusion**

Cập nhật đồng hồ logic: Lấy giá trị lớn hơn giữa đồng hồ hiện tại và thời gian của tin nhắn. Tăng đồng hồ logic lên 1 đơn vị.

```
def receive(self):
  msg = self.chan.recvFrom(self.otherProcs)[1]        # Pick up any message
  self.clock = max(self.clock, msg[0])                # Adjust clock value...
  self.clock = self.clock + 1                         # ...and increment
```

1) Lamport's logical clocks

*Sử dụng đồng hồ Lamport để đạt được **mutual exclusion**

```
def receive(self):
  msg = self.chan.recvFrom(self.otherProcs)[1]      # Pick up any message
  self.clock = max(self.clock, msg[0])              # Adjust clock value...
  self.clock = self.clock + 1                       # ...and increment
  if msg[2] == ENTER:
    self.queue.append(msg)                          # Append an ENTER request
    self.ackToEnter(msg[1])                         # and unconditionally allow
```

Nếu nhận được tin nhắn loại ENTER: Tiến trình thêm tin nhắn đó vào hàng đợi, đồng thời gửi tin nhắn loại ACK để đồng ý cho tiến trình khác vào vùng tới hạn.

1) Lamport's logical clocks

*Sử dụng đồng hồ Lamport để đạt được **mutual exclusion**

```
def receive(self):
  msg = self.chan.recvFrom(self.otherProcs)[1]        # Pick up any message
  self.clock = max(self.clock, msg[0])                # Adjust clock value...
  self.clock = self.clock + 1                         # ...and increment
  if msg[2] == ENTER:
    self.queue.append(msg)                            # Append an ENTER request
    self.ackToEnter(msg[1])                           # and unconditionally allow
  elif msg[2] == ACK:
    self.queue.append(msg)                            # Append a received ACK
```

Nếu nhận được tin nhắn loại ACK: Lưu trữ vào hàng đợi để kiểm tra điều kiện tới hạn xem đã tới lượt mình chưa.

1) Lamport's logical clocks

*Sử dụng đồng hồ Lamport để đạt được **mutual exclusion**

```
def receive(self):
  msg = self.chan.recvFrom(self.otherProcs)[1]      # Pick up any message
  self.clock = max(self.clock, msg[0])              # Adjust clock value...
  self.clock = self.clock + 1                       # ...and increment
  if msg[2] == ENTER:
    self.queue.append(msg)                          # Append an ENTER request
    self.ackToEnter(msg[1])                         # and unconditionally allow
  elif msg[2] == ACK:
    self.queue.append(msg)                          # Append a received ACK
  elif msg[2] == RELEASE:
    del(self.queue[0])                              # Just remove first message
  self.cleanupQ()                                   # And sort and cleanup
```

- Nếu nhận tin nhắn loại Release: Xóa yêu cầu ENTER ở đầu hàng đợi vì tiến trình đó đã rời khỏi vùng tới hạn. Dọn dẹp hàng đợi với phương thức (cleanupQ()).

Figure 5.10: (b) Using Lamport's logical clocks for mutual exclusion: handling incoming requests.

2) Vector clocks

- Đồng hồ logic Lamport cung cấp một cách sắp xếp tổng thể cho các sự kiện trong hệ thống phân tán dựa trên quy tắc: Nếu sự kiện a xảy ra trước sự kiện b, thì a được xếp trước b trong hàng đợi, đồng thời thỏa mãn điều kiện C(a) < C(b).

➡ Vấn đề đặt ra: Đồng hồ Lamport chỉ đảm bảo thứ tự tổng thể mà không thể xác định quan hệ nhân quả giữa hai sự kiện. Nói cách khác, nếu C(a) < C(b), chúng ta không thể khẳng định chắc chắn rằng a thực sự xảy ra trước b -> cần có thêm thông tin để chứng minh điều đó.

2) Vector clocks

- Để minh họa, giả sử ba tiến trình trong hệ thống gửi và nhận tin nhắn như trong hình 5.11. Ta định nghĩa:

  +) T_{snd} (m_i): thời gian logic khi tin nhắn m_i được gửi

  +) T_{rcv} (m_i): thời gian logic khi tin nhắn m_i được nhận

-> Theo định nghĩa trên, với mỗi tin nhắn, ta luôn có T_{snd} (m_i) < T_{rcv} (m_i)

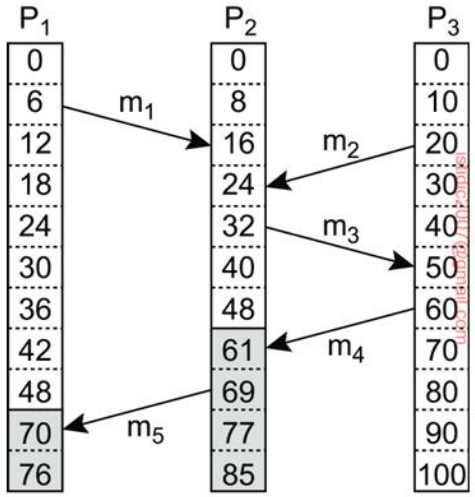


Figure 5.11: Concurrent message transmission using logical clocks.

2) Vector clocks

- Vấn đề của đồng hồ Lamport là không thể phản ánh được quan hệ nhân quả (causality) giữa các sự kiện. Để giải quyết hạn chế này, ta sử dụng đồng hồ vector (Vector Clocks).

  - Theo Baquero và Preguiça, việc theo dõi quan hệ nhân quả có thể thực hiện đơn giản bằng cách gán cho mỗi sự kiện một định danh duy nhất, kết hợp giữa ID tiến trình và bộ đếm tăng dần. Ví dụ, một sự kiện P(k) biểu diễn sự kiện thứ k xảy ra tại tiến trình P. Khi đó, lịch sử nhân quả của một sự kiện có thể được biểu diễn dưới dạng tập hợp. Chẳng hạn, nếu có hai sự kiện cục bộ tại tiến trình P, thì lịch sử nhân quả của sự kiện thứ hai (p2) là H(p2) = {p1, p2}.

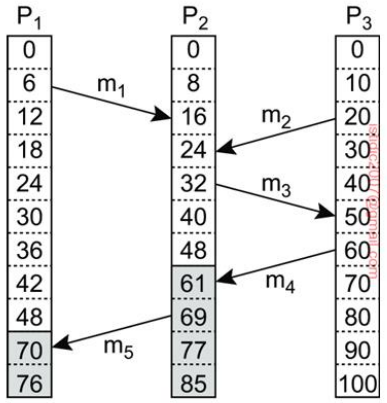


Figure 5.11: Concurrent message transmission using logical clocks.

2) Vector clocks

- Vấn đề của đồng hồ Lamport là không thể phản ánh được quan hệ nhân quả (causality) giữa các sự kiện. Để giải quyết hạn chế này, ta sử dụng đồng hồ vector (Vector Clocks).

  - Theo Baquero và Preguiça, việc theo dõi quan hệ nhân quả có thể thực hiện đơn giản bằng cách gán cho mỗi sự kiện một định danh duy nhất, kết hợp giữa ID tiến trình và bộ đếm tăng dần. Ví dụ, một sự kiện P(k) biểu diễn sự kiện thứ k xảy ra tại tiến trình P. Khi đó, lịch sử nhân quả của một sự kiện có thể được biểu diễn dưới dạng tập hợp. Chẳng hạn, nếu có hai sự kiện cục bộ tại tiến trình P, thì lịch sử nhân quả của sự kiện thứ hai (p2) là H(p2) = {p1, p2}.

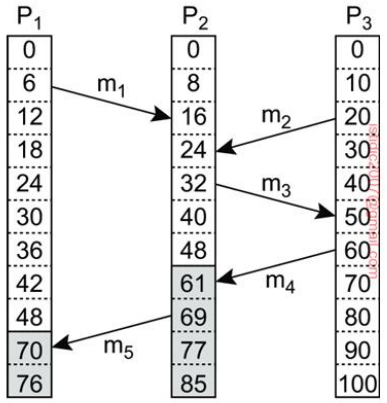


Figure 5.11: Concurrent message transmission using logical clocks.

2) Vector clocks

- Truyền thông điệp đồng thời bằng cách sử dụng đồng hồ logic:

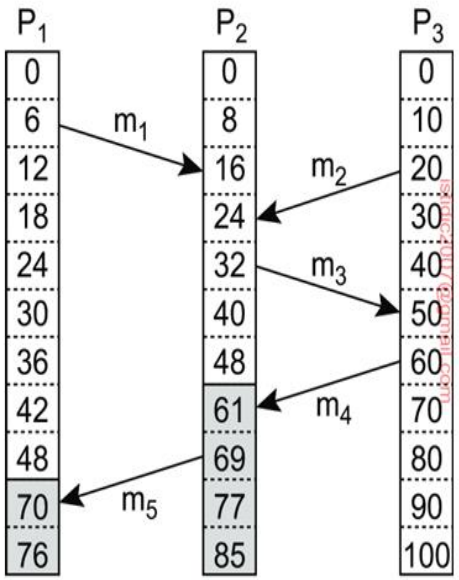


Figure 5.11: Concurrent message transmission using logical clocks.

- Sự kiện a: Thông điệp m1 được nhận tại thời điểm T=16.

- Sự kiện b: Thông điệp m2 được gửi tại thời điểm T=20.

2) Vector clocks

*Causality (Phụ thuộc nhận quả):

- Định nghĩa: Ta nói rằng b có thể phụ thuộc nhân quả vào a nếu ts(a)<ts(b), với điều kiện:

+) Với mọi k, ts(a)[k] ≤ ts(b)[k]

+) Tồn tại ít nhất một chỉ mục k′ sao cho ts(a)[k′] < ts(b)[k′]

- So sánh “happens – before” và “causality”:

+) Nếu a xảy ra trước nhân quả b, ta ký hiệu a → b

+) Nếu b có thể phụ thuộc nhân quả vào a, điều đó có nghĩa là có thể có thông tin từ a được truyền vào b

2) Vector clocks

*Capturing potential causality

- Giải pháp: Mỗi tiến trình Pi duy trì một vector Vci

+) VCi[i] là đồng hồ logic cục bộ tại tiến trình Pi.

+) Nếu VCi[j] = k, thì Pi biết rằng có k sự kiện đã xảy ra tại Pj.

- Duy trì đồng hồ vector

+) Trước khi thực thi một sự kiện, tiến trình Pi thực hiện:

$$VCi[i] \leftarrow VCi[i]+1$$

+) Khi Pi gửi một thông điệp mmm đến Pj: Đặt dấu thời gian vector của thông điệp ts(m) bằng giá trị của VCi sau bước 1

+) Khi Pj nhận thông điệp m: Cập nhật giá trị vector đồng hồ:

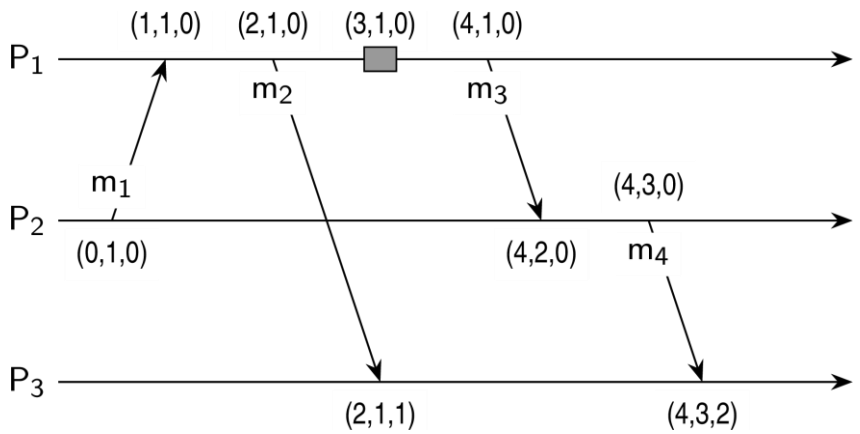
$$VCj[k] \leftarrow \max(VCj[k],ts(m)[k])$$

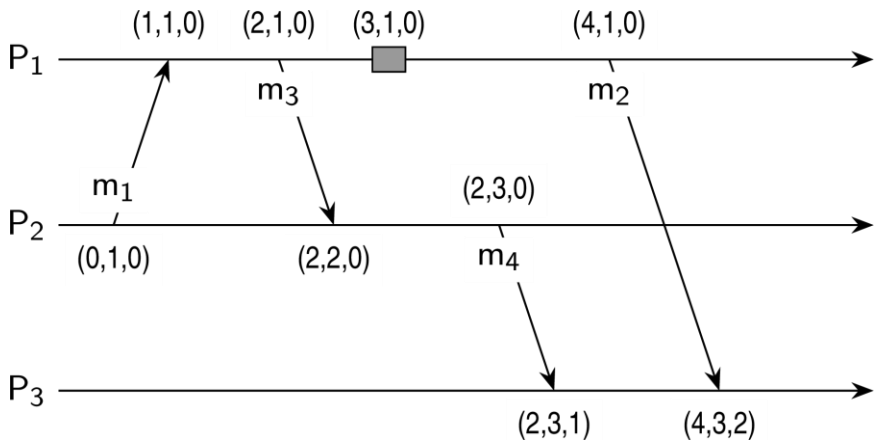
Sau đó, thực thi bước 1 rồi chuyển tiếp thông điệp đến ứng dụng.

2) Vector clocks

*Ví dụ Vector clocks

- Ghi nhận quan hệ nhân quả tiềm tàng khi trao đổi thông điệp





- Bảng tham chiếu

Situation	$ts(m_2)$	$ts(m_4)$	$ts(m_2) < ts(m_4)$	$ts(m_2) > ts(m_4)$	Conclusion
(a)	(2, 1, 0)	(4, 3, 0)	Yes	No	m_2 may causally precede m_4
(b)	(4, 1, 0)	(2, 3, 0)	No	No	m_2 and m_4 may conflict

2) Vector clocks

*Causally ordered multicasting

- Quan sát: Có thể đảm bảo rằng một thông điệp chỉ được xử lý khi tất cả các thông điệp có quan hệ nhân quả trước đó đã được xử lý.
- Điều chỉnh thuật toán:

+) Tiến trình Pi chỉ tăng giá trị của VCi[i] khi gửi một thông điệp.

+) Khi nhận thông điệp, tiến trình Pj điều chỉnh VCj nhưng không thay đổi giá trị VCj[j] của chính nó.

+) Tiến trình Pj sẽ trì hoãn việc xử lý thông điệp m cho đến khi:

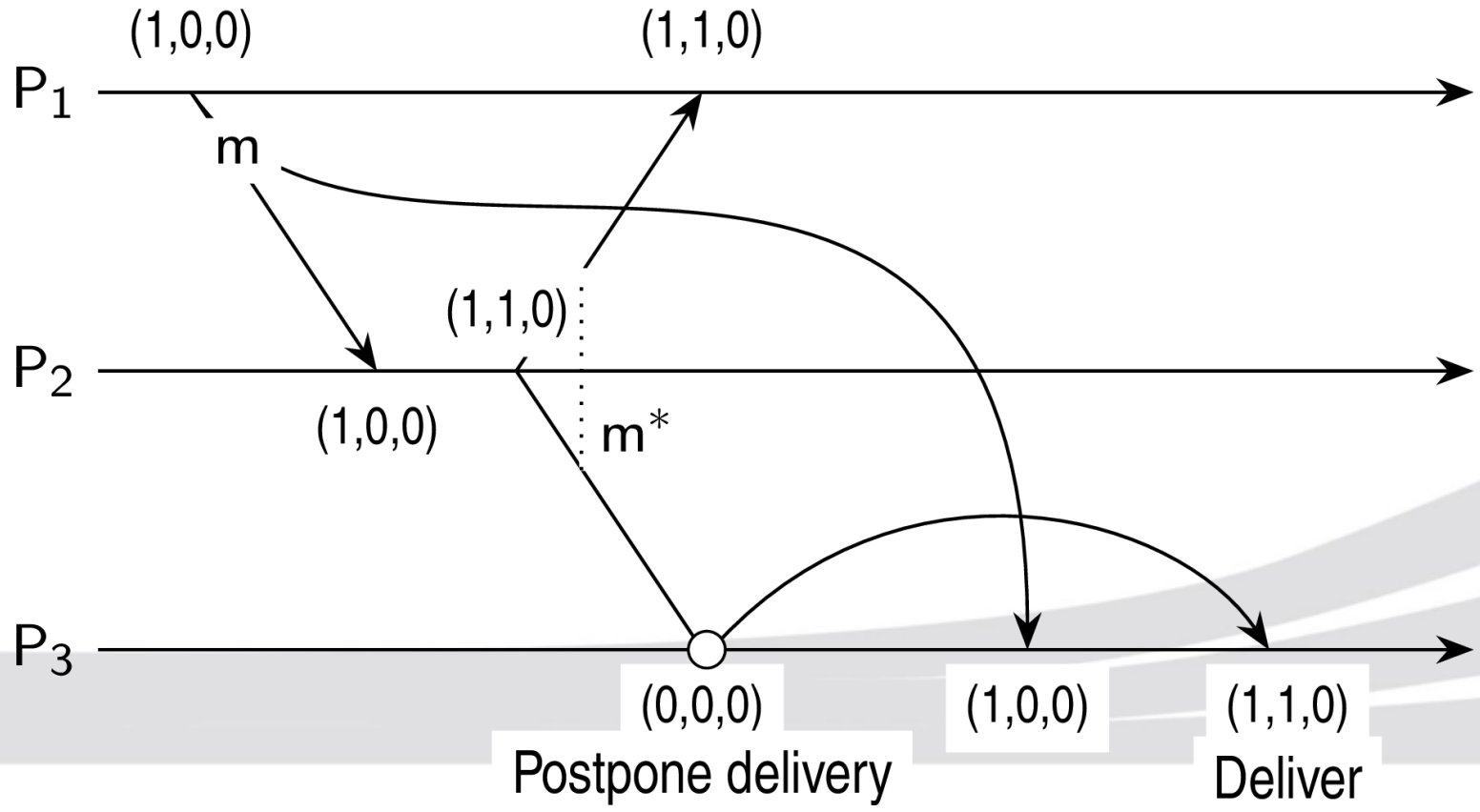
1. Điều kiện về tiến trình gửi: $ts(m)[i] = VCj[i] + 1$

2. Điều kiện về tiến trình khác: $ts(m)[k] \leq VCj[k]$ với mọi $k \neq i$

2) Vector clocks

*Causally ordered multicasting

- Đảm bảo truyền thông theo quan hệ nhân quả



Distributed Systems - Coordination

Nền tảng của các hệ thống phân tán là tính đồng thời và sự hợp tác giữa nhiều tiến trình. Trong nhiều trường hợp, điều này cũng có nghĩa là các tiến trình sẽ cần truy cập đồng thời vào cùng một tài nguyên. Để ngăn chặn việc truy cập đồng thời có thể làm hỏng tài nguyên hoặc khiến nó trở nên không nhất quán, cần có các giải pháp để đảm bảo quyền truy cập độc quyền cho từng tiến trình.

Trong phần này, chúng ta sẽ xem xét một số thuật toán phân tán quan trọng và tiêu biểu đã được đề xuất để giải quyết vấn đề này. Các nghiên cứu tổng quan về các thuật toán loại trừ tương hỗ trong hệ thống phân tán được cung cấp bởi Saxena và Rai [2003] cũng như Velazquez [1993]. Ngoài ra, nhiều thuật toán khác cũng được trình bày trong tài liệu của Kshemkalyani và Singhal [2008].

- Clock synchronization
- Logical clocks
- Mutual exclusion
- Election algorithm
- Gossip-based coordination
- Distributed event matching
- Location systems

Distributed Systems - Coordination

1) Overview

Các thuật toán loại trừ tương hỗ trong hệ thống phân tán có thể được phân thành hai loại khác nhau.

1.Giải pháp dựa trên token (Token-based solutions):

-Loại trừ tương hỗ được đảm bảo bằng cách truyền một thông điệp đặc biệt giữa các tiến trình, gọi là token.

-Chỉ có duy nhất một token tồn tại, và tiến trình nào giữ token sẽ được phép truy cập vào tài nguyên chia sẻ.

-Khi tiến trình hoàn thành việc sử dụng tài nguyên, nó sẽ chuyển token cho tiến trình tiếp theo. Nếu một tiến trình có token nhưng không cần truy cập tài nguyên, nó cũng sẽ chuyển token đi.

-Giải pháp này có một số đặc điểm quan trọng:

+Dễ dàng đảm bảo rằng mỗi tiến trình đều có cơ hội truy cập tài nguyên, tránh hiện tượng chờ đợi vô thời hạn (starvation).

+Tránh được deadlock, tức là tránh tình trạng nhiều tiến trình chờ nhau vô thời hạn để tiếp tục thực thi.

-Nhược điểm lớn nhất của phương pháp này là nếu token bị mất (ví dụ: tiến trình đang giữ token bị lỗi hoặc gặp sự cố), thì cần một cơ chế phức tạp để tạo lại token mới. Điều quan trọng là phải đảm bảo rằng chỉ có một token duy nhất tồn tại trong hệ thống.

2.Giải pháp dựa trên quyền cho phép (Permission-based solutions):

-Thay vì sử dụng token, tiến trình muốn truy cập tài nguyên cần yêu cầu quyền từ các tiến trình khác.

-Có nhiều cách khác nhau để cấp quyền, và các phần sau sẽ trình bày một số phương pháp tiêu biểu trong nhóm này.

Distributed Systems - Coordination

2) A centralized algorithm

Cách tiếp cận tập trung (Centralized Approach) để loại trừ tương hỗ
Một cách đơn giản để đạt được loại trừ tương hỗ (mutual exclusion) trong hệ thống phân tán là mô phỏng cách nó hoạt động trong một hệ thống có một bộ xử lý duy nhất.
* Cơ chế hoạt động

- Chọn một tiến trình làm bộ điều phối (Coordinator)
  +Tiến trình này chịu trách nhiệm kiểm soát quyền truy cập vào tài nguyên dùng chung.
- Tiến trình yêu cầu quyền truy cập tài nguyên
  +Khi một tiến trình muốn sử dụng tài nguyên, nó gửi yêu cầu (Request) đến bộ điều phối.
  +Nếu tài nguyên đang rảnh, bộ điều phối gửi lại phản hồi OK (Grant) để cấp quyền sử dụng.
  +Khi nhận được phản hồi này, tiến trình có thể sử dụng tài nguyên.
  +(Hình 5.14a): P1 gửi yêu cầu và được cấp quyền truy cập vì hàng đợi trống.

2) A centralized algorithm

- Trường hợp tài nguyên đang bận
  +Nếu một tiến trình khác đã chiếm dụng tài nguyên, bộ điều phối không cấp quyền ngay lập tức.
  +Có hai cách xử lý:
    Không phản hồi → Tiến trình yêu cầu bị chặn (blocking).
    Gửi phản hồi từ chối → Tiến trình biết mình chưa được cấp quyền.
  +(Hình 5.14b): P2 gửi yêu cầu nhưng không nhận phản hồi vì P1 đang giữ tài nguyên. Bộ điều phối xếp P2 vào hàng đợi.
- Giải phóng tài nguyên và cấp quyền cho tiến trình tiếp theo
  +Khi tiến trình đang giữ tài nguyên hoàn thành, nó gửi thông báo giải phóng (Release) đến bộ điều phối.
  +Bộ điều phối lấy tiến trình đầu tiên trong hàng đợi và gửi thông báo OK (Grant) để cấp quyền.
  +(Hình 5.14c): P1 giải phóng tài nguyên, bộ điều phối cấp quyền cho P2.

Distributed Systems - Coordination

2) A centralized algorithm

Ưu điểm của phương pháp tập trung

-Đảm bảo loại trừ tương hỗ: Chỉ có một tiến trình truy cập tài nguyên tại một thời điểm.

- Không có tình trạng bỏ đói (No starvation): Yêu cầu được xử lý theo thứ tự đến trước.
- Hiệu quả về thông điệp: Chỉ cần 3 thông điệp để quản lý tài nguyên (Request, Grant, Release).
- Triển khai đơn giản: Không cần cơ chế phức tạp như trong các thuật toán phân tán khác.

Nhược điểm của phương pháp tập trung

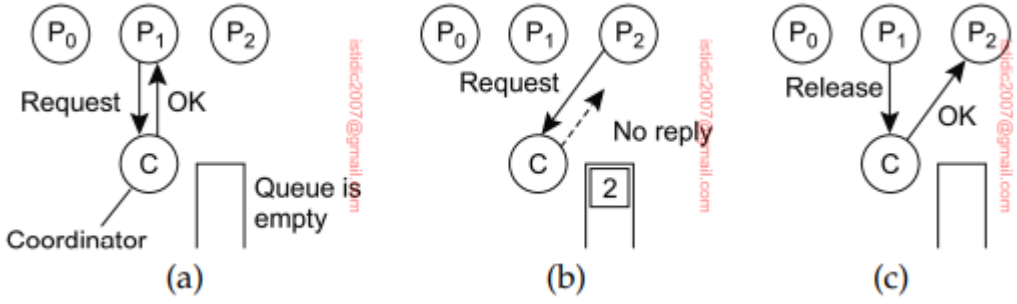
-Điểm lỗi đơn lẻ (Single Point of Failure): Nếu bộ điều phối bị lỗi, toàn bộ hệ thống có thể ngừng hoạt động.

-Tiến trình bị chặn không phân biệt lỗi hệ thống: Nếu bộ điều phối không phản hồi, tiến trình yêu cầu không biết là do bị từ chối hay bộ điều phối đã chết.

-Tắc nghẽn (Bottleneck): Trong hệ thống lớn, bộ điều phối có thể bị quá tải do nhận quá nhiều yêu cầu.

Distributed Systems - Coordination

2) A centralized algorithm



(a) P1 gửi yêu cầu và được cấp quyền
- P1 gửi Request đến bộ điều phối C.
- Bộ điều phối kiểm tra hàng đợi, thấy trống nên gửi OK (Grant).
- P1 được phép sử dụng tài nguyên.

(b) P2 gửi yêu cầu nhưng bị chặn
- P2 cũng gửi Request đến bộ điều phối C.
- Bộ điều phối thấy P1 đang sử dụng tài nguyên nên không phản hồi ngay.
- P2 bị chặn và được xếp vào hàng đợi.

(c) P1 giải phóng tài nguyên, P2 nhận quyền truy cập
- P1 gửi Release để trả lại tài nguyên.
- Bộ điều phối lấy yêu cầu đầu tiên trong hàng đợi (P2) và gửi OK (Grant).
- P2 nhận được quyền truy cập tài nguyên.

3) A distributed algorithm

Thuật toán phân tán của Ricart và Agrawala (1981)
Dựa trên đồng hồ logic của Lamport, Ricart và Agrawala đề xuất một thuật toán phân tán để giải quyết vấn đề loại trừ tương hỗ trong hệ thống phân tán. Thuật toán này đảm bảo thứ tự tổng thể (total ordering) của các sự kiện trong hệ thống.

Cách thức hoạt động
🔷 Bước 1: Gửi yêu cầu
•Khi một tiến trình muốn truy cập tài nguyên dùng chung, nó sẽ gửi một thông điệp yêu cầu (Request Message) đến tất cả các tiến trình khác, bao gồm:
-Tên tài nguyên
-Số hiệu tiến trình
-Dấu thời gian logic (Logical Timestamp)
•Các thông điệp gửi đi được giả định là đáng tin cậy, tức là không bị mất.

3) A distributed algorithm

◆ Bước 2: Xử lý yêu cầu

Khi một tiến trình nhận được yêu cầu từ một tiến trình khác, nó sẽ xử lý theo ba trường hợp:

1.Tiến trình chưa dùng tài nguyên & không có nhu cầu sử dụng

-Trả lời ngay bằng một thông điệp OK để cấp quyền.

2.Tiến trình đang sử dụng tài nguyên

-Không phản hồi ngay, thay vào đó xếp yêu cầu vào hàng đợi.

3.Tiến trình cũng muốn sử dụng tài nguyên

-So sánh dấu thời gian của yêu cầu đến với dấu thời gian của chính nó.

-Tiến trình có dấu thời gian nhỏ hơn sẽ thắng.

-Nếu tiến trình nhận yêu cầu có dấu thời gian lớn hơn (thua), nó sẽ gửi OK cho tiến trình thắng.

-Nếu tiến trình nhận yêu cầu có dấu thời gian nhỏ hơn, nó sẽ xếp yêu cầu vào hàng đợi mà không phản hồi.

◆ Bước 3: Chờ xác nhận

•Sau khi gửi yêu cầu, tiến trình chờ cho đến khi tất cả tiến trình khác đã gửi OK thì mới được phép truy cập tài nguyên.

◆ Bước 4: Giải phóng tài nguyên

•Sau khi sử dụng xong tài nguyên, tiến trình sẽ gửi OK đến tất cả tiến trình đang chờ trong hàng đợi, cho phép tiến trình tiếp theo tiếp tục.

3) A distributed algorithm

Phân tích thuật toán

-Đảm bảo loại trừ tương hỗ: Chỉ có một tiến trình sử dụng tài nguyên tại một thời điểm.

-Không có deadlock & starvation: Nhờ vào việc sử dụng dấu thời gian logic để quyết định thứ tự.

-Số lượng thông điệp cần gửi là 2(N-1):

+ N-1 yêu cầu gửi đến tất cả các tiến trình khác.

+ N-1 phản hồi OK từ các tiến trình khác.

Hạn chế

-N điểm lỗi (N points of failure): Nếu một tiến trình bị lỗi, nó sẽ không gửi OK → Có thể chặn tất cả tiến trình khác.

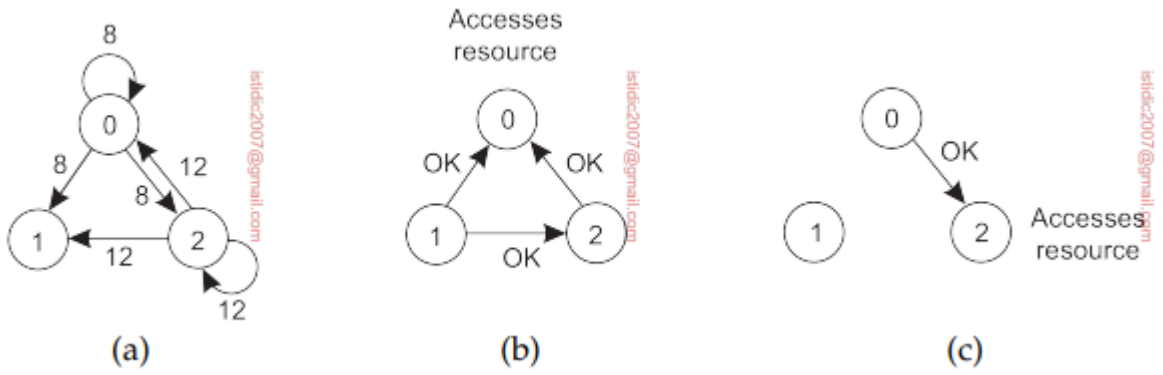
-Chi phí cao với hệ thống lớn: Mỗi tiến trình phải quản lý danh sách nhóm, bao gồm cả tiến trình mới tham gia hoặc rời khỏi hệ thống.

-Gánh nặng truyền thông: Mọi tiến trình đều tham gia vào quyết định cấp quyền, điều này có thể gây quá tải trong hệ thống tài nguyên hạn chế.

Cách cải tiến

-Không cần nhận OK từ tất cả các tiến trình, chỉ cần thu thập OK từ đa số (majority voting) là đủ.

3) A distributed algorithm



(a) Hai tiến trình gửi yêu cầu cùng lúc
•P0 gửi yêu cầu với timestamp = 8.
•P2 gửi yêu cầu với timestamp = 12.
•P1 không cần tài nguyên, nên gửi OK cho cả P0 và P2.
•P0 và P2 phát hiện xung đột và so sánh timestamp.
(b) P0 thắng do timestamp nhỏ hơn
•P2 nhận thấy timestamp của nó lớn hơn, nên gửi OK cho P0.
•P0 được phép sử dụng tài nguyên.
•P0 lưu yêu cầu của P2 vào hàng đợi để xử lý sau.
(c) P0 giải phóng tài nguyên, P2 được cấp quyền
•P0 gửi OK cho P2 để cho phép P2 sử dụng tài nguyên.
•P2 giờ có thể truy cập tài nguyên.

4) A token-ring algorithm

Một cách tiếp cận hoàn toàn khác để đạt được loại trừ tương hỗ (mutual exclusion) trong hệ thống phân tán được minh họa trong Hình 5.16.

- Phần mềm tạo một mạng phủ (overlay network) theo dạng vòng tròn logic (logical ring).
- Mỗi tiến trình được chỉ định một vị trí trong vòng tròn và chỉ cần biết tiến trình tiếp theo sau nó.

Khi vòng tròn được khởi tạo:

- Tiến trình P0 được cấp một mã thông báo (token).
- Mã thông báo này luân chuyển quanh vòng tròn.
- Nếu có N tiến trình, mã thông báo được truyền từ tiến trình Pk đến tiến trình P(k+1) mod N bằng các thông điệp điểm-đến-điểm (point-to-point messages).

◆ Cách tiến trình sử dụng tài nguyên:

- Khi một tiến trình nhận mã thông báo, nó sẽ kiểm tra xem mình có cần truy cập tài nguyên hay không.
- Nếu cần truy cập:
  - Tiến trình sử dụng tài nguyên, hoàn thành công việc.
  - Sau đó, nó giải phóng tài nguyên và chuyển mã thông báo cho tiến trình tiếp theo.
  - Không được phép giữ lại mã thông báo để sử dụng liên tiếp.
- Nếu không cần tài nguyên:
  - Tiến trình chuyển ngay mã thông báo cho tiến trình kế tiếp.

◆ Khi không có tiến trình nào cần tài nguyên:

- Mã thông báo vẫn tiếp tục di chuyển trong vòng tròn.

4) A token-ring algorithm

Phân tích thuật toán

Ưu điểm

1.Đảm bảo loại trừ tương hỗ:

-Chỉ có một mã thông báo tồn tại trong hệ thống, nên chỉ một tiến trình có thể sử dụng tài nguyên tại một thời điểm.

2.Không xảy ra deadlock:

-Do mã thông báo luôn được truyền đi theo vòng tròn.

3.Không xảy ra starvation:

-Mỗi tiến trình sẽ nhận được mã thông báo theo thứ tự nhất định.

-Tiến trình cần tài nguyên chỉ cần đợi tối đa N-1 lần nhận mã thông báo.

Nhược điểm

1.Mất mã thông báo

-Nếu tiến trình giữ mã thông báo bị sập (crash) hoặc thông điệp chứa mã thông báo bị mất, hệ thống sẽ bị đóng băng.

-Việc phát hiện mất mã thông báo là khó khăn, vì không có giới hạn thời gian cố định cho vòng tuần hoàn của mã thông báo.

2.Tiến trình bị lỗi (Crash của một tiến trình trong vòng)

-Nếu một tiến trình bị lỗi, tiến trình trước đó không thể truyền mã thông báo cho nó.

-Cách khắc phục:

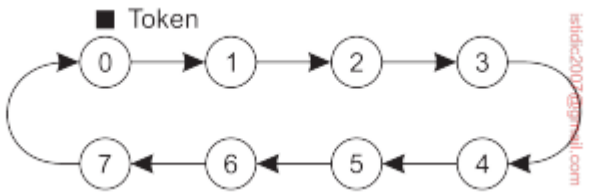
Yêu cầu ACK (Acknowledgment - xác nhận) khi truyền mã thông báo.

Nếu không nhận được ACK, tiến trình trước có thể bỏ qua tiến trình bị lỗi và truyền mã thông báo cho tiến trình tiếp theo.

Đòi hỏi tất cả tiến trình phải duy trì thông tin về vòng tròn hiện tại.

Distributed Systems - Coordination

4) A token-ring algorithm



- Cấu trúc vòng tròn logic với 8 tiến trình (P0 → P7).
- Mã thông báo (Token) hiện đang ở tiến trình P0 (được biểu diễn bằng ô vuông).
- Mũi tên thể hiện hướng luân chuyển của mã thông báo (P0 → P1 → P2 → ... → P7 → P0).
- Nếu một tiến trình có nhu cầu sử dụng tài nguyên, nó sẽ giữ mã thông báo để thực hiện công việc, rồi mới chuyển tiếp.
- Nếu không, nó sẽ truyền mã thông báo ngay cho tiến trình tiếp theo.

5) A decentralized algorithm

Thuật toán bỏ phiếu phân tán của Lin et al. [2004]
Một giải pháp hoàn toàn phi tập trung (fully decentralized) cho bài toán loại trừ tương hỗ (mutual exclusion) được đề xuất bởi Lin et al. [2004] dựa trên thuật toán bỏ phiếu (voting algorithm):

- Tài nguyên được nhân bản (replicated) N lần.
- Mỗi bản sao (replica) có một bộ điều phối (coordinator) riêng để kiểm soát quyền truy cập.
- Khi một tiến trình muốn sử dụng tài nguyên, nó phải nhận được sự chấp thuận của đa số (m > N/2) bộ điều phối.

🔷 Cách thức hoạt động:

- Nếu một bộ điều phối đã cấp quyền cho tiến trình khác, nó sẽ từ chối yêu cầu mới.
- Giả định: Khi một bộ điều phối bị lỗi (crash), nó sẽ khôi phục nhanh chóng nhưng quên mọi phiếu bầu trước đó.
- Điều này có thể dẫn đến lỗi cấp quyền sai, nếu bộ điều phối cấp quyền lại cho một tiến trình khác sau khi khởi động lại.

5) A decentralized algorithm

◆ Xác suất vi phạm tính đúng đắn:

•p = Δt / T là xác suất một bộ điều phối bị reset trong khoảng thời gian Δt, với T là tuổi thọ trung bình của nó.

•P[k] là xác suất có k trong số m bộ điều phối bị reset đồng thời:

$$P[k] = \binom{m}{k} p^k (1-p)^{m-k}$$

Nếu f bộ điều phối bị reset, tính đúng đắn sẽ bị vi phạm khi f ≥ 2m - N.

5) A decentralized algorithm

Xác suất vi phạm tính đúng đắn là:

$$\sum_{k=2m-N}^{m} P[k]$$

◆ Triển khai thực tế:

- Tài nguyên có một tên duy nhất (rname).
- Mỗi bản sao có một khóa duy nhất (rnamei) được tạo bằng hàm băm (hash function).
- Tiến trình có thể tìm kiếm bộ điều phối của từng bản sao thông qua hệ thống định danh chung.
- Nếu không đủ phiếu bầu, tiến trình sẽ chờ một thời gian ngẫu nhiên trước khi thử lại.

5) A decentralized algorithm

Thuật toán	Số tin nhắn (messages) để vào/ra vùng tới hạn	Độ trễ trước khi vào (message times)
Tập trung (Centralized)	3 (Yêu cầu → Cấp quyền → Giải phóng)	2 MTTU
Phân tán (Distributed)	2(N-1) (Yêu cầu & Cấp quyền từ N-1 tiến trình)	2(N-1) MTTU
Vòng mã thông báo (Token Ring)	Từ 1 đến vô hạn (Tùy vào trạng thái của token)	Từ 0 đến N-1 MTTU
Phân tán hoàn toàn (Decentralized Voting)	2kN + (k-1)N/2 + N (với k ≥ 1)	2kN + (k-1)N/2 MTTU

Nhận xét:
-Thuật toán tập trung là đơn giản nhất và hiệu quả nhất.
-Thuật toán phân tán đòi hỏi nhiều tin nhắn hơn, nhưng không có điểm lỗi trung tâm.
-Vòng mã thông báo có độ trễ thay đổi tùy vào trạng thái của token.
-Bỏ phiếu phân tán phức tạp hơn, nhưng giảm nguy cơ điểm lỗi trung tâm.

6) Example: Simple locking with ZooKeeper

1. Giới thiệu về ZooKeeper

•ZooKeeper được thiết kế để hỗ trợ các tác vụ điều phối trong hệ thống phân tán, bao gồm khóa (locking), bầu chọn leader, giám sát (monitoring), v.v.

•Nó có khả năng mở rộng và chịu lỗi, tuy nhiên phần chịu lỗi sẽ được bàn ở chương 8.

•Một nguyên tắc quan trọng là ZooKeeper không hỗ trợ các hoạt động chặn (blocking): Khi một client gửi yêu cầu, nó luôn nhận được phản hồi ngay lập tức.

•Không có khóa chờ đợi: Nếu một client không lấy được khóa, nó phải thử lại sau.

2. Cấu trúc dữ liệu của ZooKeeper

•ZooKeeper tổ chức không gian tên thành cây thư mục (giống hệ thống file Unix).

•Các thao tác cơ bản: tạo, xóa, đọc, cập nhật nút (node).

•Không hỗ trợ cập nhật một phần dữ liệu trong node (mỗi lần cập nhật là ghi đè toàn bộ nội dung).

•Hai loại node trong ZooKeeper:

- oNode bền vững (Persistent node): Phải được tạo và xóa tường minh.
- oNode tạm thời (Ephemeral node): Tự động bị xóa khi client tạo ra nó bị mất kết nối.

6) Example: Simple locking with ZooKeeper

3. Cơ chế khóa trong ZooKeeper

- Cách thực hiện khóa đơn giản:
  - Khi một client muốn khóa tài nguyên, nó tạo node /lock.
  - Nếu node /lock đã tồn tại, nghĩa là khóa đã bị một client khác chiếm giữ.
  - Khi muốn giải phóng khóa, client chỉ cần xóa node /lock.
- Cơ chế thông báo thay đổi (watchers):
  - Nếu một client không thể tạo node /lock (tức là bị chặn bởi một client khác), nó có thể đăng ký nhận thông báo khi node này bị xóa.
  - Khi node /lock bị xóa, ZooKeeper sẽ gửi thông báo để client có thể thử lại.
  - Lưu ý về điều kiện tranh chấp (race condition): Nếu client chưa kịp đăng ký nhận thông báo mà node đã bị xóa, client sẽ không biết rằng mình có thể thử lại. ZooKeeper đảm bảo rằng những thay đổi này không bị bỏ sót.

4. Cơ chế kiểm tra phiên bản để đảm bảo tính nhất quán

- Mỗi node có một số phiên bản (version number).
- Khi một client muốn cập nhật dữ liệu trong node, nó phải chỉ định phiên bản hiện tại của node đó.
- Nếu phiên bản của node đã thay đổi bởi một client khác, cập nhật sẽ thất bại và client phải đọc lại dữ liệu mới nhất trước khi thử lại.

6) Example: Simple locking with ZooKeeper

5. Một số vấn đề quan trọng khi sử dụng khóa trong ZooKeeper

- Phải đảm bảo client có thể thoát ra sau một số lần thử nhất định, tránh vòng lặp vô tận.
- Xử lý lỗi khi client giữ khóa bị crash: Nếu một client tạo node /lock nhưng sau đó gặp sự cố, ZooKeeper sẽ tự động xóa node nếu đó là ephemeral node.
- Vấn đề điều kiện tranh chấp (race condition) giữa các client: Nếu một client không kịp đăng ký theo dõi node /lock, nó có thể bỏ lỡ thông báo rằng khóa đã được giải phóng. ZooKeeper giải quyết vấn đề này bằng cách kiểm tra trạng thái của node ngay khi client đăng ký.

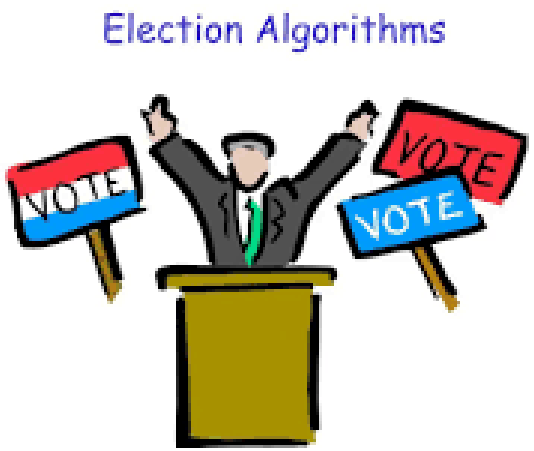
4) Thuật toán bầu chọn (Election Algorithms)

- Vấn đề đặt ra

- Nhiều thuật toán phân tán yêu cầu một tiến trình đóng vai trò là điều phối viên, khởi tạo viên hoặc thực hiện một nhiệm vụ đặc biệt nào đó.

- Nhìn chung, tiến trình nào đảm nhận vai trò đặc biệt này không quan trọng, nhưng phải có một tiến trình đảm nhận nó. Trong phần này, chúng ta sẽ xem xét các thuật toán để bầu chọn một điều phối viên.

➡ Quá trình này thường được gọi là bầu chọn lãnh đạo (leader election)



4) Thuật toán bầu chọn (Election Algorithms)

- Giả định thuật toán

- Chúng ta giả định rằng mỗi tiến trình đều biết định danh của tất cả các tiến trình khác. Nói cách khác, mỗi tiến trình có toàn bộ thông tin về nhóm tiến trình trong đó điều phối viên cần được bầu chọn.
- Tuy nhiên, điều mà các tiến trình không biết là tiến trình nào đang hoạt động (up) và tiến trình nào đang dừng (down) tại thời điểm hiện tại.
- Mục tiêu của một thuật toán bầu chọn là đảm bảo rằng khi quá trình bầu chọn bắt đầu, nó sẽ kết thúc với tất cả các tiến trình đồng thuận về việc ai là điều phối viên mới.

4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán bắt nạt (The Bully Algorithm)

- Một giải pháp nổi tiếng để bầu chọn điều phối viên là thuật toán bắt nạt (bully algorithm) do Garcia-Molina [1982] đề xuất.

- Giả sử chúng ta có N tiến trình $\{P_0, P_1, \ldots, P_{N-1}\}$ và mỗi tiến trình Pk có một định danh duy nhất id(P_k) = k.

- Khi một tiến trình phát hiện điều phối viên hiện tại không còn phản hồi, nó sẽ khởi xướng một cuộc bầu chọn.

4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán bắt nạt (The Bully Algorithm)

- Cách tiến trình P_k tổ chức cuộc bầu chọn như sau:

- 1. Gửi thông báo bầu chọn:

+ P_k gửi một thông điệp ELECTION đến tất cả các tiến trình có định danh cao hơn nó: P_{k+1} , P_{k+2} ,…, P_{N-1}

- 2. Kiểm tra phản hồi:

+ Nếu không có tiến trình nào phản hồi, điều đó có nghĩa là P_k có định danh cao nhất trong số các tiến trình còn sống, nó chiến thắng và trở thành điều phối viên.

+ Nếu có một tiến trình có định danh cao hơn trả lời, tiến trình đó sẽ đảm nhận quá trình bầu chọn, và P_k dừng lại.

4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán bắt nạt (The Bully Algorithm)

- Cách một tiến trình phản hồi khi nhận được thông báo bầu chọn:

+ Nếu một tiến trình nhận được thông báo ELECTION từ một tiến trình có định danh thấp hơn, nó phản hồi bằng một thông điệp OK, báo hiệu rằng nó vẫn còn hoạt động và sẽ đảm nhận việc bầu chọn.

+ Sau khi phản hồi, tiến trình đó sẽ khởi xướng cuộc bầu chọn mới (trừ khi nó đã có sẵn một cuộc bầu chọn đang diễn ra).

+ Cuối cùng, tất cả các tiến trình ngoài một tiến trình có định danh cao nhất sẽ từ bỏ, và tiến trình đó trở thành điều phối viên mới.

+ Nó thông báo chiến thắng của mình bằng cách gửi một thông điệp đến tất cả các tiến trình khác rằng nó là điều phối viên mới từ thời điểm này.

Clock synchronization
Logical clocks
Mutual exclusion
Election algorithm
Gossip-based coordination
Distributed event matching
Location systems

4) Thuật toán bầu chọn (Election Algorithms)

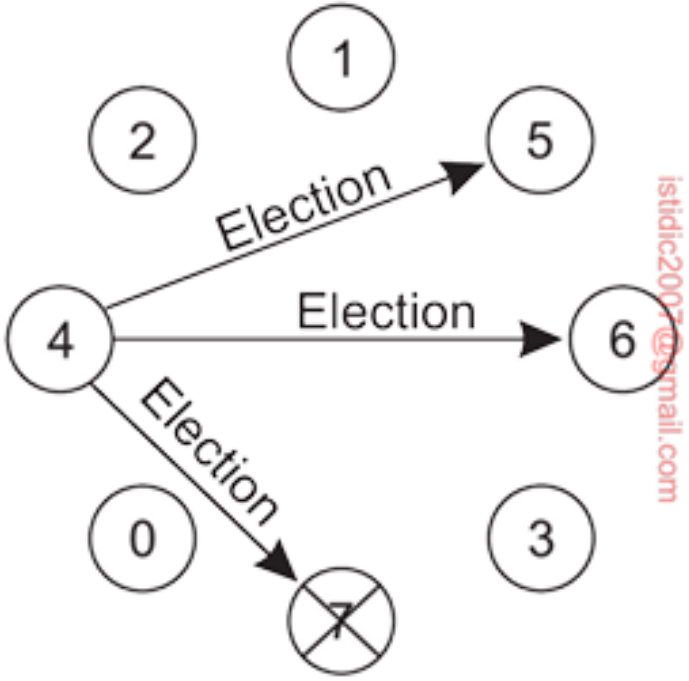
- Thuật toán bắt nạt (The Bully Algorithm)

- Ví dụ minh họa cách thuật toán hoạt động:

+ Giả sử hệ thống có 8 tiến trình, đánh số từ P_0 đến P_7.

+ Ban đầu, tiến trình P_7 là điều phối viên, nhưng nó bị sập.

+ P_4 là tiến trình đầu tiên phát hiện ra điều này, nên nó khởi xướng cuộc bầu chọn bằng cách gửi thông điệp ELECTION đến các tiến trình có số định danh cao hơn nó: P_5, P_6, và P_7



(a)

istidic2007@gmail.com

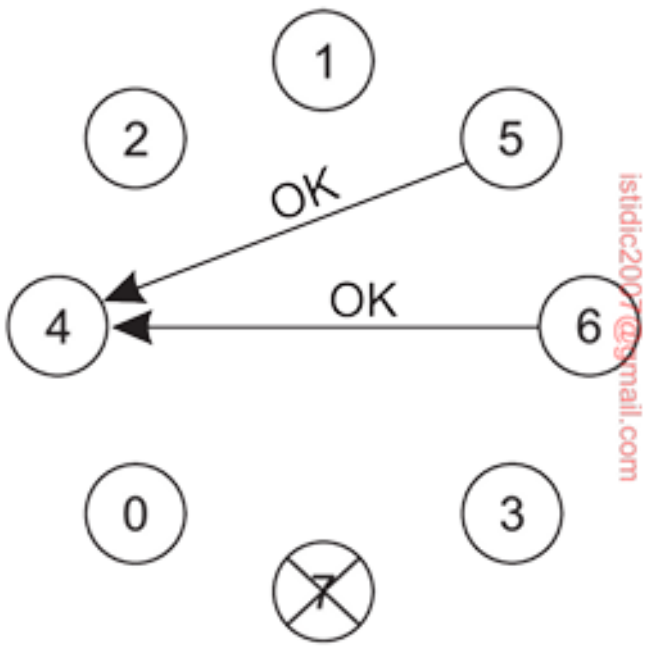
4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán bắt nạt (The Bully Algorithm)

- Ví dụ minh họa cách thuật toán hoạt động:

+ P_5 và P_6 phản hồi bằng OK (Hình 5.20(b)), báo hiệu rằng chúng vẫn đang hoạt động và sẽ đảm nhận bầu chọn.

+ P_4 nhận được phản hồi, biết rằng nhiệm vụ của nó đã xong, vì chắc chắn một trong hai tiến trình P_5 hoặc P_6 sẽ trở thành điều phối viên mới.



(b)

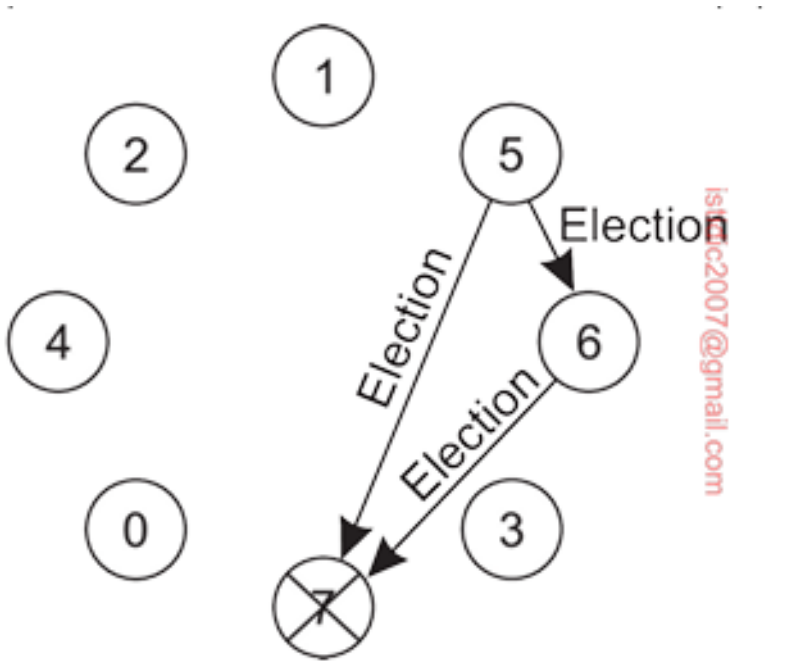
istidic2007@gmail.com

4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán bắt nạt (The Bully Algorithm)

- Ví dụ minh họa cách thuật toán hoạt động:

+ P_5 và P_6 sau đó tự tổ chức bầu chọn, chỉ gửi tin nhắn đến những tiến trình có số định danh cao hơn nó (Hình 5.20(c)).



(c)

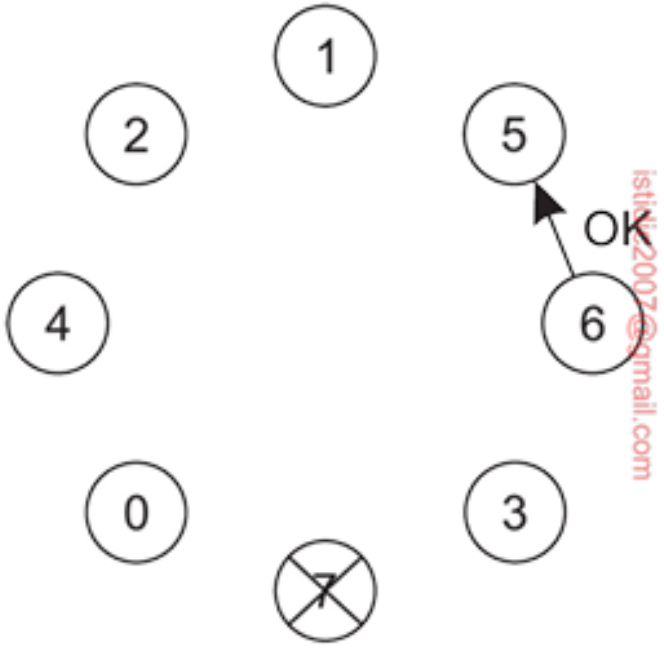
istic2007@gmail.com

4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán bắt nạt (The Bully Algorithm)

- Ví dụ minh họa cách thuật toán hoạt động:

+ P_6 thông báo với P_5 rằng nó sẽ đảm nhận vai trò điều phối viên (Hình 5.20(d)).



(d)

istitik2007@gmail.com

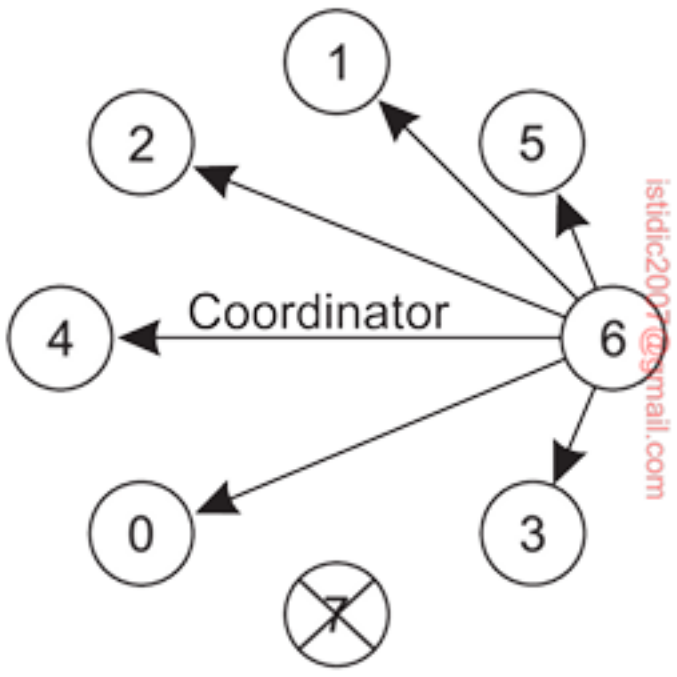
4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán bắt nạt (The Bully Algorithm)

- Ví dụ minh họa cách thuật toán hoạt động:

+ P_6 thắng cử và gửi thông báo COORDINATOR đến toàn bộ hệ thống, thông báo rằng từ bây giờ nó là điều phối viên mới.

+ P_4 tiếp tục công việc mà nó đang làm dở, nhưng lần này với P_6 làm điều phối viên.



(e)

istidic2007@gmail.com

4) Thuật toán bầu chọn (Election Algorithms)

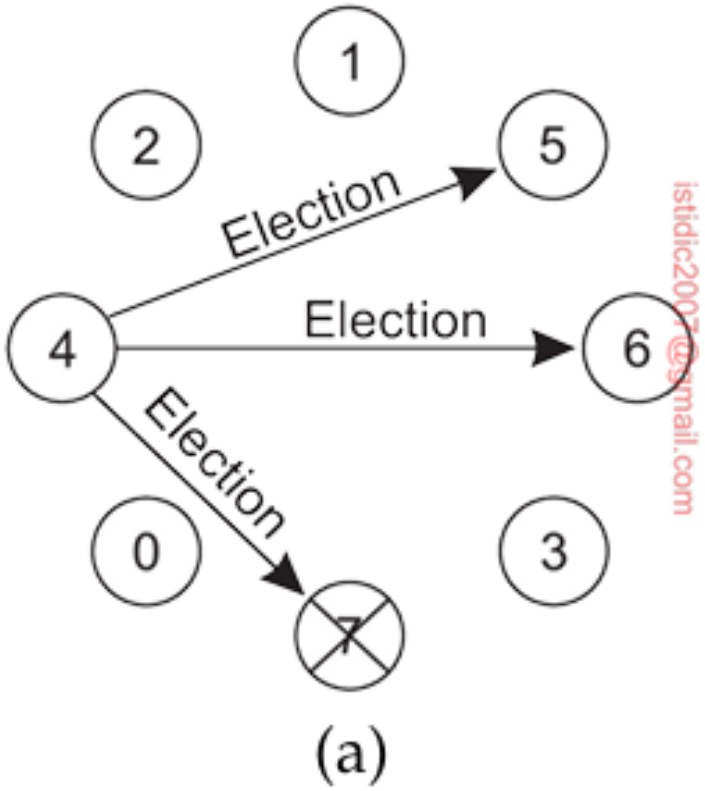
- Thuật toán bắt nạt (The Bully Algorithm)

- Khi tiến trình bị sập quay trở lại:

+ Nếu P_7 khởi động lại, nó sẽ gửi thông điệp COORDINATOR đến tất cả các tiến trình khác, bắt nạt chúng để giành lại quyền điều phối.

+ Vì P_7 có số định danh cao nhất, nó sẽ chiến thắng cuộc bầu chọn và trở thành điều phối viên mới.

+ Các tiến trình khác bị buộc phải chấp nhận sự thay đổi này.



istidic2007@gmail.com

4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán bắt nạt (The Bully Algorithm)

- Tổng quan thuật toán:

+ Thuật toán bắt nạt đảm bảo rằng tiến trình có số định danh cao nhất luôn chiến thắng và trở thành điều phối viên.

+ Khi một tiến trình bị sập khởi động lại, nó có thể giành lại quyền điều phối nếu nó có số định danh cao nhất.

+ Thuật toán này giúp hệ thống tự động phục hồi sau lỗi và tiếp tục hoạt động ổn định.

4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán vòng(The Ring Algorithm)

- Một giải pháp nổi tiếng khác để bầu chọn điều phối viên là thuật toán vòng (ring algorithm).

- Thuật toán vòng bắt đầu quá trình lựa chọn điều phối viên khi bất kỳ quy trình nào được kết nối trong vòng nhận thấy điều phối viên không hoạt động. Sau đó, quy trình khởi tạo chuẩn bị một thông báo bầu cử có chứa số quy trình của riêng nó và sau đó gửi thông báo này đến người kế nhiệm của nó trong vòng.

- Nếu người kế nhiệm không hoạt động thì nó sẽ bỏ qua người kế nhiệm đó và gửi đến người tiếp theo.

4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán vòng(The Ring Algorithm)

- Không giống như một số thuật toán vòng khác, thuật toán này không sử dụng token.

- Chúng ta giả định rằng mỗi tiến trình biết tiến trình kế tiếp của nó trong vòng lặp.

- Tiến trình có ID cao nhất sẽ thắng cuộc bầu cử. Tiến trình khởi tạo sau đó sẽ gửi một thông điệp khác vào vòng về tiến trình chiến thắng.

4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán vòng(The Ring Algorithm)

- Cách hoạt động của thuật toán:

1. Khi một tiến trình nhận thấy điều phối viên hiện tại không hoạt động, nó sẽ tạo một thông điệp ELECTION chứa ID của chính nó và gửi tin nhắn đến tiến trình kế tiếp.

2. Nếu tiến trình kế tiếp đang bị sập, nó sẽ bỏ qua tiến trình đó và tiếp tục gửi đến tiến trình tiếp theo trong vòng.

3. Mỗi tiến trình nhận được thông điệp ELECTION sẽ thêm ID của nó vào danh sách, xem như ứng cử viên cho vị trí điều phối viên.

4. Khi thông điệp quay trở lại tiến trình đã khởi tạo nó, tiến trình đó nhận ra sự kiện này khi thấy ID của mình trong danh sách.

4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán vòng(The Ring Algorithm)

- Cách hoạt động của thuật toán:

5. Lúc này, thông điệp ELECTION được chuyển thành thông điệp COORDINATOR, xác định tiến trình có ID cao nhất là điều phối viên mới.

6. Thông điệp COORDINATOR được gửi lại qua vòng để thông báo kết quả và danh sách các tiến trình trong vòng.

7. Khi thông điệp này đi hết một vòng, nó bị xóa và hệ thống tiếp tục hoạt động bình thường.

4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán vòng(The Ring Algorithm)

- Ví dụ về cách hoạt động của thuật toán:

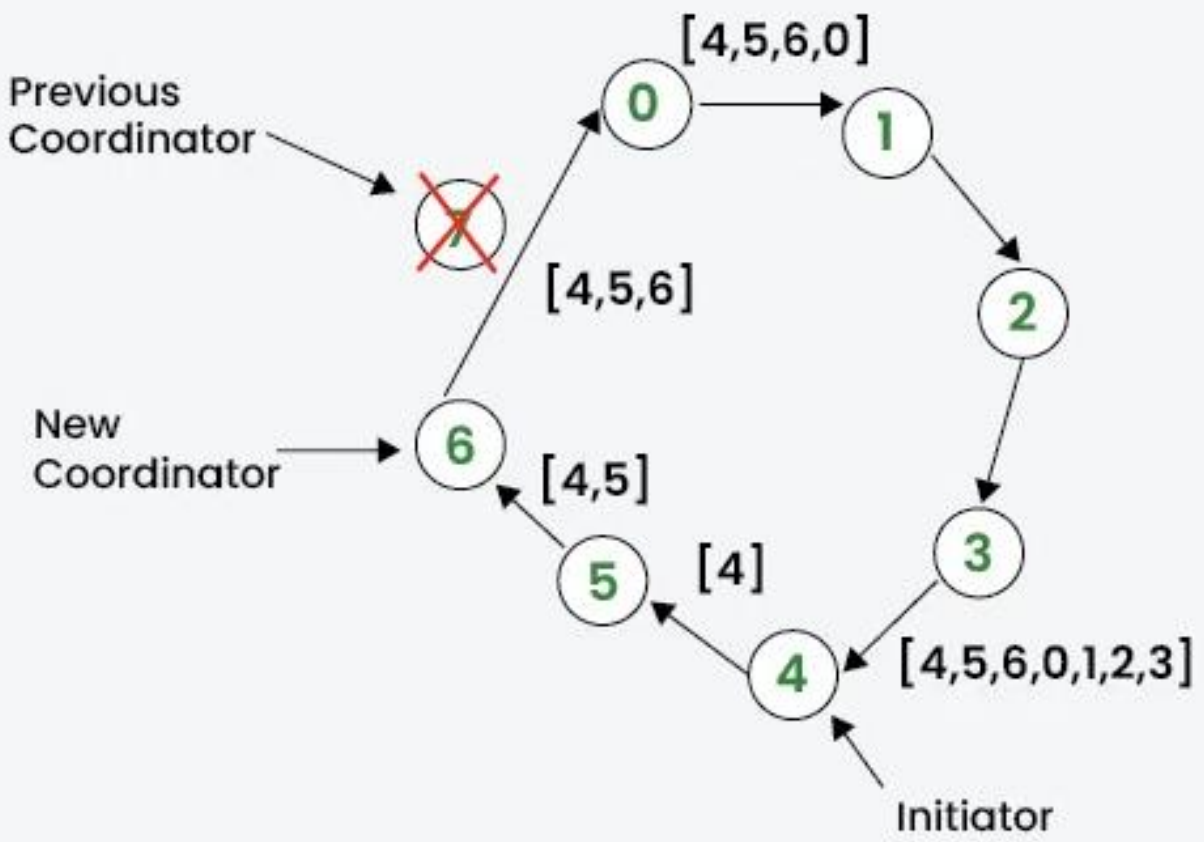
+ Giả sử có một hệ thống gồm 8 tiến trình, từ P_0 đến P_7.

+ Tiến trình P_7 bị sập, và P_4 phát hiện ra điều này và trở thành tiến trình khởi tạo cuộc bầu cử.

+ Tiến trình tạo thông điệp ELECTION và gửi đi trong vòng

+ Cuối cùng, thông điệp bầu cử hoàn thành vòng lặp, P_4 chuyển đổi chúng thành thông điệp COORDINATOR.

➡ Tiến trình P_6 có ID cao nhất (=6) và trở thành điều phối viên mới



4) Thuật toán bầu chọn (Election Algorithm

- Thuật toán vòng(The Ring Algorithm)

- Ví dụ về cách hoạt động của thuật toán:

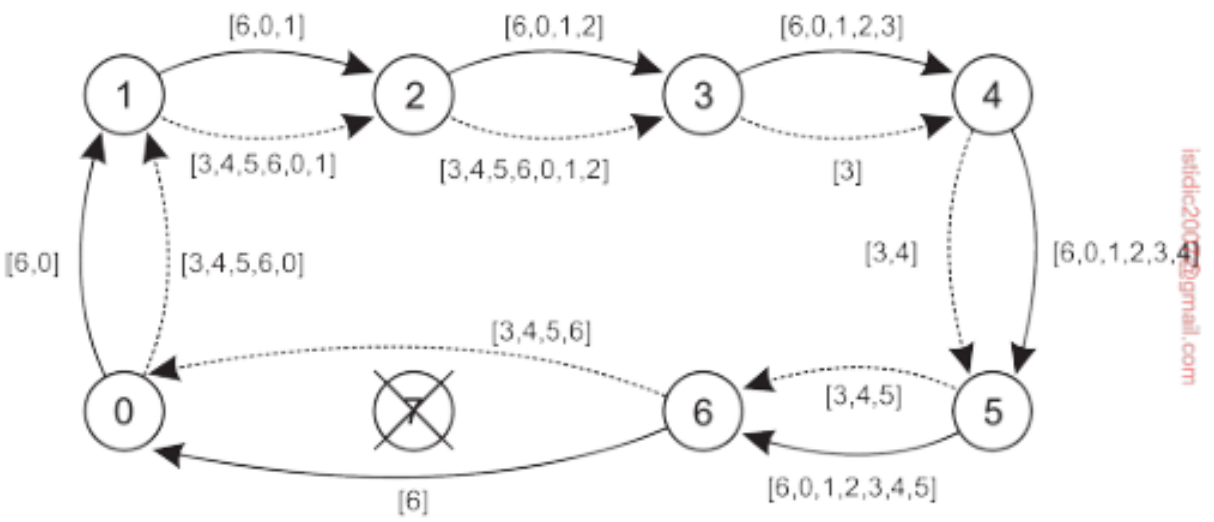
+ Giả sử có một hệ thống gồm 8 tiến trình, từ P_0 đến P_7.

+ Tiến trình P7 bị sập, và P3 cùng P6 phát hiện ra điều này cùng lúc.

+ Cả hai tiến trình tạo thông điệp ELECTION và gửi đi trong vòng, hoạt động độc lập với nhau.

+ Cuối cùng, cả hai thông điệp đều hoàn thành vòng lặp, và cả P3 và P6 chuyển đổi chúng thành thông điệp COORDINATOR.

➡ Tiến trình P_6 có ID cao nhất (=6) và trở thành điều phối viên mới



+ Danh sách tiến trình trong cả hai thông điệp giống nhau và theo cùng thứ tự, nên không có vấn đề gì xảy ra.

+ Việc có nhiều thông điệp ELECTION cùng lúc không gây hại, chỉ tiêu tốn một lượng băng thông nhỏ, nhưng không bị xem là lãng phí.

Election algorithm

Gossip-based coordination

Distributed event matching

Location systems

4) Thuật toán bầu chọn (Election Algorithms)

- Sự khác biệt giữa thuật toán vòng và thuật toán bắt nạt

Tham số	Thuật toán vòng	Thuật toán bắt nạt
Chiến lược truyền thông	Trong thuật toán vòng, giao tiếp giữa các tiến trình diễn ra theo cấu trúc vòng logic.	Trong thuật toán bắt nạt, giao tiếp trực tiếp diễn ra giữa tất cả các quy trình.
Quá trình bầu cử	Quá trình bầu cử được thực hiện bằng cách truyền tin nhắn dọc theo chiếc vòng.	Quá trình bầu cử diễn ra bằng cách nhắn tin trực tiếp đến các tiến trình có ID tiến trình cao hơn.
Tuyển chọn Điều phối viên mới	Tiến trình có ID cao nhất trong vòng sẽ trở thành Điều phối viên mới.	Tiến trình có ID cao nhất trong số các tiến trình phản hồi sẽ trở thành Điều phối viên mới.
Cơ chế phản ứng	Tin nhắn sẽ được chuyển tiếp hoặc phản hồi bằng ID cao hơn.	Một tiến trình phản hồi tiến trình khác bằng cách gửi tin nhắn “OK”

4) Thuật toán bầu chọn (Election Algorithms)

- Sự khác biệt giữa thuật toán vòng và thuật toán bắt nạt

Tham số	Thuật toán vòng	Thuật toán bắt nạt
Độ phức tạp	Thuật toán vòng ít phức tạp hơn.	Thuật toán bắt nạt phức tạp hơn thuật toán vòng tròn.
Khả năng mở rộng	Có khả năng mở rộng hơn khi số lượng quy trình nhỏ.	Có thể hoạt động tốt khi số lượng tiến trình nhiều hơn.
Tuyển chọn Điều phối viên mới	Tiến trình có ID cao nhất trong vòng sẽ trở thành Điều phối viên mới.	Tiến trình có ID cao nhất trong số các tiến trình phản hồi sẽ trở thành Điều phối viên mới.
Sử dụng tài nguyên	Thuật toán vòng yêu cầu ít tài nguyên hơn cho việc giao tiếp vì các thông điệp được chia sẻ giữa các quy trình được kết nối trong vòng.	Thuật toán Bully làm tăng chi phí truyền thông vì các tin nhắn được gửi trực tiếp từ quy trình này sang quy trình khác.

4) Thuật toán bầu chọn (Election Algorithms)

- Bầu chọn Leader trong ZooKeeper

- Tổng quan về ZooKeeper:

+ ZooKeeper là một dịch vụ điều phối tập trung về mặt logic.

+ Thực tế, ZooKeeper duy trì một nhóm nhỏ các máy chủ, gọi là ensemble.

+ Từ góc nhìn của client, một ensemble trông như một máy chủ duy nhất.

+ Trong ZooKeeper, ensemble được điều phối bởi một server duy nhất, gọi là leader.

+ Các máy chủ còn lại được gọi là followers, hoạt động như bản sao dự phòng để thay thế leader nếu nó bị lỗi.

+ Nếu leader gặp sự cố, một trong các followers sẽ được bầu chọn làm leader mới.

+ Client không nhận thấy sự thay đổi này vì quá trình bầu chọn leader diễn ra nhanh chóng.

4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán bầu chọn leader trong ZooKeeper

- Nguyên tắc chung:

+ Bầu chọn leader trong ZooKeeper không giống như sử dụng ZooKeeper để bầu chọn leader cho một hệ thống khác.

+ Điều quan trọng là luôn chọn được một leader duy nhất và đảm bảo đa số các máy chủ còn hoạt động bình thường.

4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán bầu chọn leader trong ZooKeeper

- ZooKeeper sử dụng một phiên bản đơn giản của thuật toán Bully, hoạt động như sau:

1. Mỗi máy chủ s trong hệ thống có:

+ ID riêng biệt: id(s)

+ Bộ đếm giao dịch tăng dần tx(s), đại diện cho giao dịch mới nhất mà nó xử lý

2. Leader hiện tại thực hiện giao dịch trên không gian lưu trữ cục bộ của nó, sau đó yêu cầu các followers cập nhật theo.

3. Khi một follower sss nghi ngờ leader bị lỗi, nó sẽ gửi thông điệp ELECTION đến tất cả các máy chủ khác, kèm theo cặp giá trị:

+ voteID = id(s) (ID của chính nó)

+ voteTX = tx(s) (Giao dịch mới nhất mà nó biết)

4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán bầu chọn leader trong ZooKeeper

- ZooKeeper sử dụng một phiên bản đơn giản của thuật toán Bully, hoạt động như sau:

4. Trong quá trình bầu chọn, mỗi máy chủ s duy trì hai biến:

+ leader(s): ID của máy chủ mà s cho là có khả năng trở thành leader. Ban đầu, leader(s) = id(s).

+ lastTX(s): Giao dịch mới nhất mà s biết, ban đầu là tx(s)

5. Khi một máy chủ s∗ nhận được thông điệp (voteID, voteTX), nó thực hiện

+ Nếu lastTX(s∗) < voteTX:

Máy chủ khác có giao dịch mới hơn, nên cập nhật lại leader:

o leader(s∗) ← voteID.

o lastTX(s∗)←voteTX.

4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán bầu chọn leader trong ZooKeeper

- ZooKeeper sử dụng một phiên bản đơn giản của thuật toán Bully, hoạt động như sau:

  + Nếu lastTX(s∗) = voteTX và leader(s∗) < voteID:

    Hai máy chủ có giao dịch mới nhất giống nhau, nhưng ID của máy chủ mới cao hơn, nên cập nhật lại leader:

    o leader(s∗) ← voteID.

6. Mỗi khi một máy chủ nhận thông điệp (voteID, voteTX), nó có thể cập nhật thông tin leader.

7. Nếu một máy chủ tin rằng nó xứng đáng làm leader, nhưng chưa gửi thông báo nào, nó sẽ gửi thông điệp broadcast với (id(s^), tx(s^)) để khẳng định vị trí leader của mình.

8. Sau khi leader mới được bầu chọn, tất cả các máy chủ sẽ đồng bộ với giao dịch mới nhất của leader.

4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán bầu chọn leader trong ZooKeeper

- Ví dụ minh họa cách thuật toán hoạt động:

1.Mỗi tiến trình (server) có các thuộc tính chính:

+ txID: Giao dịch gần nhất mà tiến trình đã xử lý.

+ leader: Tiến trình mà nó tin là có thể trở thành leader.

+ lastTX: Giao dịch gần nhất mà tiến trình nhận biết.

+ noleader: Biến boolean để xác định tiến trình có còn trong cuộc đua làm leader không.

```
class Process:
  def __init__(self, chanID, procID, procIDSet, initTX):
    self.txID       = initTX       # Your own most recent transaction
    self.leader     = self.procID  # Who you believe may become leader
    self.lastTX     = self.txID    # What is the most recent transaction
    self.noleader   = False        # Are you still in the race for leader?
```

4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán bầu chọn leader trong ZooKeeper

2. Nhận và xử lý tin nhắn bầu cử (ELECTION):

+ Khi một tiến trình phát hiện cần bầu leader, nó gửi một tin nhắn ELECTION chứa ID của nó và txID.

+ Nếu tiến trình nhận được tin nhắn có voteTX (giao dịch của sender) mới hơn lastTX của chính nó, thì nó cập nhật leader nghi ngờ (voteID).

+ Nếu voteTX bằng với lastTX nhưng voteID lớn hơn leader, thì nó cập nhật leader nghi ngờ.

+ Nếu tiến trình hiện tại có procID lớn hơn voteID và có txID mới hơn hoặc bằng voteTX mà chưa bị loại khỏi cuộc đua (noleader chưa được đặt thành True), thì nó thông báo với các tiến trình khác rằng nó có thể là leader bằng cách gửi tin nhắn LEADER.

```
def receive(self):
  while True:
    msg              = self.chan.recvFrom(self.otherProcs)
    sender, payload = msg[0], msg[1]
    if payload[0] == ELECTION: # A process started an election
      voteID, voteTX = payload[1], payload[2]

      if self.lastTX < voteTX: # You're not up to date on most recent transaction
        self.leader = voteID   # Record the suspected leader
        self.lastTX = voteTX   # As well as the likely most recent transaction

      elif (self.lastTX == voteTX) and (self.leader < voteID): # Wrong leader
        self.leader = voteID   # Update your suspected leader

      elif (self.procID > voteID) and (self.txID >= voteTX) and (not self.noleader):
        # At this point, you may very well be the new leader (having a sufficiently
        # high process identifier as well as perhaps the most recent transaction).
        # No one has told you so far that you could not be leader. Tell the others.
        self.chan.sendTo(self.otherProcs, (LEADER, self.procID, self.txID))

    if payload[0] == LEADER:
      # Check if the sender should indeed be leader
      if ((self.lastTX < payload[2]) or
          ((self.lastTX == payload[2]) and (self.leader <= payload[1]))):
        # The sender is more up-to-date than you, or is equally up-to-date but
        # has a higher process identifier. Declare yourself follower.
        self.chan.sendTo(sender, (FOLLOWER, self.procID))
      else:
        # Sender is wrong: you have information that the sender based its decision
        # on outdated information
        self.chan.sendTo(sender, (NOLEADER))
```

4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán bầu chọn leader trong ZooKeeper

3. Nhận và xử lý tin nhắn LEADER:

+ Khi nhận tin nhắn LEADER, tiến trình kiểm tra xem sender có phải leader hợp lệ hay không:

+ Nếu lastTX của sender lớn hơn hoặc bằng lastTX của tiến trình, và leader của sender có procID lớn hơn hoặc bằng leader hiện tại, thì tiến trình tự đặt mình thành follower bằng cách gửi tin nhắn FOLLOWER.

+ Nếu tiến trình phát hiện sender dựa trên thông tin cũ, nó phản hồi với tin nhắn NOLEADER

```
def receive(self):
  while True:
    msg              = self.chan.recvFrom(self.otherProcs)
    sender, payload = msg[0], msg[1]
    if payload[0] == ELECTION: # A process started an election
      voteID, voteTX = payload[1], payload[2]

      if self.lastTX < voteTX: # You're not up to date on most recent transaction
        self.leader = voteID   # Record the suspected leader
        self.lastTX = voteTX   # As well as the likely most recent transaction

      elif (self.lastTX == voteTX) and (self.leader < voteID): # Wrong leader
        self.leader = voteID   # Update your suspected leader

      elif (self.procID > voteID) and (self.txID >= voteTX) and (not self.noleader):
        # At this point, you may very well be the new leader (having a sufficiently
        # high process identifier as well as perhaps the most recent transaction).
        # No one has told you so far that you could not be leader. Tell the others.
        self.chan.sendTo(self.otherProcs, (LEADER, self.procID, self.txID))

    if payload[0] == LEADER:
      # Check if the sender should indeed be leader
      if ((self.lastTX < payload[2]) or
          ((self.lastTX == payload[2]) and (self.leader <= payload[1]))):
        # The sender is more up-to-date than you, or is equally up-to-date but
        # has a higher process identifier. Declare yourself follower.
        self.chan.sendTo(sender, (FOLLOWER, self.procID))
      else:
        # Sender is wrong: you have information that the sender based its decision
        # on outdated information
        self.chan.sendTo(sender, (NOLEADER))
```

4) Thuật toán bầu chọn (Election Algorithms)

- Bầu chọn leader trong môi trường không dây

- Các thuật toán bầu chọn truyền thống thường dựa trên các giả định không thực tế trong môi trường không dây. Ví dụ:

+ Chúng giả định rằng việc truyền tin nhắn luôn đáng tin cậy.

+ Giả định rằng cấu trúc mạng không thay đổi.

- Tuy nhiên, những giả định này không đúng trong hầu hết các môi trường không dây, đặc biệt là khi làm việc với các thiết bị di động.

- Hiện có rất ít giao thức bầu chọn được phát triển để hoạt động hiệu quả trong môi trường di động. Vasudevan et al. [2004] đã đề xuất một giải pháp có thể xử lý:

+ Các nút bị lỗi

+ Mạng bị phân đoạn

➡ Điểm quan trọng trong giải pháp của họ là chọn ra leader tốt nhất, thay vì chọn một leader ngẫu nhiên như các thuật toán trước đây.

4) Thuật toán bầu chọn (Election Algorithms)

- Mô tả thuật toán bầu chọn leader trong mạng không dây

- Quá trình bầu chọn leader diễn ra như sau:

1. Bắt đầu cuộc bầu chọn:

+ Một nút bất kỳ trong mạng (gọi là nguồn, source) gửi tin nhắn ELECTION đến các nút lân cận (nằm trong phạm vi của nó).

2. Lan truyền tin nhắn bầu cử:

+ Khi một nút nhận được tin nhắn ELECTION lần đầu tiên, nó sẽ:

Chỉ định nút gửi tin làm cha (parent) của nó.

Gửi tiếp tin nhắn ELECTION đến tất cả các nút lân cận trừ cha của nó.

+ Nếu một nút nhận được tin nhắn ELECTION từ một nút khác ngoài cha nó, nó chỉ xác nhận đã nhận tin nhắn mà không gửi tiếp.

4) Thuật toán bầu chọn (Election Algorithms)

- Mô tả thuật toán bầu chọn leader trong mạng không dây

- Quá trình bầu chọn leader diễn ra như sau:

3. Thu thập thông tin từ các nút con:

+ Khi một nút đã chỉ định cha, nó sẽ gửi tiếp tin nhắn ELECTION đến các nút lân cận (trừ cha).

+ Sau khi gửi tin, nó đợi xác nhận từ các nút con trước khi xác nhận tin nhắn từ cha.

+ Hệ quả quan trọng:

Nếu tất cả các nút lân cận đã có cha, thì nút đó là nút lá và có thể nhanh chóng báo cáo thông tin lại cho cha nó.

Trong quá trình báo cáo, nút lá cũng gửi kèm thông tin về năng lượng pin và tài nguyên của nó.

4) Thuật toán bầu chọn (Election Algorithms)

- Mô tả thuật toán bầu chọn leader trong mạng không dây

- Quá trình bầu chọn leader diễn ra như sau:

4. Lựa chọn leader tốt nhất:

+ Cha của nút lá sẽ so sánh thông tin tài nguyên của tất cả các nút con để tìm ra nút có năng lực tốt nhất.

+ Thông tin về nút tốt nhất sẽ được truyền ngược lên cha của nó, tiếp tục như vậy cho đến khi về đến nguồn.

+ Nguồn cuối cùng biết được nút nào là leader tốt nhất và sẽ phát thông tin này đến toàn bộ mạng.

4) Thuật toán bầu chọn (Election Algorithms)

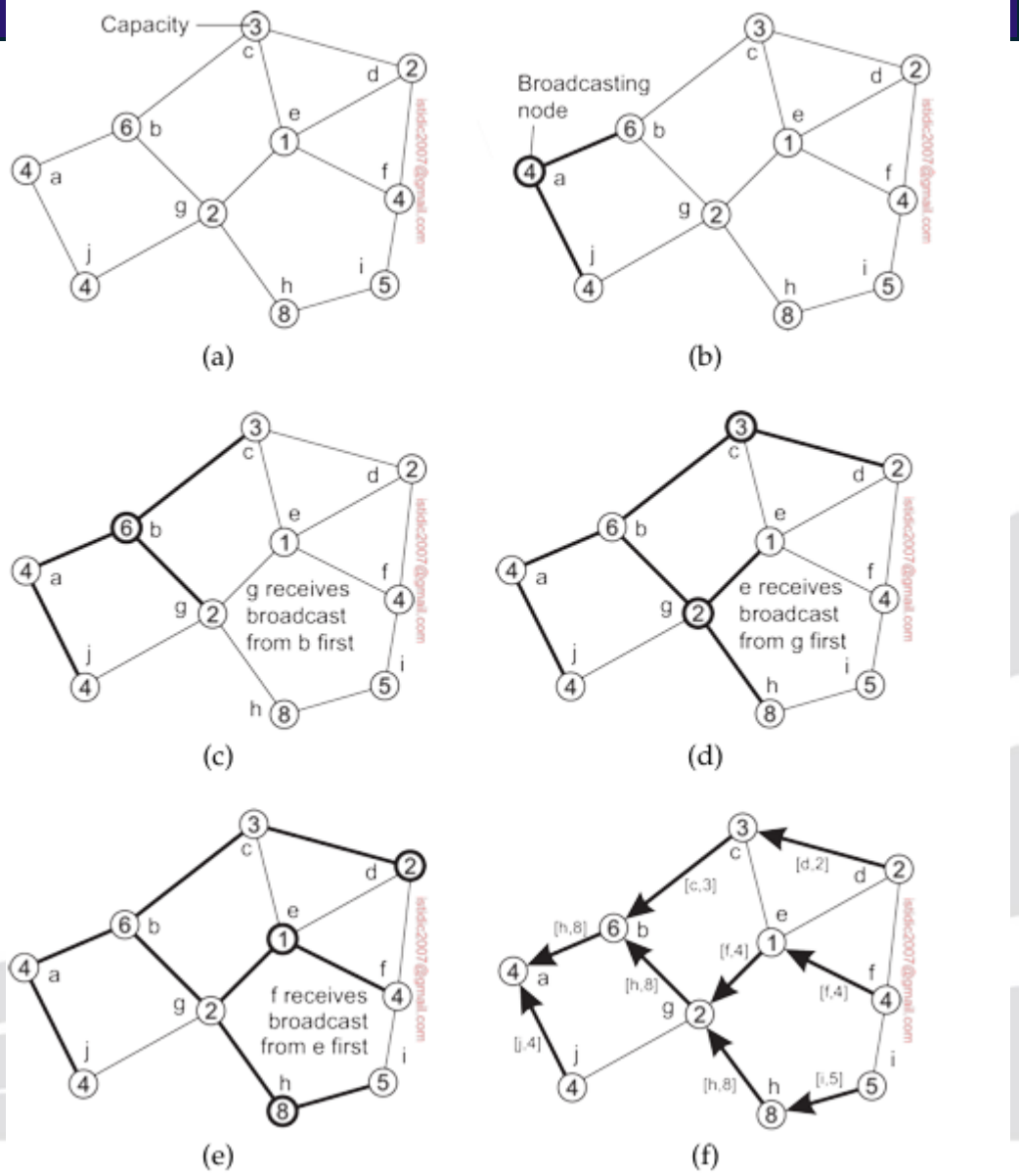
- Thuật toán bầu chọn leader trong mạng không dây

- Ví dụ minh họa cách thuật toán hoạt động:

+ Nút a là nguồn, bắt đầu cuộc bầu chọn.

+ Tin nhắn ELECTION lan truyền đến các nút khác (b, j, ...).

+ Các nút báo cáo lại thông tin tài nguyên của con tốt nhất lên cha, cuối cùng nguồn biết được nút nào là leader tốt nhất và thông báo cho toàn mạng.



4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán bầu chọn leader trong mạng không dây

- Ví dụ minh họa cách thuật toán hoạt động:

(a) Cấu trúc mạng ban đầu

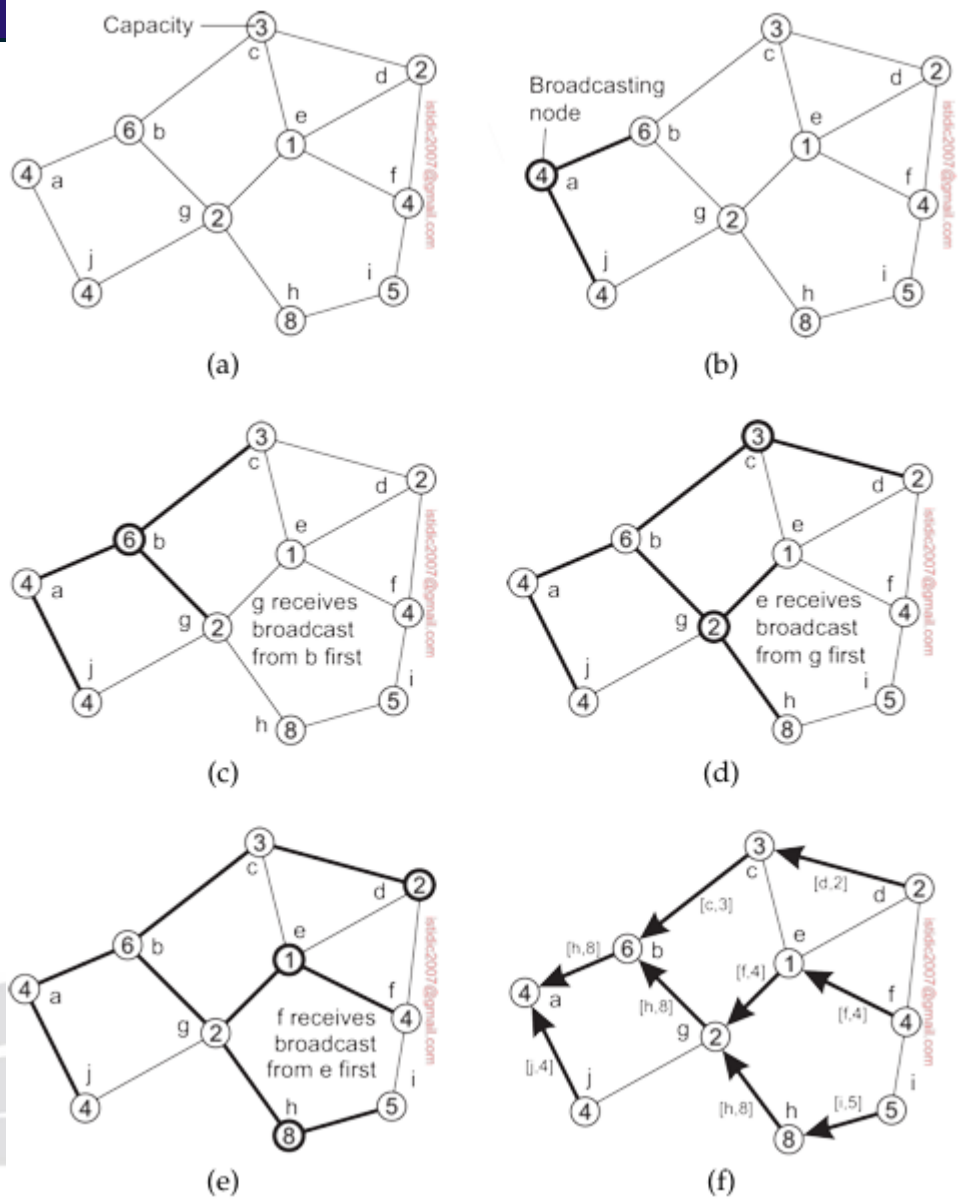
+ Đây là một mạng được biểu diễn dưới dạng đồ thị, trong đó mỗi nút có một chỉ số năng lực (capacity) được hiển thị trong hình tròn bên cạnh.

+ Các cạnh giữa các nút thể hiện khả năng kết nối giữa các thiết bị.

(b) Nút bắt đầu quá trình bầu chọn

+ Nút "a" (nút có chỉ số năng lực là 4) bắt đầu quá trình bầu chọn bằng cách gửi tin nhắn ELECTION đến các nút lân cận.

+ Đây là nút được chọn làm nguồn khởi tạo.



4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán bầu chọn leader trong mạng không dây

- Ví dụ minh họa cách thuật toán hoạt động:

(c) Lan truyền tin nhắn bầu chọn (Bước 1)

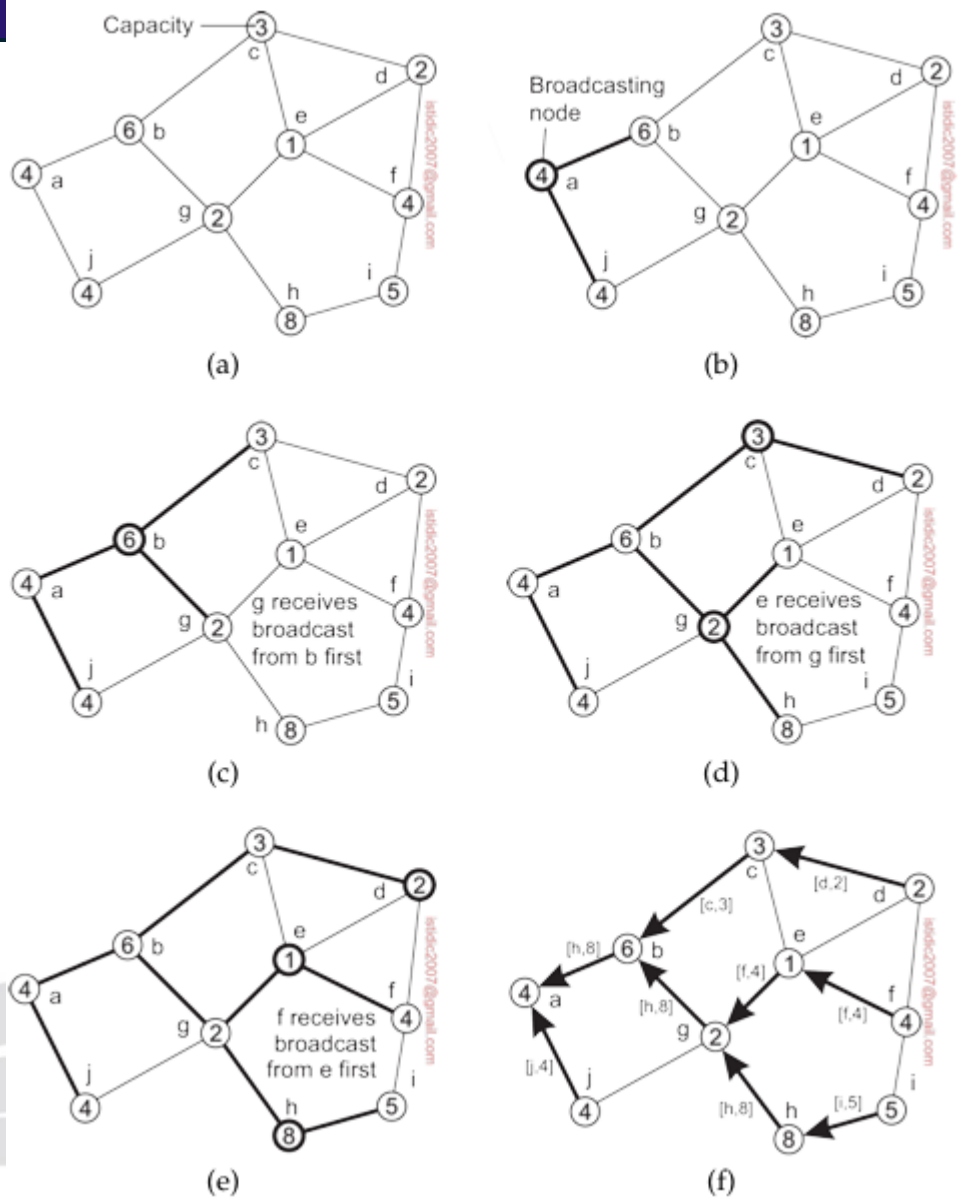
+ Nút b nhận được tin nhắn ELECTION từ a và tiếp tục truyền đến các nút khác.

+ Nút g nhận tin nhắn từ b trước tiên, điều này có nghĩa là nó sẽ coi b là cha của nó.

(d) Lan truyền tin nhắn bầu chọn (Bước 2)

+ Nút g tiếp tục gửi tin nhắn đến các nút khác.

+ Nút e nhận tin từ g đầu tiên, vì vậy nó chọn g làm cha của nó.



4) Thuật toán bầu chọn (Election Algorithms)

- Thuật toán bầu chọn leader trong mạng không dây

- Ví dụ minh họa cách thuật toán hoạt động:

(e) Lan truyền tin nhắn bầu chọn (Bước 3)

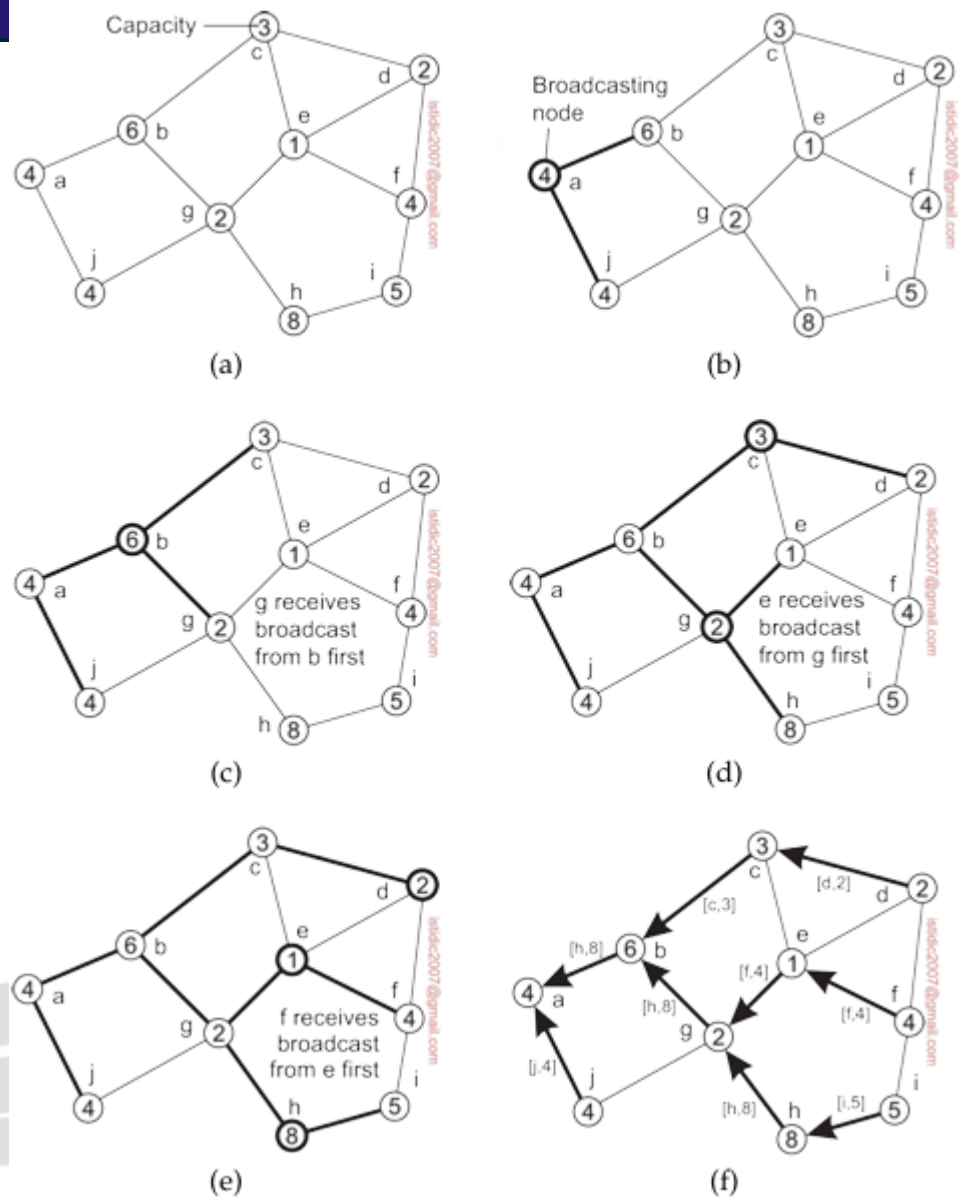
+ Nút f nhận tin nhắn từ e trước tiên, và chọn e làm cha của nó.

(f) Thu thập thông tin và quyết định leader

+ Các nút lá bắt đầu gửi thông tin ngược lại cho cha của chúng.

+ Các giá trị [h:8], [f:4], [d:2] biểu diễn khả năng của các nút, được tổng hợp lại qua từng bước.

+ Quá trình này giúp hệ thống xác định nút có năng lực cao nhất để làm leader.



5) Phối hợp dựa trên Gossip

Xem xét một số ví dụ quan trọng về việc áp dụng cơ chế gossiping. Cụ thể:

- Tổng hợp dữ liệu (aggregation),
- Lấy mẫu ngang hàng trên quy mô lớn (large-scale peer sampling)
- Xây dựng lớp phủ mạng (overlay construction).

Clock synchronization
Logical clocks
Mutual exclusion
Election algorithm
Gossip-based coordination
Distributed event matching
Location systems

5.1) Tổng hợp dữ liệu (aggregation)

Gossiping trong hệ thống phân tán:

- Kỹ thuật phổ biến trong các giao thức lan truyền (epidemic protocols).
- Dùng để truyền bá bản cập nhật và phát hiện các nút có ít liên kết rộng (wide-area links). Để sau đó áp dụng kỹ thuật gossiping có hướng (directional gossiping).

Ứng dụng thu thập hoặc tổng hợp thông tin *[Jelasity et al., 2005]*

Ví dụ: Mỗi nút P_i có một giá trị ban đầu ngẫu nhiên v_i

Khi hai nút P_i và P_j liên hệ, giá trị được cập nhật như sau:

$$v_i, v_j \leftarrow \frac{v_i + v_j}{2}$$

5.1) Tổng hợp dữ liệu (aggregation)

Ví dụ: Mỗi nút P_i có một giá trị ban đầu ngẫu nhiên v_i

Khi hai nút P_i và P_j liên hệ, giá trị được cập nhật như sau:

$$v_i, v_j \leftarrow \frac{v_i + v_j}{2}$$

Sau mỗi lần trao đổi, cả P_i và P_j sẽ có cùng một giá trị.

Thực tế, ta nhận thấy rằng cuối cùng tất cả các nút sẽ hội tụ về cùng một giá trị, chính là trung bình của tất cả các giá trị ban đầu.

Tốc độ lan truyền của quá trình này cũng có tính chất exponential (theo cấp số nhân).

Clock synchronization
Logical clocks
Mutual exclusion
Election algorithm
Gossip-based coordination
Distributed event matching
Location systems

5.1) Tổng hợp dữ liệu (aggregation)

Ứng dụng của việc tính trung bình:

Xét tình huống sau: tất cả các nút P_i đều đặt giá trị $v_i = 0$, ngoại trừ P_1 có giá trị $v_1 = 1$:

$$v_i = \begin{cases} 1, & nếu\ i = 1 \\ 0, & ngược\ lại \end{cases}$$

Nếu hệ thống có N nút, thì cuối cùng tất cả các nút sẽ tính được giá trị trung bình, tức là $\frac{1}{N}$.

Nhờ đó, mỗi nút P_i có thể ước lượng kích thước của toàn bộ hệ thống bằng công thức:

$$size\ of\ the\ system \approx \frac{1}{v_i}$$

5.1) Tổng hợp dữ liệu (aggregation)

Ứng dụng của việc tính trung bình:

Ý nghĩa:

- Đây là một cách đơn giản nhưng hiệu quả để ước tính số lượng nút trong một hệ thống phân tán mà không cần có một thực thể trung tâm giám sát.
- Cơ chế này phù hợp với các hệ thống có quy mô lớn, nơi các nút có thể tham gia hoặc rời đi liên tục.

5.1) Tổng hợp dữ liệu (aggregation)

Ứng dụng của việc tính trung bình:

Việc tính toán giá trị trung bình trở nên khó khăn khi các nút thường xuyên tham gia hoặc rời khỏi hệ thống.

Một giải pháp thực tiễn khắc phục vấn đề này là sử dụng epoch (chu kỳ đặt lại quá trình tính toán trung bình).

- Giả sử nút P_1 là một nút ổn định trong hệ thống. Nó sẽ khởi động một epoch mới sau một khoảng thời gian nhất định.
- Khi một nút P_i phát hiện một epoch mới lần đầu tiên, nó sẽ:
  - Đặt lại giá trị $v_i = 0$.
  - Bắt đầu lại quá trình tính toán giá trị trung bình từ đầu.

5.1) Tổng hợp dữ liệu (aggregation)

Ngoài phép tính trung bình, ta cũng có thể tính toán các kết quả khác bằng phương pháp tương tự.

Ví dụ: Thay vì cố định một nút như P_1 để khởi động quá trình tính trung bình, ta có thể chọn một nút ngẫu nhiên bằng cách:

- Mỗi nút P_i ban đầu gán giá trị v_i là một số ngẫu nhiên trong khoảng $(0,1]$ và lưu giá trị này vĩnh viễn dưới dạng m_i .
- Khi hai nút P_i và P_j trao đổi thông tin, chúng cập nhật giá trị theo ct:

$$v_i, v_j \leftarrow \max\{v_i, v_j\}$$

Tất cả các nút sẽ hội tụ về cùng một giá trị lớn nhất ban đầu $\max\{m_1, m_2, \dots, m_N\}$.

Được sử dụng để tìm giá trị lớn nhất trong một hệ thống phân tán mà không cần một trung tâm điều phối.

5.1) Tổng hợp dữ liệu (aggregation)

Cạnh tranh để trở thành nút khởi tạo tính trung bình

- Mỗi nút P_i có giá trị ban đầu m_i được lưu vĩnh viễn và một giá trị biến đổi v_i.
- Nếu $m_i < v_i$, nút đó sẽ thua trong quá trình cạnh tranh để trở thành nút khởi tạo tính trung bình.
- Chỉ có một nút duy nhất chiến thắng và giữ quyền khởi tạo tính trung bình.

-> Dễ xác định nút thua, nhưng khó xác định nút thắng vì còn có thể có những giá trị lớn hơn chưa được gửi đến.

5.1) Tổng hợp dữ liệu (aggregation)

Cạnh tranh để trở thành nút khởi tạo tính trung bình

Giải pháp: Giả định chiến thắng cho đến khi bị loại bỏ

- Mỗi nút mặc định cho rằng mình là người chiến thắng.
- Nếu nhận được một giá trị lớn hơn, nó chấp nhận thua cuộc và đặt lại giá trị tính toán trung bình về 0.

Lưu ý:

- Quá trình tìm giá trị lớn nhất và tính trung bình có thể diễn ra đồng thời trong hệ thống.
- Điều này giúp hệ thống xử lý nhiều loại thông tin khác nhau mà không cần một cơ chế đồng bộ phức tạp.

5.2) Dịch vụ lấy mẫu ngang hàng (Peer-Sampling Service)

Một khía cạnh quan trọng trong các giao thức lan truyền (epidemic protocols) là khả năng để một nút P **chọn ngẫu nhiên** một nút Q từ tất cả các nút có sẵn trong mạng.

Vấn đề đặt ra:

- Nếu mạng có **hàng nghìn nút**, làm thế nào P có thể chọn một nút ngẫu nhiên khi nó **không có cái nhìn tổng thể** về toàn bộ hệ thống?
- Với **mạng nhỏ**, ta có thể sử dụng một **dịch vụ trung tâm** lưu trữ danh sách tất cả các nút tham gia.
- Tuy nhiên, cách tiếp cận này **không thể mở rộng** cho các mạng lớn vì nó đòi hỏi một hệ thống tập trung, gây ra vấn đề **tắc nghẽn và đơn điểm lỗi (single point of failure)**.

5.2) Dịch vụ lấy mẫu ngang hàng (Peer-Sampling Service)

PSS có thể được xây dựng bằng chính một giao thức lan truyền (epidemic protocol), mặc dù điều này nghe có vẻ trực giác không rõ ràng.

Cách tiếp cận của Jelasity et al. (2007):

- Mỗi nút duy trì một danh sách chứa *c* nút láng giềng, trong đó **mỗi nút láng giềng lý tưởng là một nút ngẫu nhiên đang hoạt động trong mạng**.
- Danh sách này được gọi là **partial view** (danh sách láng giềng cục bộ), vì nó chỉ chứa một phần nhỏ của toàn bộ mạng.

5.2) Dịch vụ lấy mẫu ngang hàng (Peer-Sampling Service)

Cách xây dựng partial view:

- Các nút **thường xuyên trao đổi thông tin** trong danh sách partial view với nhau.
- Mỗi mục trong danh sách lưu thông tin về một nút khác trong mạng và có một **chỉ số tuổi (age)** để chỉ ra thông tin này đã cũ bao lâu.
- **Hai luồng (threads)** được sử dụng để duy trì danh sách này (Hình 5.25).

Lợi ích của cách tiếp cận này

- Không cần một trung tâm điều phối nhưng vẫn có thể chọn các nút ngẫu nhiên hiệu quả.
- Phù hợp với hệ thống phân tán lớn, nơi các nút tham gia và rời khỏi mạng liên tục.

5.2) Dịch vụ lấy mẫu ngang hàng (Peer-Sampling Service)

*Các thao tác lựa chọn trong PSS: Hệ thống PSS sử dụng ba thao tác chính để duy trì **partial view** và đảm bảo tính ngẫu nhiên khi chọn các nút trong mạng:*

- **selectPeer:** Chọn ngẫu nhiên một nút từ partial view hiện tại. Đây là nút mà giao thức lan truyền sẽ trao đổi thông tin cùng.
- **selectToSend:** Chọn một số nút khác từ partial view. Thêm các nút này vào danh sách sẽ được gửi đến nút được chọn từ selectPeer.
- **selectToKeep:** Nhận danh sách các nút từ một nút khác. Thêm các nút mới vào partial view. Loại bỏ các nút bị trùng lặp và cắt danh sách về kích thước tối đa c.

5.2) Dịch vụ lấy mẫu ngang hàng (Peer-Sampling Service)

(a) Active Thread

selectPeer(&Q);

- Chọn một nút ngẫu nhiên Q từ partial view.

selectToSend(&bufs);

- Chọn các nút từ partial view để gửi đi.

sendTo(Q, bufs);

- Gửi danh sách đã chọn đến nút Q.

receiveFrom(Q, &bufr);

- Nhận danh sách partial view từ Q.

```
selectPeer(&Q);
selectToSend(&bufs);
sendTo(Q, bufs);

receiveFrom(Q, &bufr);
selectToKeep(p_view, bufr);
```

(a)

```
                                  receiveFromAny(&P, &bufr);
                                  selectToSend(&bufs);
                                  sendTo(P, bufs);
                                  selectToKeep(p_view, bufr);
```

(b)

Hình 5.25: Giao tiếp giữa (a) Luồng chủ động và (b) luồng thụ động trong dịch vụ lấy mẫu ngang hàng

selectToKeep(p_view,bufr);

- Cập nhật partial view bằng dữ liệu nhận được từ Q.

5.2) Dịch vụ lấy mẫu ngang hàng (Peer-Sampling Service)

(b) Passive Thread

receiveFromAny(&P, &bufr);

- Nhận dữ liệu từ một nút P.

selectToSend(&bufs);

- Chọn danh sách partial view để gửi lại cho P.

sendTo(P, bufs);

- Gửi danh sách đến P.

selectToKeep(p_view, bufr);

- Cập nhật partial view bằng dữ liệu từ P.

```
selectPeer(&Q);
selectToSend(&bufs);
sendTo(Q, bufs);

receiveFrom(Q, &bufr);
selectToKeep(p_view, bufr);
```

(a)

```
receiveFromAny(&P, &bufr);
selectToSend(&bufs);
sendTo(P, bufs);
selectToKeep(p_view, bufr);
```

(b)

Hình 5.25: Giao tiếp giữa (a) Luồng chủ động và (b) luồng thụ động trong dịch vụ lấy mẫu ngang hàng

5.2) Dịch vụ lấy mẫu ngang hàng (Peer-Sampling Service)

Luồng chủ động (Active Thread) sẽ chủ động liên lạc với một nút khác trong mạng.

- Nó chọn một nút từ danh sách partial view hiện tại.
- Tạo một danh sách gồm **c/2 + 1 entries**, bao gồm chính nó và một số nút khác từ partial view.
- Gửi danh sách này đến nút được chọn.
- Sau đó, chờ phản hồi từ nút đó.

Luồng bị động (Passive Thread) ở nút nhận cũng tạo một danh sách tương tự (giống như active thread) và gửi lại phản hồi.

5.2) Dịch vụ lấy mẫu ngang hàng (Peer-Sampling Service)

Điểm quan trọng nhất là cách xây dựng partial view mới.

Sau khi hai nút đã trao đổi danh sách, mỗi nút cần cập nhật lại partial view của mình, đảm bảo nó vẫn có đúng **c entries**. Có hai cách tiếp cận:

1. **Hoán đổi một phần danh sách**

- Mỗi nút có thể loại bỏ các entries mà nó đã gửi đi.
- Điều này có nghĩa là hai nút sẽ hoán đổi một phần danh sách của nhau, giúp thông tin lan truyền hiệu quả hơn.

2. **Loại bỏ các entries cũ nhất**

- Mỗi entry trong partial view có một age (tuổi).
- Sau mỗi lần gossip, tuổi của các entries sẽ tăng lên một đơn vị.
- Các nút sẽ loại bỏ các entries có tuổi lớn nhất, ưu tiên giữ lại thông tin mới.

5.2) Dịch vụ lấy mẫu ngang hàng (Peer-Sampling Service)

- Nếu các **nút (peers) chạy thuật toán trao đổi danh sách (exchange algorithm) một cách đều đặn**,
- Và chọn một **nút ngẫu nhiên** từ **partial view** đã được cập nhật liên tục,
- Thì việc chọn nút này gần như không thể phân biệt với việc chọn ngẫu nhiên từ toàn bộ mạng.

=> Mạng có thể đạt được một cơ chế chọn nút ngẫu nhiên **mà không cần biết toàn bộ topology**.

Một trong những triển khai phổ biến của cơ chế peer-sampling này là **Cyclon *(Voulgaris et al., 2005)***, một giao thức dựa trên gossip để duy trì và cập nhật danh sách partial view một cách hiệu quả.

5.3) Xây dựng overlay bằng gossip

Mạng P2P có thể là **có cấu trúc (structured)** hoặc **không có cấu trúc (unstructured)**, nhưng hai loại này **không nhất thiết phải tách biệt hoàn toàn** *(Castro et al., 2005).*

Nếu các nút **cẩn thận lựa chọn và trao đổi entries trong partial view**, ta có thể **xây dựng và duy trì một topology cụ thể cho mạng overlay**.

Cách thực hiện:

- Sử dụng một **mô hình hai lớp (two-layered approach)** (Hình 5.26).
- Một lớp sẽ **quản lý việc duy trì kết nối ngẫu nhiên** (giống như peer sampling service).
- Lớp còn lại sẽ **định hình topology mong muốn** bằng cách chọn lựa các nút theo tiêu chí nhất định.

5.3) Xây dựng overlay bằng gossip

1. Random Overlay - Mạng ngẫu nhiên

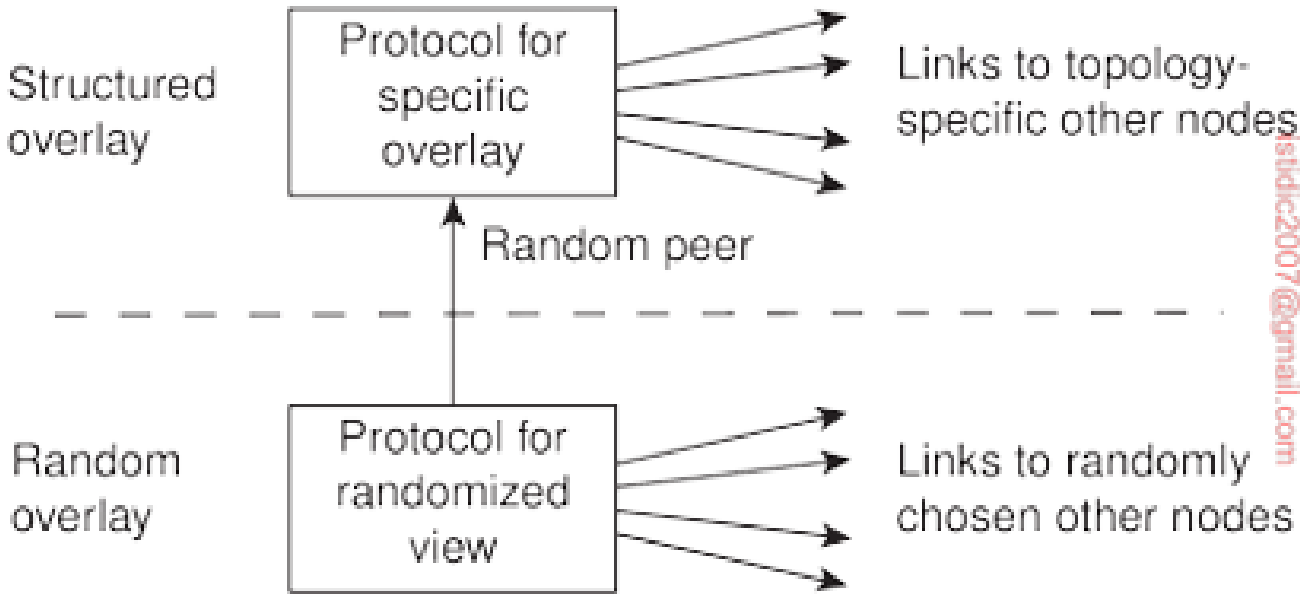
Protocol for randomized view: Giao thức duy trì một tập hợp các kết nối ngẫu nhiên giữa các nút.

Mục đích: Cung cấp một peer-sampling service – tức là chọn ngẫu nhiên một nút từ toàn bộ mạng mà không cần lưu toàn bộ danh sách các nút.

2. Structured Overlay - Mạng có cấu trúc

Protocol for specific overlay: Giao thức điều chỉnh tập hợp các kết nối theo một topology cụ thể (VD: vòng, cây, mesh, DHT...).

Mục đích: Định hình overlay network sao cho phù hợp với yêu cầu của ứng dụng (chẳng hạn như tăng tốc độ truy vấn, tối ưu truyền tải dữ liệu).



Hình 5.26: Cách tiếp cận hai lớp giao thức để xây dựng và duy trì một overlay topology cụ thể từ một mạng P2P không có cấu trúc.

Gossip-based coordination

Distributed event matching

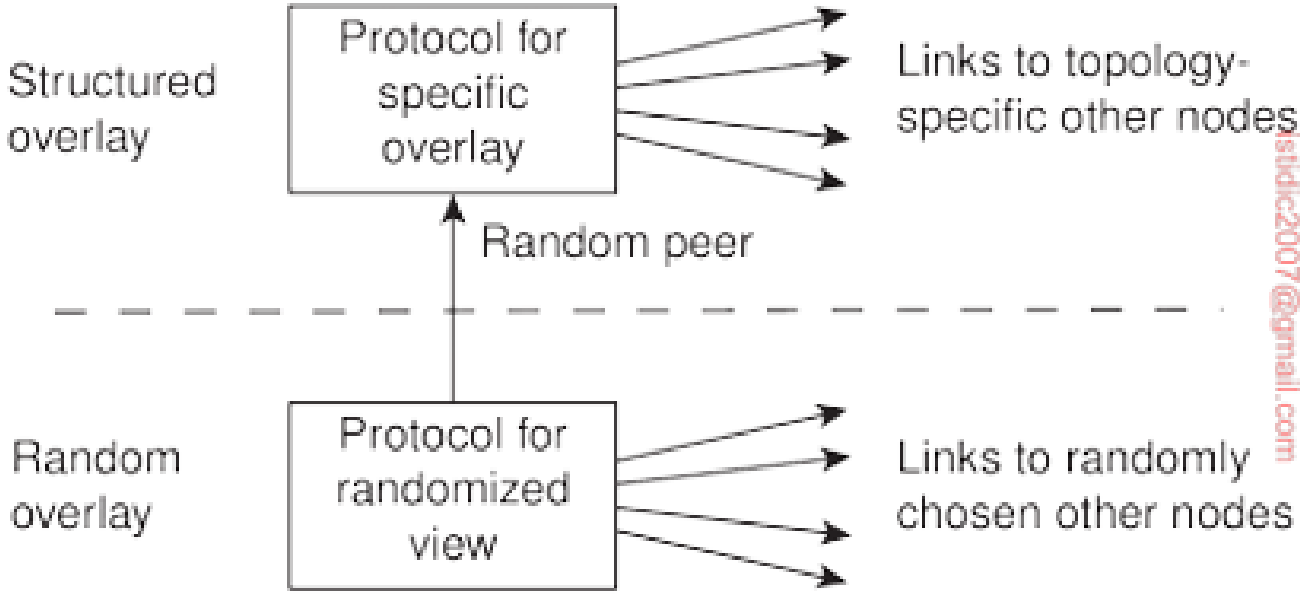
Location systems

5.3) Xây dựng overlay bằng gossip

Cách hoạt động:

- Lớp dưới chọn các nút ngẫu nhiên và truyền danh sách này lên lớp trên.
- Lớp trên sử dụng một tiêu chí lọc (ranking function) để chọn ra danh sách phù hợp với topology mong muốn.

Ví dụ về ranking function: Xếp hạng các nút theo khoảng cách đến một nút P cụ thể.



Hình 5.26: Cách tiếp cận hai lớp giao thức để xây dựng và duy trì một overlay topology cụ thể từ một mạng P2P không có cấu trúc.

- Nếu áp dụng, danh sách dần dần sẽ bao gồm các nút gần nhất với P thay vì các nút ngẫu nhiên hoàn toàn.

Gossip-based coordination

Distributed event matching

Location systems

5.3) Xây dựng overlay bằng gossip

Khái niệm về độ chính xác (Accuracy)

- **Độ chính xác trong peer-sampling service** có nghĩa là **partial view** của mỗi nút phải **chứa các entries đại diện cho các nút ngẫu nhiên trong mạng**.
- Nếu độ chính xác cao, bất kỳ nút nào được chọn từ partial view đều gần như là một nút ngẫu nhiên thực sự trong hệ thống.
- Nếu độ chính xác thấp, partial view có thể bị lệch về một số khu vực nhất định của mạng, làm giảm hiệu quả của việc chọn ngẫu nhiên.

5.3) Xây dựng overlay bằng gossip

Minh họa về cách mạng overlay phát triển thành torus

Giả sử ta có một lưới logic kích thước $N \times N$, trong đó mỗi nút được đặt tại một điểm trên lưới.

Mỗi nút cần duy trì danh sách c nút gần nhất, dựa trên khoảng cách:

$$d = d_1 + d_2, \qquad với\ d_i = min(N - |a_i - b_i|, |a_i - b_i|)$$

Công thức này giúp tạo **hiệu ứng "torus",** nghĩa là biên của lưới nối liền với biên đối diện, giống như bản đồ của một trò chơi không có giới hạn rìa.

5.3) Xây dựng overlay bằng gossip

Minh họa về cách mạng overlay phát triển thành torus

Quá trình hình thành topology torus

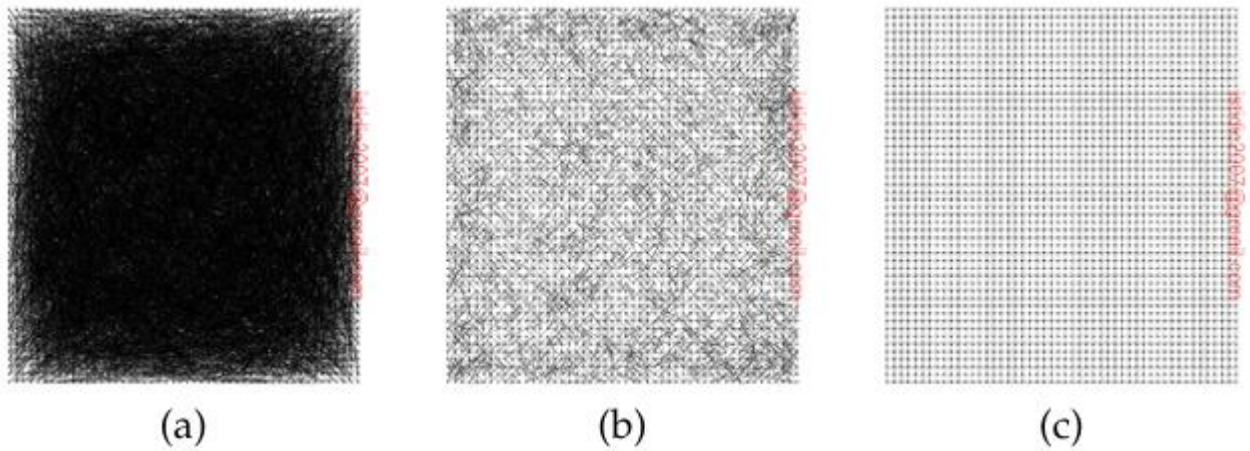
1. Ban đầu (vòng 0)

- Mỗi nút khởi tạo danh sách ngẫu nhiên gồm c nút khác. Mạng vẫn là một tập hợp liên kết hỗn loạn.

2. Sau 5 vòng cập nhật (hình 5.27b)

- Các nút dần loại bỏ liên kết xa, thay bằng các nút gần hơn (theo tiêu chí khoảng cách d). Cấu trúc dần hiện rõ hơn.

3. Sau 20 vòng cập nhật (hình 5.27c)



Hình 5.27: Tạo một mạng overlay dạng torus bằng cách sử dụng hệ thống peer-to-peer không cấu trúc hai lớp.

- Mạng hoàn toàn ổn định thành dạng torus 2D. Mỗi nút chỉ kết nối với các nút gần nhất theo khoảng cách d.

5.3) Xây dựng overlay bằng gossip

Ghi chú 5.8 (Nâng cao: Xây dựng overlay dựa trên gossip trong Python)

Nguyên tắc: chúng ta có hai lớp riêng biệt và mỗi nút chịu trách nhiệm trao đổi liên kết với các nút láng giềng đã chọn.

Một cách triển khai đơn giản có vẻ là sử dụng một luồng riêng cho mỗi lớp, như gợi ý trong Hình 5.26. Tuy nhiên, lựa chọn này có thể dẫn đến vấn đề deadlock ngay lập tức.

- Giả sử nút A quyết định trao đổi liên kết với nút B, nhưng B lại chọn một nút khác, chẳng hạn như C, để trao đổi liên kết.
- Nếu C chọn A để trao đổi liên kết, có thể xảy ra deadlock nếu cả ba nút đầu tiên cùng chọn láng giềng của mình để trao đổi liên kết, rồi sau đó đồng thời chờ phản hồi từ láng giềng đó.

5.3) Xây dựng overlay bằng gossip

Ghi chú 5.8 (Nâng cao: Xây dựng overlay dựa trên gossip trong Python)

Giải pháp chung: tránh hành vi yêu cầu-phản hồi đồng bộ, thay vào đó sử dụng giao tiếp bất đồng bộ và liên tục.

Nói cách khác, middleware sẽ lưu trữ một thông điệp trước khi nó được gửi đi, cho phép bên gửi tiếp tục thực hiện các công việc khác.

Loại giao tiếp này được hỗ trợ chính xác bởi gói **channel**, vốn được sử dụng trong nhiều ví dụ Python của chúng ta.

5.3) Xây dựng overlay bằng gossip

Ghi chú 5.8 (Nâng cao: Xây dựng overlay dựa trên gossip trong Python)

Lưu ý: với mỗi tầng, cách tiếp cận về cơ bản luôn giống nhau:

- Chọn một peer.
- Quyết định các liên kết cần trao đổi.
- Gửi thông tin đó một cách bất đồng bộ đến peer được chọn.

Sau đó, một node khởi tạo trao đổi sẽ phải chờ phản hồi. Khi nhận được phản hồi, nó chỉ cần cập nhật lại danh sách của mình.

Về nguyên tắc, sẽ có một lời gọi thường xuyên đến **maintainViews()**, ít nhất để đảm bảo rằng một node cũng phản hồi các yêu cầu trao đổi đến từ những node khác.

Clock synchronization
Logical clocks
Mutual exclusion
Election algorithm
Gossip-based coordination
Distributed event matching
Location systems

5.3) Xây dựng overlay bằng gossip

Ghi chú 5.8 (Nâng cao: Xây dựng overlay dựa trên gossip trong Python)

Lưu ý: nếu một node đang thực hiện một trao đổi mà nó đã khởi tạo, nó sẽ bỏ qua bất kỳ yêu cầu trao đổi đến nào khác. Điều này được thực hiện bằng cách yêu cầu node khởi tạo từ bỏ yêu cầu của chính nó.

Mặc dù cách tiếp cận này có thể ngăn chặn deadlock, nhưng nó cũng mang rủi ro là không có trao đổi nào diễn ra.

5.3) Xây dựng overlay bằng gossip

Ghi chú 5.8 (Nâng cao: Xây dựng overlay dựa trên gossip trong Python)

Ở tầng trên cùng, khi xây dựng overlay, các liên kết được trao đổi nên **có ý nghĩa nhất đối với peer nhận**.

Cụ thể, trong ví dụ này:

- Khi một **nút A gửi các liên kết đến nút B**, nó nên chọn những liên kết chứa các **hàng xóm gần B nhất**.
- Ngoài ra, **B không nên trả lại các liên kết mà A vừa gửi**. Thay vào đó, tốt hơn là **A nên học về các nút mới trước tiên**.

5.3) Xây dựng overlay bằng gossip

Ghi chú 5.8 (Nâng cao: Xây dựng overlay dựa trên gossip trong Python)

maintainViews(): duy trì các view.

(overlay view/peer sampling service view).

- Nếu đến thời điểm cập nhật một view, nó **chọn một peer**, **chọn danh sách liên kết (links)**, và **gửi chúng đi**.

```
def maintainViews():
  for viewType in [viewOverlay, viewPSS]: # For each view, do the same
    peer[viewType] = None
    if time to maintain viewType: # This viewType needs to be updated
      peer[viewType] = selectPeer(viewType)            # Select a peer
      links = selectLinks(viewType, peer[viewType])    # Select links
      sendTo(peer[viewType], Request[viewType], links) # Send links asynchronously

  while True:
    block = (peer[viewOverlay] != None) or (peer[viewPSS] != None)
    sender, msgType, msgData = recvFromAny(block) # Block if expecting something

    if msg == None: # All work has been done, simply return from the call
      return

    for viewType in [viewOverlay, viewPSS]: # For each view, do the same
      if msgType == Response[viewType]:  # Response to previously sent links
        updateOwnView(viewType, msgData) # Just update the own view

      elif msgType == Request[viewType]: # Request for exchanging links
        if peer[viewType] == None:       # No outstanding exchange request
          links = selectLinks(viewType, sender)     # Select links
          sendTo(sender, Response[viewType], links) # Send them asynchronously
          updateOwnView(viewType,msgData)           # Update own view
        else: # This node already has a pending exchange request, ignore this one
          sendTo(sender, IgnoreRequest[viewType])

      elif msgType == IgnoreRequest[viewType]: # Request has been denied, give up
        peer[viewType] = None
```

Figure 5.28: Pseudocode in Python for implementing a gossip-based approach toward overlay construction.

5.3) Xây dựng overlay bằng gossip

Ghi chú 5.8 (Nâng cao: Xây dựng overlay dựa trên gossip trong Python)

Vòng lặp xử lý tin nhắn:

- Chương trình chạy vô hạn và lắng nghe tin nhắn từ các peer khác.
- block là một giá trị Boolean, xác định xem có đang đợi phản hồi từ peer nào không.
- recvFromAny(block) sẽ **chặn (block)** nếu cần phải đợi tin nhắn đến từ một peer.

```
def maintainViews():
  for viewType in [viewOverlay, viewPSS]: # For each view, do the same
    peer[viewType] = None
    if time to maintain viewType: # This viewType needs to be updated
      peer[viewType] = selectPeer(viewType)            # Select a peer
      links = selectLinks(viewType, peer[viewType])    # Select links
      sendTo(peer[viewType], Request[viewType], links) # Send links asynchronously

  while True:
    block = (peer[viewOverlay] != None) or (peer[viewPSS] != None)
    sender, msgType, msgData = recvFromAny(block) # Block if expecting something

    if msg == None: # All work has been done, simply return from the call
      return

    for viewType in [viewOverlay, viewPSS]: # For each view, do the same
      if msgType == Response[viewType]:  # Response to previously sent links
        updateOwnView(viewType, msgData) # Just update the own view

      elif msgType == Request[viewType]: # Request for exchanging links
        if peer[viewType] == None:       # No outstanding exchange request
          links = selectLinks(viewType, sender)     # Select links
          sendTo(sender, Response[viewType], links) # Send them asynchronously
          updateOwnView(viewType,msgData)           # Update own view
        else: # This node already has a pending exchange request, ignore this one
          sendTo(sender, IgnoreRequest[viewType])

      elif msgType == IgnoreRequest[viewType]: # Request has been denied, give up
        peer[viewType] = None
```

Figure 5.28: Pseudocode in Python for implementing a gossip-based approach toward overlay construction.

5.3) Xây dựng overlay bằng gossip

Ghi chú 5.8 (Nâng cao: Xây dựng overlay dựa trên gossip trong Python)

Nếu không nhận được tin nhắn nào (msg == None), dừng hàm.

```
def maintainViews():
  for viewType in [viewOverlay, viewPSS]: # For each view, do the same
    peer[viewType] = None
    if time to maintain viewType: # This viewType needs to be updated
      peer[viewType] = selectPeer(viewType)            # Select a peer
      links = selectLinks(viewType, peer[viewType])    # Select links
      sendTo(peer[viewType], Request[viewType], links) # Send links asynchronously

  while True:
    block = (peer[viewOverlay] != None) or (peer[viewPSS] != None)
    sender, msgType, msgData = recvFromAny(block) # Block if expecting something

    if msg == None: # All work has been done, simply return from the call
      return

    for viewType in [viewOverlay, viewPSS]: # For each view, do the same
      if msgType == Response[viewType]:  # Response to previously sent links
        updateOwnView(viewType, msgData) # Just update the own view

      elif msgType == Request[viewType]: # Request for exchanging links
        if peer[viewType] == None:       # No outstanding exchange request
          links = selectLinks(viewType, sender)     # Select links
          sendTo(sender, Response[viewType], links) # Send them asynchronously
          updateOwnView(viewType,msgData)           # Update own view
        else: # This node already has a pending exchange request, ignore this one
          sendTo(sender, IgnoreRequest[viewType])

      elif msgType == IgnoreRequest[viewType]: # Request has been denied, give up
        peer[viewType] = None
```

Figure 5.28: Pseudocode in Python for implementing a gossip-based approach toward overlay construction.

5.3) Xây dựng overlay bằng gossip

Ghi chú 5.8 (Nâng cao: Xây dựng overlay dựa trên gossip trong Python)

Xử lý phản hồi (Response):

- Nếu nhận được **phản hồi từ một peer khác, thì cập nhật view của chính nó**.

```
def maintainViews():
  for viewType in [viewOverlay, viewPSS]: # For each view, do the same
    peer[viewType] = None
    if time to maintain viewType: # This viewType needs to be updated
      peer[viewType] = selectPeer(viewType)            # Select a peer
      links = selectLinks(viewType, peer[viewType])    # Select links
      sendTo(peer[viewType], Request[viewType], links) # Send links asynchronously

  while True:
    block = (peer[viewOverlay] != None) or (peer[viewPSS] != None)
    sender, msgType, msgData = recvFromAny(block) # Block if expecting something

    if msg == None: # All work has been done, simply return from the call
      return

    for viewType in [viewOverlay, viewPSS]: # For each view, do the same
      if msgType == Response[viewType]:  # Response to previously sent links
        updateOwnView(viewType, msgData) # Just update the own view

      elif msgType == Request[viewType]: # Request for exchanging links
        if peer[viewType] == None:       # No outstanding exchange request
          links = selectLinks(viewType, sender)      # Select links
          sendTo(sender, Response[viewType], links) # Send them asynchronously
          updateOwnView(viewType,msgData)           # Update own view
        else: # This node already has a pending exchange request, ignore this one
          sendTo(sender, IgnoreRequest[viewType])

      elif msgType == IgnoreRequest[viewType]: # Request has been denied, give up
        peer[viewType] = None
```

Figure 5.28: Pseudocode in Python for implementing a gossip-based approach toward overlay construction.

5.3) Xây dựng overlay bằng gossip

Ghi chú 5.8 (Nâng cao: Xây dựng overlay dựa trên gossip trong Python)

Xử lý yêu cầu trao đổi (Request):

- Khi nhận được yêu cầu (Request), kiểm tra xem có yêu cầu nào đang chờ xử lý không.
- Nếu không có yêu cầu đang chờ, chọn các liên kết và gửi phản hồi.
- Nếu đã có một yêu cầu đang chờ, thì từ chối yêu cầu mới (IgnoreRequest).

```
def maintainViews():
  for viewType in [viewOverlay, viewPSS]: # For each view, do the same
    peer[viewType] = None
    if time to maintain viewType: # This viewType needs to be updated
      peer[viewType] = selectPeer(viewType)            # Select a peer
      links = selectLinks(viewType, peer[viewType])    # Select links
      sendTo(peer[viewType], Request[viewType], links) # Send links asynchronously

  while True:
    block = (peer[viewOverlay] != None) or (peer[viewPSS] != None)
    sender, msgType, msgData = recvFromAny(block) # Block if expecting something

    if msg == None: # All work has been done, simply return from the call
      return

    for viewType in [viewOverlay, viewPSS]: # For each view, do the same
      if msgType == Response[viewType]:  # Response to previously sent links
        updateOwnView(viewType, msgData) # Just update the own view

      elif msgType == Request[viewType]: # Request for exchanging links
        if peer[viewType] == None:       # No outstanding exchange request
          links = selectLinks(viewType, sender)      # Select links
          sendTo(sender, Response[viewType], links) # Send them asynchronously
          updateOwnView(viewType,msgData)           # Update own view
        else: # This node already has a pending exchange request, ignore this one
          sendTo(sender, IgnoreRequest[viewType])

      elif msgType == IgnoreRequest[viewType]: # Request has been denied, give up
        peer[viewType] = None
```

Figure 5.28: Pseudocode in Python for implementing a gossip-based approach toward overlay construction.

5.3) Xây dựng overlay bằng gossip

Ghi chú 5.8 (Nâng cao: Xây dựng overlay dựa trên gossip trong Python)

Xử lý yêu cầu bị từ chối (IgnoreRequest):

- Nếu nhận được phản hồi IgnoreRequest, nghĩa là yêu cầu bị từ chối

→ **đặt lại peer[viewType] = None.**

```
def maintainViews():
  for viewType in [viewOverlay, viewPSS]: # For each view, do the same
    peer[viewType] = None
    if time to maintain viewType: # This viewType needs to be updated
      peer[viewType] = selectPeer(viewType)            # Select a peer
      links = selectLinks(viewType, peer[viewType])    # Select links
      sendTo(peer[viewType], Request[viewType], links) # Send links asynchronously

  while True:
    block = (peer[viewOverlay] != None) or (peer[viewPSS] != None)
    sender, msgType, msgData = recvFromAny(block) # Block if expecting something

    if msg == None: # All work has been done, simply return from the call
      return

    for viewType in [viewOverlay, viewPSS]: # For each view, do the same
      if msgType == Response[viewType]:  # Response to previously sent links
        updateOwnView(viewType, msgData) # Just update the own view

      elif msgType == Request[viewType]: # Request for exchanging links
        if peer[viewType] == None:       # No outstanding exchange request
          links = selectLinks(viewType, sender)      # Select links
          sendTo(sender, Response[viewType], links) # Send them asynchronously
          updateOwnView(viewType,msgData)           # Update own view
        else: # This node already has a pending exchange request, ignore this one
          sendTo(sender, IgnoreRequest[viewType])

      elif msgType == IgnoreRequest[viewType]: # Request has been denied, give up
        peer[viewType] = None
```

Figure 5.28: Pseudocode in Python for implementing a gossip-based approach toward overlay construction.

5.3) Xây dựng overlay bằng gossip

Ghi chú 5.8 (Nâng cao: Xây dựng overlay dựa trên gossip trong Python)

Hơn nữa, **khi A chọn một peer**, nó nên **tránh chọn một peer mà nó vừa trao đổi gần đây**.

Nói cách khác, **A nên tối đa hóa tính đa dạng khi chọn peer.**

- Cách để đạt được điều này là chọn peer **theo vòng tuần hoàn (round-robin).**
- Cách tiếp cận này có lợi thế trong **mạng động**, vì **các nút đã rời khỏi hệ thống sẽ nhanh chóng bị phát hiện** và có thể bị loại bỏ khỏi danh sách.

Clock synchronization
Logical clocks
Mutual exclusion
Election algorithm
Gossip-based coordination
Distributed event matching
Location systems

5.3) Xây dựng overlay bằng gossip

Ghi chú 5.8 (Nâng cao: Xây dựng overlay dựa trên gossip trong Python)

Nếu chỉ sử dụng lớp trên cùng, việc tham lam trong trao đổi chỉ các liên kết tốt nhất có thể dẫn đến tình huống:

- Một nhóm các nút có thể tạo thành một tập khép kín trong mạng.
- Các nút này chỉ tham chiếu lẫn nhau, khiến chúng bỏ lỡ các nút tốt hơn bên ngoài.

Lý do tại sao cần phải có yếu tố ngẫu nhiên:

- Lớp phủ ngẫu nhiên ở tầng dưới giúp đảm bảo rằng mạng không bị kẹt trong cực tiểu cục bộ, giống như trong thuật toán leo đồi (hill-climbing).
- Thêm một chút ngẫu nhiên giúp hướng đến tối ưu toàn cục thay vì bị mắc kẹt trong một cấu trúc hạn chế.

5.4) Gossip bảo mật (Secure Gossiping)

Gossiping là một phương pháp hiệu quả để **đồng bộ và phối hợp** giữa các nút trong mạng. Tuy nhiên, tốc độ lan truyền thông tin cũng là một điểm yếu lớn, vì **tin giả hoặc dữ liệu độc hại có thể lây lan nhanh chóng.**

Ví dụ về tấn công trong Peer-Sampling:

- Giả sử một dịch vụ chọn mẫu ngang hàng (peer-sampling service) đang chạy trên một mạng lớn.
- Mỗi nút duy trì một partial view có kích thước c.
- Trung bình, mỗi nút trao đổi $c/2$ tham chiếu đến các nút lân cận.

5.4) Gossip bảo mật (Secure Gossiping)

Ví dụ về tấn công trong Peer-Sampling:

Kịch bản tấn công:

- Có c kẻ tấn công hợp tác với nhau.
- Mỗi kẻ tấn công cũng duy trì một partial view nhưng chỉ gửi tham chiếu đến các kẻ tấn công khác.
- Kết quả: Dần dần, các partial view của các nút hợp lệ bị nhiễm bẩn, chỉ chứa tham chiếu đến những kẻ tấn công.
- Cuối cùng, toàn bộ mạng bị kiểm soát bởi kẻ tấn công, làm mất tính ngẫu nhiên của mạng ngang hàng.

Clock synchronization
Logical clocks
Mutual exclusion
Election algorithm
Gossip-based coordination
Distributed event matching
Location systems

5.4) Gossip bảo mật (Secure Gossiping)

Ví dụ về tấn công trong Peer-Sampling:

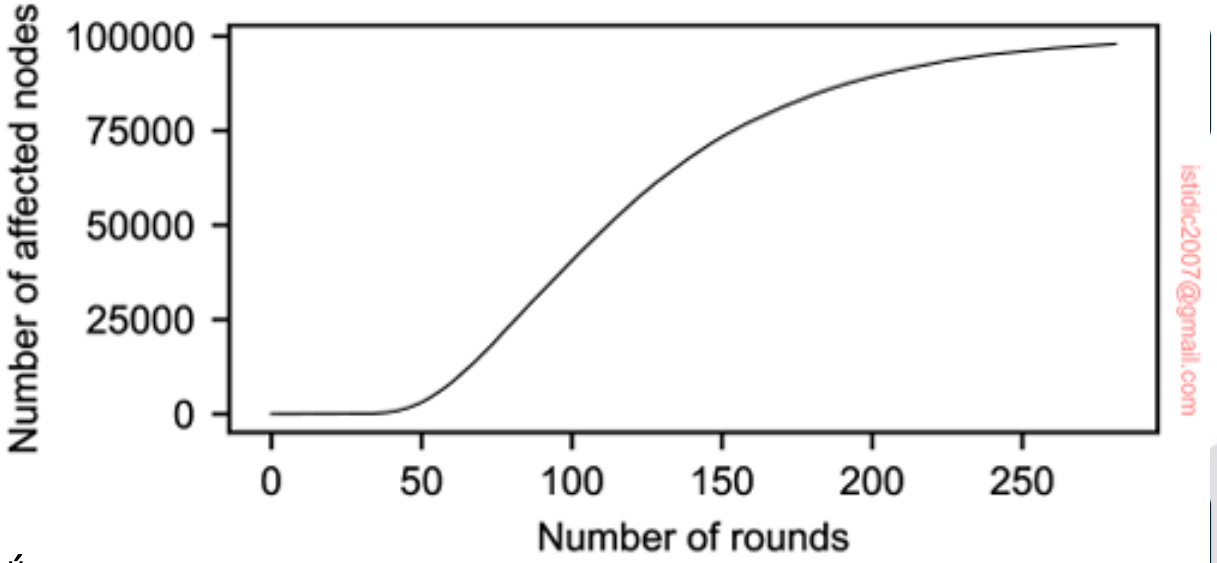
Quy mô mạng: 100,000 nút

Kích thước partial view: 30

Số lượng kẻ tấn công: 30

Tốc độ lan truyền:

- Ban đầu: ảnh hưởng của tấn công không đáng kể.
- Sau khoảng 50 rounds: số lượng nút bị ảnh hưởng bắt đầu tăng nhanh.
- Gần 300 rounds, tất cả các nút hợp lệ đều bị ô nhiễm, tức là partial view của chúng chỉ chứa các kẻ tấn công.
- Đây là một hiệu ứng lan truyền theo cấp số nhân, phù hợp với cơ chế push-pull của giao thức dịch bệnh (epidemic protocols).



Hình 5.29: Minh họa tốc độ lan truyền của một **hub attack** trong mạng gossip-based.

algorithm

Gossip-based coordination

Distributed event matching

Location systems

5.4) Gossip bảo mật (Secure Gossiping)

Một giải pháp trong trường hợp lấy mẫu ngang hàng, được phát triển bởi *Jesi et al. [2010].*

- Bản chất của giải pháp là phát hiện xem có nút nào có hành vi xấu hay không.
- Cuộc tấn công mà chúng ta vừa mô tả có đặc điểm, theo thời gian, nhiều nút sẽ tham chiếu đến một tập hợp rất nhỏ các nút.
- Xét dưới góc độ đồ thị, điều này có nghĩa là bậc vào (indegree) của một số nút sẽ cực kỳ cao, trong khi đối với hầu hết các nút khác, bậc vào sẽ thấp, thậm chí bằng 0.
- Một bậc vào bằng 0 có nghĩa là một nút không được bất kỳ nút nào khác trong mạng biết đến.

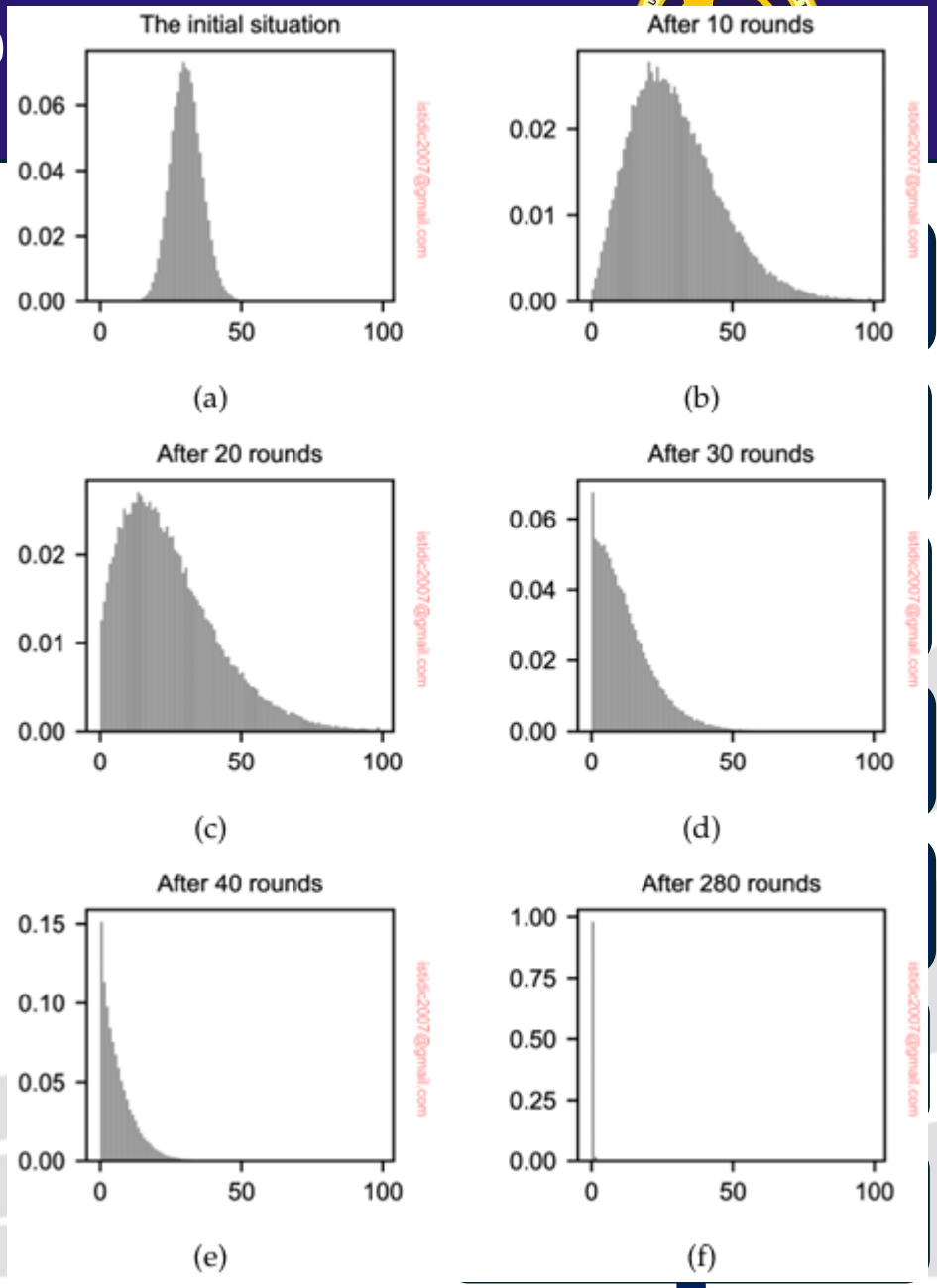
5.4) Gossip bảo mật (Secure Gossiping)

Khi sử dụng Cyclon [Voulgaris et al., 2005] làm dịch vụ lấy mẫu peer, chúng ta biết rằng bậc vào tuân theo phân phối chuẩn (Hình 5.30(a)).

Khi có 30 kẻ tấn công hợp tác:

- Sau 10 vòng: phân phối bậc vào bắt đầu rộng hơn và dịch sang trái.
- Sau 40 vòng: khoảng 15% số nút có bậc vào bằng 0.
- Sau dưới 300 vòng: hầu như tất cả các nút có bậc vào bằng 0, kẻ tấn công kiểm soát toàn bộ mạng.

(Lưu ý rằng trong Hình 5.29, chúng ta đang đếm số lượng nút chỉ tham chiếu đến các nút tấn công, điều này khác với việc đo lường bậc vào. Một nút bị tấn công vẫn có thể được tham chiếu bởi các nút khác.)



5.4) Gossip bảo mật (Secure Gossiping)

Như Jesi et al. [2010] đã giải thích, **mỗi nút có thể thu thập thống kê về những nút nào đang được tham chiếu**. Nếu một số nút **dần dần được tham chiếu nhiều hơn** một cách bất thường, thì có thể có điều gì đó đáng ngờ đang diễn ra. Có một số vấn đề cần giải quyết:

- **Việc lan truyền các liên kết "xấu"** có thể diễn ra quá hiệu quả, đến mức khi **một nút hợp lệ kết luận rằng nó đã xác định được kẻ tấn công**, thì có thể đã quá muộn.
- Chúng ta muốn **tránh việc mỗi nút hợp lệ phải xây dựng hiểu biết đầy đủ về toàn bộ mạng**, đồng thời cũng cần **xác định kẻ tấn công nhanh chóng.**
- Những kẻ tấn công không muốn bị phát hiện, vì chúng cần thời gian để thực hiện thành công chiến lược của mình.

5.4) Gossip bảo mật (Secure Gossiping)

Giải pháp giảm thiểu tấn công trong hệ thống gossiping:

- **Bắt buộc kẻ tấn công tuân theo quy tắc:** Nút benign có thể ép kẻ tấn công trả về liên kết hợp lệ bằng cách thu thập và phân tích các liên kết nhận được.
- **Khởi tạo nhiều lần trao đổi liên kết:** Một nút benign có thể thực hiện nhiều trao đổi đồng thời, nhưng chỉ xử lý một phản hồi để đẩy nhanh quá trình phát hiện.
- **Tiến thoái lưỡng nan cho kẻ tấn công:** Nếu kẻ tấn công luôn trả về liên kết đến đồng phạm, chúng sẽ bị phát hiện nhanh chóng. Để tránh bị lộ, chúng buộc phải trả về liên kết đến các nút benign, làm suy yếu cuộc tấn công.
- **Ẩn danh trong quá trình thu thập dữ liệu:** Kẻ tấn công không thể biết liệu nút nhận có đang thu thập thống kê hay không, buộc chúng phải hành xử đúng mực để tránh bị phát hiện.

[Jesi et al., 2010].

5.4) Gossip bảo mật (Secure Gossiping)

Các kiểu tấn công khác và giải pháp:

- **Tấn công bằng tần suất cao**: Kẻ tấn công có thể khởi tạo trao đổi thông tin với tần suất cao hơn quy định, đẩy nhanh quá trình làm ô nhiễm dữ liệu của các nút benign.
- **Ảnh hưởng của cách tiếp cận push-based**: Khi tấn công sử dụng phương pháp push, các giá trị ô nhiễm có thể nhanh chóng lan rộng trong mạng.
- **Giải pháp đề xuất**: Theo nghiên cứu của *Mousazadeh và Ladani [2015]*, chỉ nên sử dụng cơ chế pull-based, tức là các nút benign chủ động kiểm soát tốc độ trao đổi thông tin, giúp hạn chế sự lan truyền của dữ liệu độc hại.

5.4) Gossip bảo mật (Secure Gossiping)

Tấn công bằng dữ liệu sai lệch và giải pháp:

Tấn công trong quá trình tổng hợp dữ liệu: Kẻ tấn công có thể cố tình trả về giá trị sai lệch. Ví dụ, nếu đang tính trung bình, một nút tấn công A luôn trả về giá trị của nó v_A, khiến các nút khác dần hội tụ về v_A. Mọi nút v_i giao tiếp với A sẽ tính toán:

$$v_i \leftarrow (v_i + v_A)/2$$

Giải pháp:

- Phát hiện các nút không tuân theo giao thức.
- Cơ chế phát hiện phải đủ hiệu quả để buộc kẻ tấn công phải "ẩn mình" bằng cách hành xử đúng quy tắc, giảm thiểu tác động tiêu cực.
- Các biện pháp đối phó với **data-injection attack** được mô tả chi tiết bởi ***Gentz et al. [2016].***

Distributed Systems - Communication

So khớp sự kiện phân tán (Distributed Event Matching)

So khớp sự kiện là cốt lõi của hệ thống publish-subscribe. Nó bao gồm:

- Một tiến trình đăng ký S để chỉ định sự kiện quan tâm.
- Khi một tiến trình phát thông báo N, hệ thống kiểm tra S có khớp với N không.
- Nếu khớp, thông báo sẽ được gửi đến thực thể đăng kí.

Hai nhiệm vụ chính của hệ thống:

1. So khớp đăng ký với events.
2. Thông báo cho thực thể đăng kí khi có kết quả phù hợp.

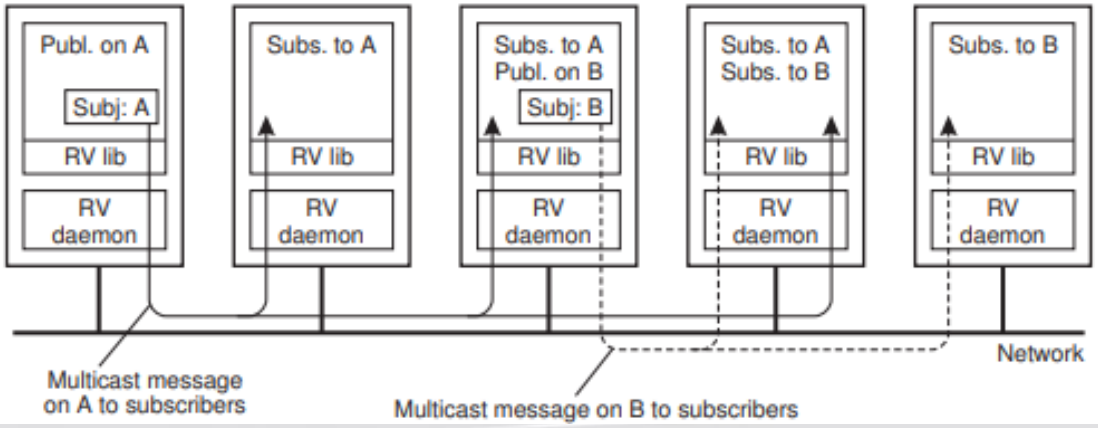
Một vấn đề lớn của publish-subscribe là khả năng mở rộng khi hàm so khớp không thể phân tán hiệu quả. Ngoài ra, còn có thách thức về bảo mật và quyền riêng tư, đảm bảo publisher và thực thể đăng kí không biết về nhau mà vẫn duy trì đủ thông tin để hệ thống hoạt động.

Triển khai tập trung (Centralized Implementations)

- **Cách tiếp cận đơn giản:** Dùng một máy chủ trung tâm lưu các đăng ký(subscription) và kiểm tra mọi thông báo(notification). Cách này dễ triển khai nhưng không mở rộng tốt.
- **Cách mở rộng:**
  - Phân chia công việc giữa nhiều máy chủ bằng hàm ánh xạ đăng ký và thông báo đến các Nút hội tụ(rendezvous nodes).
  - Dùng mạng overlay để phân phối nhiệm vụ, trong đó mỗi broker xử lý một phần matching.

Note 5.9 (Ví dụ: TIB/Rendezvous)

- TIB/Rendezvous dùng multicast để gửi thông báo đến mọi node.
- Mỗi tiến trình gửi đăng ký đến daemon cục bộ, daemon này sẽ lưu đăng ký và lọc thông báo.
- Nếu không có thực thể đăng kí(subscriber), thông báo bị loại bỏ ngay lập tức.

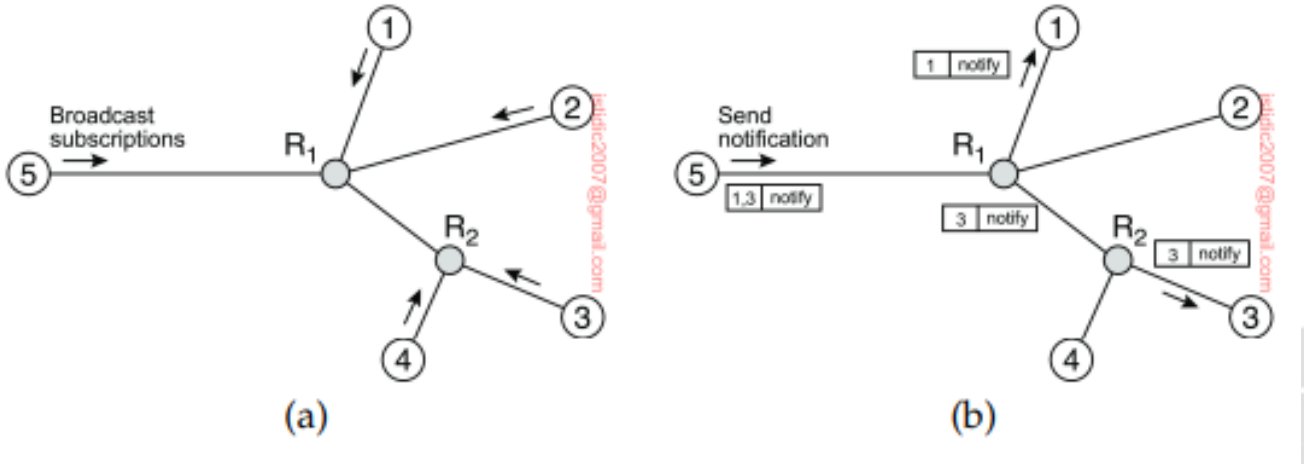


- Khi đăng ký phức tạp hơn (biểu thức điều kiện, phạm vi giá trị), khả năng mở rộng có thể gặp vấn đề.

Định tuyến chọn lọc (Selective Routing)

Thay vì flooding, có thể tối ưu bằng cách **lọc đường đi** của thông báo:

- Mỗi broker phát đăng ký đến các broker khác để tạo bảng định tuyến.
- Khi có thông báo, broker gửi nó đến broker liên quan thay vì gửi đi toàn bộ mạng.
- Các bộ lọc routing giúp tối ưu bằng cách lưu thông tin về phạm vi quan tâm của từng node.



Interface	Filter
To node 3	$a \in [0,3]$
To node 4	$a \in [2,5]$
Toward router R_1	(unspecified)

Figure 5.33: A partially filled routing table.

Lan truyền tin đồn (Gossiping)

Khi định tuyến chọn lọc chưa đủ mở rộng, ta có thể dùng lan truyền tin đồn để xây dựng mạng overlay động:

- Các thực thể đăng kí có cùng sở thích tạo thành mạng riêng.
- Thông báo chỉ cần được gửi đến overlay phù hợp.

Phương pháp này được triển khai trong hệ thống TERA và Sub-2-Sub:

- Sub-2-Sub sử dụng không gian đa chiều (hyperspace) để biểu diễn đăng ký và phân chia vùng quan tâm thành tập hợp con nhỏ hơn.
- Các node liên tục trao đổi đăng ký qua gossiping, tìm các node có vùng quan tâm giao nhau để lập nhóm.
- Khi có thông báo, nó được chuyển tiếp theo vòng tròn nhóm phù hợp để tối ưu tốc độ phân tán.

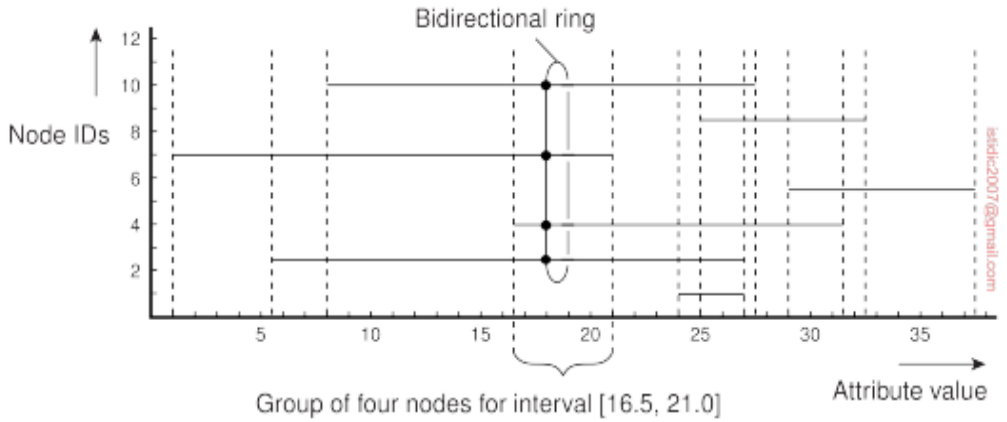


Figure 5.34: Grouping nodes for supporting range queries in a gossip-based publish-subscribe system.

Giải pháp bảo mật cho hệ thống publish-subscribe

Hệ thống **publish-subscribe** có đặc trưng là các publisher và thực thể đăng kí không liên kết trực tiếp (referential decoupling), có thể cũng không liên kết theo thời gian (temporally decoupled). Một vấn đề quan trọng là làm sao bảo mật tin nhắn giữa publisher và thực thể đăng kí mà vẫn đảm bảo tính ẩn danh lẫn nhau.

Giải pháp đơn giản: Dùng một broker (có thể là một hệ thống phân tán) để xử lý tin nhắn. Tuy nhiên, broker cần phải đáng tin cậy, điều này không phải lúc nào cũng đảm bảo, nhất là khi dịch vụ được cung cấp bởi bên thứ ba (ví dụ: dịch vụ đám mây).

Các **thách thức chính**:

1. **Đảm bảo ẩn danh** giữa publisher và thực thể đăng kí.
2. **Bảo toàn tính toàn vẹn**: Làm sao đảm bảo tin nhắn không bị chỉnh sửa khi nguồn gốc bị ẩn?
3. **Bảo mật dữ liệu**: Không thể để broker thấy nội dung tin nhắn, nhưng điều này lại gây khó khăn trong việc định tuyến tin nhắn.
4. **Đảm bảo tính sẵn sàng**: Thực thể đăng kí có quyền nhận tin nhắn cần phải thực sự nhận được.
5. **Chặn truy cập trái phép**: Tin nhắn không được gửi đến thực thể đăng kí không có quyền.
6. **Vấn đề đồng phạm (colluders)**: Các broker không được phép suy ra thông tin về thực thể đăng kí vô tội.

Giải pháp sử dụng mã hóa tìm kiếm (Searchable Encryption):

- Cho phép bên thứ ba xử lý dữ liệu mà không nhìn thấy nội dung thực tế.
- **Kỹ thuật PEKS (Public Key Encryption with Keyword Search)**:
  - Mỗi tin nhắn đi kèm với một tập từ khóa mã hóa.
  - Thực thể đăng kí có thể sử dụng "trapdoor" để kiểm tra xem một tin nhắn có chứa từ khóa mong muốn không mà không tiết lộ thông tin khác.

Vai trò của Key Authority (Tổ chức quản lý khóa):

- Cấp khóa mã hóa công khai (PK) cho publisher để mã hóa tin nhắn và tạo thẻ từ khóa mã hóa.
- Cấp khóa bí mật (SK) cho thực thể đăng kí để kiểm tra từ khóa và giải mã tin nhắn nếu có sự khớp.

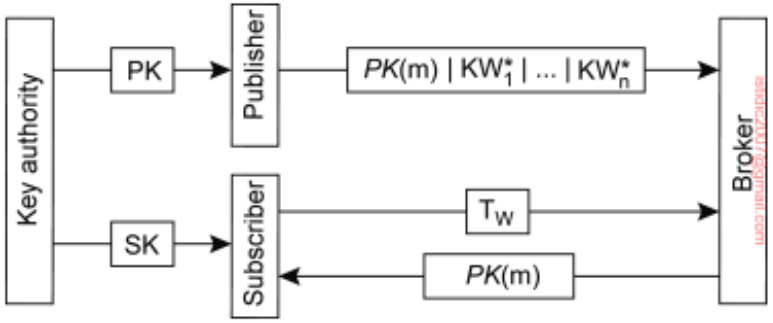


Figure 5.35: The principle of privacy-preserving publish-subscribe.

Distributed Systems - Communication

Hệ thống định vị (Location Systems)

Khi làm việc với hệ thống phân tán lớn trong mạng diện rộng, cần xem xét yếu tố vị trí địa lý để tối ưu hiệu suất. Ví dụ, hai tiến trình có thể là hàng xóm trong mạng phủ (overlay network) nhưng thực tế cách xa nhau, gây ra độ trễ trong giao tiếp. Do đó, cần có các kỹ thuật dựa trên vị trí để điều phối việc đặt tiến trình và liên lạc giữa chúng.

Hệ thống Định vị Toàn cầu (GPS: Global Positioning System)

GPS là một hệ thống phân tán dựa trên vệ tinh, ra mắt năm 1978, ban đầu phục vụ mục đích quân sự nhưng nay được dùng rộng rãi cho dân sự, như định vị giao thông, theo dõi thiết bị, xe cộ, thú cưng, trẻ em...

Clock synchronization
Logical clocks
Mutual exclusion
Election algorithm
Gossip-based coordination
Distributed event matching
Location systems

Cách hoạt động của GPS:

- GPS sử dụng tối đa 72 vệ tinh, quay quanh Trái Đất ở độ cao 20.000 km.
- Mỗi vệ tinh có 4 đồng hồ nguyên tử,

được hiệu chỉnh từ các trạm mặt đất.

- Vệ tinh phát sóng vị trí & thời gian, giúp máy
- Nguyên lý định vị:
  - Xác định khoảng cách đến ít nhất 4 vệ

  Tinh (3D) để tính ra kinh độ, vĩ độ, độ cao

  của người dùng.

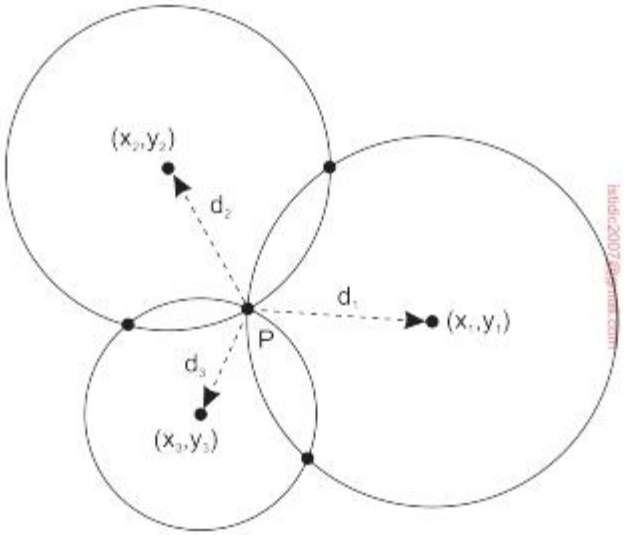


Figure 5.36: Computing a node's position in a two-dimensional space.

Thách thức trong thực tế:

1. Độ trễ truyền dữ liệu: Tín hiệu từ vệ tinh đến máy thu mất thời gian.
2. Lệch đồng hồ: Đồng hồ máy thu thường không đồng bộ với đồng hồ vệ tinh.

Phương trình tính toán khoảng cách thực tế có thêm một thành phần lệch thời gian (Δr), và cần ít nhất 4 vệ tinh để giải hệ phương trình 4 ẩn số ($x, y, z, \Delta r$).

Sai số và cải thiện độ chính xác:

- Lỗi 5–10 mét do:
  - Sai số đồng hồ nguyên tử của vệ tinh.
  - Sai số trong tính toán vị trí vệ tinh.
  - Tín hiệu bị ảnh hưởng bởi tầng điện ly.
- Cải thiện bằng kỹ thuật đặc biệt:
  - GPS vi sai (Differential GPS - DGPS): Cải thiện độ chính xác của GPS bằng cách sử dụng một trạm tham chiếu cố định để hiệu chỉnh tín hiệu GPS
  - Nguồn tham khảo: LaMarca & de Lara (2008), Zogg (2002).

5.7.2 Khi không có GPS

- GPS không hoạt động tốt trong nhà → cần kỹ thuật thay thế, như sử dụng WiFi.
- Dữ liệu vị trí của điểm truy cập WiFi có thể thu thập qua war driving:

  Tính toán vị trí điểm truy cập bằng trung bình cộng tọa độ của các lần đo GPS.
- Thách thức: độ chính xác của GPS, phạm vi truyền không đồng đều, và sự thay đổi của điểm truy cập.

Tính toán vị trí của điểm truy cập WiFi (AP)

$$\vec{x}_{AP} = \frac{\sum_{i=1}^{N} \vec{x}_i}{N}$$

- **Ý nghĩa**: Tọa độ trung bình của các vị trí phát hiện được của AP.
- **Ứng dụng**: Ước lượng vị trí của một điểm truy cập dựa trên dữ liệu GPS.

5.7.3 Định vị logic của các nút (Logical Positioning of Nodes)

- Thay vì xác định vị trí thực, có thể sử dụng vị trí logic dựa trên độ trễ mạng (Network Coordinates System - NCS).
- Ứng dụng:
  - Điều hướng trong CDN (Chọn máy chủ gần nhất).
  - Tối ưu vị trí đặt bản sao dữ liệu.
  - Định tuyến dựa trên vị trí.

Tiêu chí	Centralized Positioning (Định vị tập trung)	Decentralized Positioning (Định vị phi tập trung)
Cách hoạt động	Một nút trung tâm thu thập dữ liệu từ các nút khác để tính toán vị trí.	Mỗi nút tự tính toán vị trí của mình dựa trên thông tin từ các nút lân cận.
Dữ liệu đầu vào	Khoảng cách đến các nút đã biết vị trí (thường là qua độ trễ mạng).	Khoảng cách đến các nút lân cận và điều chỉnh dần vị trí.
Ví dụ công thức	Giải hệ phương trình: $\bar{d}_i = \sqrt{(x_i - x_p)^2 + (y_i - y_P)^2}$	Mô hình lò xo: $F_{ij} = (\bar{d}(P_i, P_j) - \hat{d}(P_i, P_j)) \cdot u(\overrightarrow{x_i} - \overrightarrow{x_j})$
Ưu điểm	- Dữ liệu nhất quán, dễ kiểm soát. - Độ chính xác cao hơn trong môi trường ổn định.	- Không cần máy chủ trung tâm, dễ mở rộng. - Hoạt động tốt trong môi trường động.
Nhược điểm	- Phụ thuộc vào máy chủ trung tâm. - Có thể bị lỗi nếu nút trung tâm bị tấn công hoặc mất kết nối.	- Tốn nhiều tài nguyên tính toán hơn. - Có thể mất nhiều thời gian để đạt trạng thái ổn định.
Ứng dụng thực tế	- Hệ thống định vị IP dựa trên máy chủ trung tâm. - Hệ thống CDN chọn máy chủ tối ưu dựa trên độ trễ.	- Giao thức Vivaldi trong mạng P2P. - Định tuyến trong mạng cảm biến không dây.